

UNIVERSITÀ DEGLI STUDI GUGLIELMO MARCONI

DIPARTIMENTO DI SCIENZE INGEGNERISTICHE

CORSO DI LAUREA MAGISTRALE IN INGEGNERIA INFORMATICA

EMOZIONI E INTELLIGENZA ARTIFICIALE

« UN VIAGGIO NELL'INTERAZIONE UOMO — MACCHINA »

Relatore:

Chiar.ma Prof.ssa FRANCESCA FALLUCCHI

Candidato:

MANUEL GOZZI

Matr. N°: 0020777

ANNO ACCADEMICO

2024/2025

Abstract

Questa tesi esplora l'impatto delle emozioni nell'interazione tra esseri umani e Large Language Models (LLM), analizzando come le stesse influenzano sia le prestazioni dei modelli che la percezione umana. Lo studio si articola in due esperimenti principali: Alpha e Omega. Il primo dei due valuta l'effetto di quattro diverse varianti emotive (gioia, rabbia, tristezza e apatia) su cinque LLM open-weights, utilizzando il benchmark SuperGLUE per misurare la qualità delle risposte. I risultati evidenziano che le varianti emotive di gioia e apatia producono generalmente risultati migliori nei task del benchmark, rispetto alle altre due varianti. L'esperimento Omega, invece, indaga come le varianti emotive di un modello LLM, ottenute tramite fine-tuning di Velvet-14B, influenzano le prestazioni cognitive e la percezione degli studenti in un contesto didattico. Sebbene l'esperimento sociale non sia stato condotto a causa di vincoli infrastrutturali e di costo, il suo design avrebbe permesso di raccogliere dati empirici sulla percezione degli studenti, completando l'analisi iniziata con Alpha. I risultati registrati durante la quantificazione del bias emotivo inserito addestrando le varianti di Velvet-14B suggeriscono che l'approccio adottato è efficace per introdurre nel modello dei bias emotivi. Questo lavoro offre spunti per migliorare l'efficacia dei modelli di Intelligenza Artificiale in ambiti come l'educazione e l'assistenza virtuale, basati puramente sul tuning delle emozioni in luogo di HCI, e contribuisce a una comprensione più profonda delle dinamiche emotive nell'interazione uomo-macchina, fornendo una visione olistica dell'impatto delle emozioni nei sistemi basati su LLM.

1. Introduzione

2. Intelligenza Artificiale ed emozioni: dove siamo

2.1. Da dove origina l'IA 4
2.1.1. Approcci simbolici e connessionisti 7
2.1.2. Metodologie di classificazione dell'IA 9
2.1.3. Che cosa sono e come funzionano i LLM 11
2.1.4. Che cosa sono i RAG 13
2.2. Stato dell'arte in letteratura 15
2.2.1. Dove si inserisce lo studio 16
2.3. Distinguere tra simulazione emotiva e reale comprensione delle emozioni 17
2.3.1. Rappresentare la conoscenza 18
2.4. L'utilizzo di IA in Italia oggi 21
2.4.1. Il ritardo dell'Italia nel panorama IA 22

3. Interazione emotiva uomo-macchina

3.1. Alpha: studio dell'impatto delle emozioni sulla macchina 26
3.1.1. Il benchmark SuperGLUE 27
3.1.2. Ambiente di esecuzione 28
3.1.3. I modelli utilizzati 31
3.2. Omega: studio dell'impatto delle emozioni sull'uomo 32
3.2.1. Fine tuning di Velvet-14B 33
3.2.2. Applicazione web 34

4. Impatto emotivo verso la macchina (esperimento Alpha)

4.1. Campionamento dei dataset 38
4.2. Implementazione dei task 40
4.2.1. CB, RTE e AX-g 41
4.2.2. COPA 42
4.2.3. ReCoRD 44
4.2.4. WiC 46
4.2.5. WSC 47
4.2.6. BoolQ 48
4.3. Meccanismo di checkpointing 49

5. Impatto emotivo verso l'uomo (esperimento Omega)

5.1. Processo di fine-tuning di Velvet-14B 52
5.1.1. Algoritmo di training 54
5.1.2. Esecuzione del training 56
5.2. Sviluppo dell'applicazione web 59
5.2.1. Componente frontend 59
5.2.2. Componente backend 62
5.2.3. Componente RAG 63

5.3. Database vettoriale 67
5.3.1. Algoritmo di chunking 68
5.4. Esperimento sociale 69

6. Risultati impatto emotivo
6.1. Risultati dell'esperimento Alpha 73
6.2. Risultati dell'esperimento Omega 76

7. Conclusioni

Bibliografia e sitografia

Indice

Elenco delle figure

- Figura 1. Linea temporale dell'evoluzione dell'IA.
- Figura 2. Rappresentazione insiemistica della classificazione.
- Figura 3. Gerarchia dell'IA.
- Figura 4. Visualizzazione dei Transformers (fonte poloclub.Github.Com).
- Figura 5. Come funziona Retrieval-Augmented Generation (fonte: AWS — https://aws.amazon.com/it/what-is/retrieval-augmented-generation/).
- Figura 6. Differenza tra RAG e sistemi di Information Retrieval classici.
- Figura 7. Esempio di catena del valore del dato.
- Figura 8. Esempio di flusso di comunicazione uomo-macchina.
- Figura 9. Le prime 10 attività economiche che fanno uso di IA (fonte: ISTAT 2024).
- Figura 10. Ambiti d'investimento nel contesto della transizione digitale (fonte: ISTAT 2024).
- Figura 11. Persone di 16-74 anni divise per livello di competenza digitale (fonte: Istat 2023).
- Figura 12. Occupati che utilizzano Internet divisi per età (fonte: Istat 2023).
- Figura 13. Interazione uomo-macchina, nelle due versioni dello studio.
- Figura 14. Schema delle quattro fasi di funzionamento di Ollama.
- Figura 15. Panoramica ad alto livello dell'architettura dell'esperimento.
- Figura 16. Schema riassuntivo dell'esperimento Omega.
- Figura 17. Schema di alto livello che riassume le componenti dell'architettura software.
- Figura 18. Visualizzazione della Cosine Similarity (fonte: https://www.reddit.com/r/manim/comments/npwkhl/machine_learning_visualization_cosine_clustering/).

- Figura 19. Prompt impostati per i task CB, RTE e AX-g.
- Figura 20. Prompt impostati per il task COPA.
- Figura 21. I prompt impiegati impostati per il task ReCoRD.
- Figura 22. I prompt impiegati impostati per il task WiC.
- Figura 23. I prompt impiegati impostati per il task WSC.
- Figura 24. I prompt impiegati impostati per il task BoolQ.
- Figura 25. Schema dell'algoritmo di training.
- Figura 26. Schema delle dipendenze delle classi Hugging Face.
- Figura 27. Log del training della variante "fear".
- Figura 28. Log del training della variante "anger".
- Figura 29. Log del training della variante "joy".
- Figura 30. UML Sequence Diagram delle interazioni tra le componenti software.
- Figura 31. Esempio di funzionamento dell'applicazione.
- Figura 32. Distribuzione delle distanze per i 50 prompt fuori contesto.

Elenco delle tabelle

- Tabella 1. I modelli utilizzati.
- Tabella 2. Configurazione PEFT di LoRA.
- Tabella 3. Configurazione di training.
- Tabella 4. Dettagli tecnici dell'API REST esposta dal backend.
- Tabella 5. Schema ChatIteration.
- Tabella 6. Schema ChatHistoryEntry.
- Tabella 7. Risultati di media e deviazione standard per definire la logica di filtro.
- Tabella 8. Risultati degli score ottenuti dai modelli in ogni task, per ogni variante di prompt.
- Tabella 9. Coefficienti della d di Cohen.
- Tabella 10. Regole di tagging manuale.
- Tabella 11. Risultati del calcolo del bias emotivo inserito in fase di fine-tuning.

Ringraziamenti

Scrivendo queste righe, provo emozioni indescrivibili. Il percorso che ho portato a termine mi ha cambiato la vita; la persona che sono oggi, il professionista che sono diventato e la cultura che mi sono formato sono il frutto di sacrifici consistenti e considerevoli.

Diana mi è stata vicino in tutto questo, ha portato pazienza, ha creduto in me sempre, tanto e senza mai dubitare. Una dimostrazione d'amore più bella di questa non poteva darmela. È la prima persona che voglio ringraziare per essere arrivato alla fine. Durante questo percorso, è venuta alla luce la parte più bella di noi due; i tuoi occhi, i miei sorrisi e la voglia di vivere e di spaccare il mondo che abbiamo entrambi. Alice. Insieme siete il carburante che mi dà la forza di fare della nostra vita un capolavoro. Non dimentichiamoci della nostra mascotte a quattro zampe, che ci riempie la vita regalandoci emozioni di tutti i tipi.

Francesca, la mia relatrice, mi ha dato gli strumenti giusti per poter esprimermi al meglio, dandomi non solo l'opportunità di pubblicare su rivista, ma anche la possibilità di crederci e di provarci. Lusso che, riconosco, non hanno tutti. Capace di ascoltare, capace di formulare l'idea giusta al momento giusto e di trasmettere fiducia e sicurezza negli studenti. Un docente non è solo didattica, è anche una guida, responsabile di battere la "via maestra". Grazie, sinceramente.

Guido, mentore e amico, mi ha trasmesso la passione per la ricerca e dei valori che porto sempre con me, custoditi come un tesoro. Le esperienze fatte insieme mi hanno reso la persona che sono; affamata di conoscenza più di quanto tu non lo sia di aperitivi. Un grazie non sarà mai abbastanza, ma è il minimo che la scrittura mi consente di esprimere. È per me motivo di vanto e onore aver collaborato insieme.

Federico. Amico ormai di una vita, ex-collega, co-autore. Insieme abbiamo fatto cose splendide. Condividiamo gli stessi sogni e gli stessi desideri. Spalleggiarci a vicenda per continuare a fare passi in avanti, consistenti, è sintomo di un'amicizia sana che mi sta dando tanto, e a cui devo tanto. Grazie per tutto il supporto.

Gabriel è stato capace di ascoltarmi e di incoraggiarmi quando ne avevo bisogno, senza mai mettere in discussione i risultati che avrei raggiunto. Anzi, li anticipò tutti il giorno della mia iscrizione. L'amicizia è proprio questo. Coinvolgimento profondo nei sogni, nei desideri e nelle passioni degli amici cari. Ti ringrazio.

Voglio dedicare un pensiero anche a mio nonno, Giovanni. Sei venuto a mancare prima di conoscere tua nipote Alice, non sei riuscito a vedere coi tuoi occhi la casa che ho costruito insieme a Diana, e non sei riuscito nemmeno a vedermi raggiungere questo traguardo. Non credo in Dio, ma ho sempre avvertito forte la tua presenza, così come la nostalgia di te.

Infine, ringrazio la mia famiglia, tutta: Diana, Alice, Lara, Filippo, Gianfranco, Paola, Maria, Nicola, Marius, Karola, Gabriel. Sono una persona fortunata, e questa fortuna si riflette ogni giorno nella mia vita. Siamo tutti parte di qualcosa di grande. Ognuno è unico, tutti siamo diversi, ma l'Amore è una costante e dà energia a questa famiglia. Ognuno di noi ha compiuto, per motivi diversi e con modalità diverse, imprese straordinarie. Vi ringrazio di far parte della mia vita.

1. Introduzione

Oggigiorno sempre più spesso l'Intelligenza Artificiale (IA) caratterizza le nostre vite. Vivere nel 2025 significa essere abituati alle novità tecnologiche, il cui trend emergente è l'IA. Ci stiamo abituando ad attribuire alle macchine la caratteristica di essere intelligenti, ed è un'abitudine che stiamo acquisendo velocemente. Soltanto 5 anni fa, non esisteva ancora ChatGPT, e l'opinione pubblica immaginava l'IA confinata dentro il mondo fantascientifico di Hollywood. Lo scenario odierno è totalmente diverso. Strumenti di Intelligenza Artificiale stanno gradualmente infiltrandosi nella quotidianità. Prodotti come ChatGPT, Claude, LeChat, Perplexity vengono impiegati sempre più spesso per ricevere assistenza in attività quotidiane; generiamo ricette culinarie, li utilizziamo per studiare, come supporto a processi di ragionamento e via dicendo. Ogni tanto, arriviamo persino a dimenticarci di parlare con una macchina.

In passato, l'Ingegneria del Software si è concentrata a codificare algoritmi in grado di risolvere problemi specifici, ma la sempre più crescente complessità e dimensionalità dei dati rende questi metodi insufficienti: i Big Data non possono essere elaborati ed analizzati con istruzioni imperative, ma necessitano di automatismi ben più avanzati, e di interfacce semplici.

In un mondo come questo, la cosiddetta "Human-Computer Interaction" (HCI) diviene determinante. Il modo con cui noi umani interagiamo con questa nuova specie di macchine è cruciale, ed è bene riflettere sulla modalità di interazione uomo-macchina: il linguaggio naturale. L'umano è un essere razionale ed emotivo, e l'interfaccia che usa per comunicare coi suoi simili riflette entrambe queste caratteristiche. Il risultato? Una ricchezza espressiva immensa, colorata da diverse sfumature di ambiguità e forme lessicali che una macchina fatica a riconoscere. Interagire nel modo corretto con i LLM è di fondamentale importanza; il linguaggio naturale è l'unico strumento che un utente comune ha a disposizione per comunicare con questi sistemi, e individuare il modo corretto di farlo è la chiave per ottenere il massimo valore possibile.

Per questi motivi, lo studio in oggetto si pone l'obiettivo di quantificare l'impatto delle emozioni nell'interazione uomo-macchina, e nell'interazione macchina-uomo; ci si vuole porre due tipi di domande al contempo: anzitutto, ci si chiede come si comportano i LLM quando l'interlocutore esprime gioia, rabbia e tristezza nell'interazione e come si comporta, invece, quando questi tratti sono assenti, e l'interazione è apatica; in secondo luogo, ci si chiede come reagisce l'umano davanti a una macchina che riflette gioia, rabbia, tristezza o apatia, e se questa reazione abbia qualcosa in comune con il trend di risposta alle emozioni delle macchine.

Lo scopo della ricerca è quello di quantificare queste due tipologie di impatto, ricercando un trend comune nei due versi. Per farlo, utilizzo il metodo induttivo, e si ipotizza che un'interazione apatica, priva di elementi emotivi, risulti più comprensibile per un LLM e produca output di qualità superiore rispetto a interazioni caratterizzate da emozioni esplicite. Per verificare l'ipotesi, vengono strutturati due esperimenti separati: Alpha (α) e Omega (ω). L'obiettivo di Alpha è quantificare l'impatto di quattro diverse tipologie di interazioni influenzate da gioia, rabbia, tristezza e apatia su cinque diversi LLM. Omega mira a verificare le prestazioni di uno studente in un contesto didattico che si serve di una variante di

LLM emotiva (gioia, rabbia, tristezza o apatia) per risolvere un problema complesso. Oltre alle prestazioni dello studente, l'interesse è quello di valutare la percezione che esso stesso ha avuto dell'interazione con la macchina, utilizzando le scale di Likert[1]. L'idea di fondo è quella di produrre un'analisi olistica atta a quantificare l'impatto delle emozioni.

Il motivo per cui questo studio è importante risiede nelle implicazioni che avrebbe l'individuazione di un trend comune. Adattare i modelli sfruttando questo potenziale bias significherebbe aumentare il valore che questi strumenti sono in grado di generare, sfruttando la ricchezza espressiva del linguaggio naturale.

L'esecuzione di questo esperimento comporta costi economici notevoli; predisporre l'hardware necessario per attuarlo è altamente dispendioso. Per questo motivo, l'esperimento Omega, purtroppo, è stato solamente teorizzato e progettato, ma non eseguito. La raccolta dei dati non è stata effettuata. I risultati della fase preparatoria sono comunque rilevanti ed evidenziano la validità dell'approccio e della progettazione tecnica. Nel corso della tesi, verranno esaminati i dettagli tecnici della progettazione unitamente a test quantitativi volti a valutare l'impatto di alcune scelte tecniche.

Ad ogni modo, i risultati ottenuti su Alpha comunicano che sembra esistere un effetto costante, seppur piccolo, che privilegia l'emozione della gioia nelle interazioni tra uomo e macchina; i LLM coinvolti nello studio hanno prodotto risultati migliori per un tasso pari al 10% rispetto alla paura, per esempio. Sebbene l'effetto sia contenuto, esso è tutt'altro che trascurabile. Per dare un'idea concreta, se si ipotizza di applicare la differenza del 10% nel contesto dell'antifrode, significa dire che su 100 tentativi di frode, ne vengono identificati correttamente 10 in più, semplicemente inserendo della gioia nel prompt, a parità di semantica del messaggio.

Sebbene Omega non abbia potuto ottenere risultati a causa della mancata esecuzione, la metodologia imbastita per l'esperimento dimostra che con un ammontare di dati relativamente contenuto è possibile modificare il comportamento di un LLM di 14 miliardi di parametri come Velvet; la variante emotiva legata alla gioia, costruita addestrando il modello Velvet-14B, fa trasparire gioia in una risposta su quattro, un tasso più alto del modello di base di quasi due decine di punti percentuali. Inoltre, la struttura dell'esperimento può fungere da base per futuri esperimenti ed esecuzioni, gettando le fondamenta per la costruzione di un benchmark utile a valutare l'effetto delle emozioni nell'HCI tra umani e LLM.

La tesi è strutturata in 7 capitoli principali. Dopo l'introduzione, v'è il capitolo "Intelligenza Artificiale ed emozioni: dove siamo" (2), pensato per fornire una panoramica olistica del mondo dell'IA. In tale sezione vengono menzionate e descritte ad un livello di astrazione elevato le tecnologie principali, utilizzate nel corso dei due esperimenti; l'idea è quella di fornire al lettore un'idea del contesto in cui lo studio si inserisce, fornendo gli strumenti di base utili a comprendere le motivazioni dello stesso. Segue il capitolo 3, "Interazione emotiva: uomo-macchina". In questa parte della tesi l'obiettivo è quello di descrivere i due esperimenti, fornendo al lettore una descrizione onnicomprensiva della struttura dei due; si vuole, inoltre, spiegare le motivazioni di ciascun esperimento, descrivendo il

[1] Tecnica psicometrica di misurazione dell'atteggiamento inventata dallo psicologo Rensis Likert.

ruolo dei singoli componenti e i razionali che si pongono dietro alla scelta dell'impiego e dello sviluppo degli stessi.

Una volta contestualizzato e concettualizzato lo studio oggetto di tesi, si procede a descrivere ed enunciare il dettaglio tecnico dei due esperimenti — Alpha e Omega — in due capitoli dedicati, 4 e 5. Lo scopo di questi due capitoli è quello di entrare nel merito di ogni scelta tecnica e ogni algoritmo, così da dare al lettore un dettaglio tecnico utile a comprendere la natura di certe scelte, e utile a comprendere come queste si orientano in funzione dell'obiettivo di Alpha e Omega.

I risultati ottenuti dai due esperimenti sono analizzati nel capitolo 6, "Risultati impatto emotivo", nel quale si propone anche dell'analisi statistica per validare gli output collezionati. La tesi si conclude con il capitolo numero 6, relativo alle conclusioni. In tale sede si commentano i risultati ottenuti, rimandando a sviluppi futuri.

La struttura dell'elaborato è stata concepita in questo modo per rievocare l'idea di un viaggio nell'interazione uomo-macchina; partendo dalla collocazione dello studio in un contesto spazio-temporale come quello moderno, si arriva al dettaglio dell'implementazione e si analizzano i dati raccolti a valle. La speranza è quella di far passare il messaggio per cui le emozioni non sono affatto secondarie e trascurabili, come il mondo sembra suggerirci, tutt'altro; le emozioni caratterizzano gli umani in quanto tali, e anche nell'interazione con le moderne macchine come i LLM hanno un potenziale espressivo che non deve essere visto come una limitazione, ma come un punto di forza. Lo studio vuole incentivare la ricerca a esplorare la stretta relazione che sussiste tra i risultati e le prestazioni dei modelli e la presenza di emozioni.

Le macchine rappresentano i concetti e il linguaggio tramite numeri, in spazi multidimensionali. All'interno di quegli spazi enormi e difficili da visualizzare mentalmente, tuttavia, risiede anche la codifica e la formalizzazione di concetti complessi come la gioia, la rabbia, la paura e l'apatia. Trattandosi di concetti nettamente differenti, altrettanto differenti sono anche le relative rappresentazioni all'interno delle macchine.

In ultima istanza, l'ambizione è quella di ricordare e ribadire che le emozioni non indeboliscono l'interazione uomo-macchina, anzi. Se comprese, esse possono rappresentare un elemento di valore di non poco conto.

2. Intelligenza Artificiale ed emozioni: dove siamo

L'Intelligenza Artificiale è un mondo molto ampio, e non è semplice raccontarlo. Basta pensare al significato di "Intelligenza", che è tutt'oggi oggetto di continue discussioni e modifiche. Filosofi e scienziati non si sono mai accordati del tutto sulla semantica del termine, eppure l'Intelligenza Artificiale si è sviluppata attorno a un'idea operativa di intelligenza, fondata sulla capacità di apprendere, ragionare e risolvere problemi. Da un lato, c'è l'approccio simbolico, che considera l'intelligenza come una questione di manipolazione di simboli secondo regole prestabilite, come proposto dall'Intelligenza Artificiale classica. Dall'altro, c'è l'approccio connessionista, che si ispira alla struttura neurale del cervello umano, privilegiando modelli statistici e reti neurali. Negli ultimi anni, l'IA ha conosciuto una crescita esponenziale grazie all'aumento della potenza computazionale e alla disponibilità di enormi quantità di dati. Le tecniche di apprendimento automatico, e in particolare il Deep Learning, hanno permesso di ottenere risultati impressionanti in campi come la visione artificiale, l'elaborazione del linguaggio naturale (NLP — Natural Language Processing) e la generazione di contenuti. Tuttavia, la domanda fondamentale rimane: l'Intelligenza Artificiale è davvero "intelligente" o è solo un sofisticato strumento di calcolo? Alcuni esperti, come Yann LeCun[2], sostengono che le IA attuali siano ancora ben lontane da una comprensione profonda del mondo e che la loro "intelligenza" sia solo il frutto di pattern statistici e correlazioni nei dati. Altri, invece, vedono nell'evoluzione delle architetture neurali un passo progressivo verso forme di intelligenza più avanzate, capaci di ragionamento astratto e adattamento autonomo. In questa sezione si vuole tentare di fare chiarezza, fornendo una panoramica storico-tecnologica di questo mondo, contestualizzando l'ambito di studio.

2.1. Da dove origina l'IA

L'Intelligenza Artificiale è un campo estremamente variegato che comprende diverse tipologie di sistemi e tecnologie, ciascuna con caratteristiche e applicazioni specifiche. Per contestualizzare in maniera corretta il settore è utile ripercorrere la sua evoluzione storica, descritta visualmente tramite una linea del tempo (figura 1).

La teoria su cui si erge la moderna Intelligenza Artificiale ha origine negli anni '40, quando ancora non si parlava di IA. Nel 1943 W. McCulloch e W. Pitts formalizzarono il primo modello di rete neurale artificiale, ispirato al funzionamento del cervello umano. Tale modello consisteva in una mera formalizzazione matematica teorica e, a quel tempo, le applicazioni che conosciamo oggi erano ancora molto lontane.

Nel 1950 Alan Turing inventò il celebre Test di Turing; un metodo per valutare l'intelligenza di una macchina in base alla sua capacità di sostenere una conversazione indistinguibile da quella di un essere umano. Per superare il test, un umano interagisce con due interlocutori — una macchina e un altro umano — senza sapere chi sia chi. Se l'umano non riesce a distinguere tra i due interlocutori, la

[2] Chief AI Scientist di Meta, professore alla New York University.

macchina supera il test. Questo test è diventato un riferimento per l'IA, ponendo l'accento sull'interazione umana e sulla percezione dell'intelligenza.

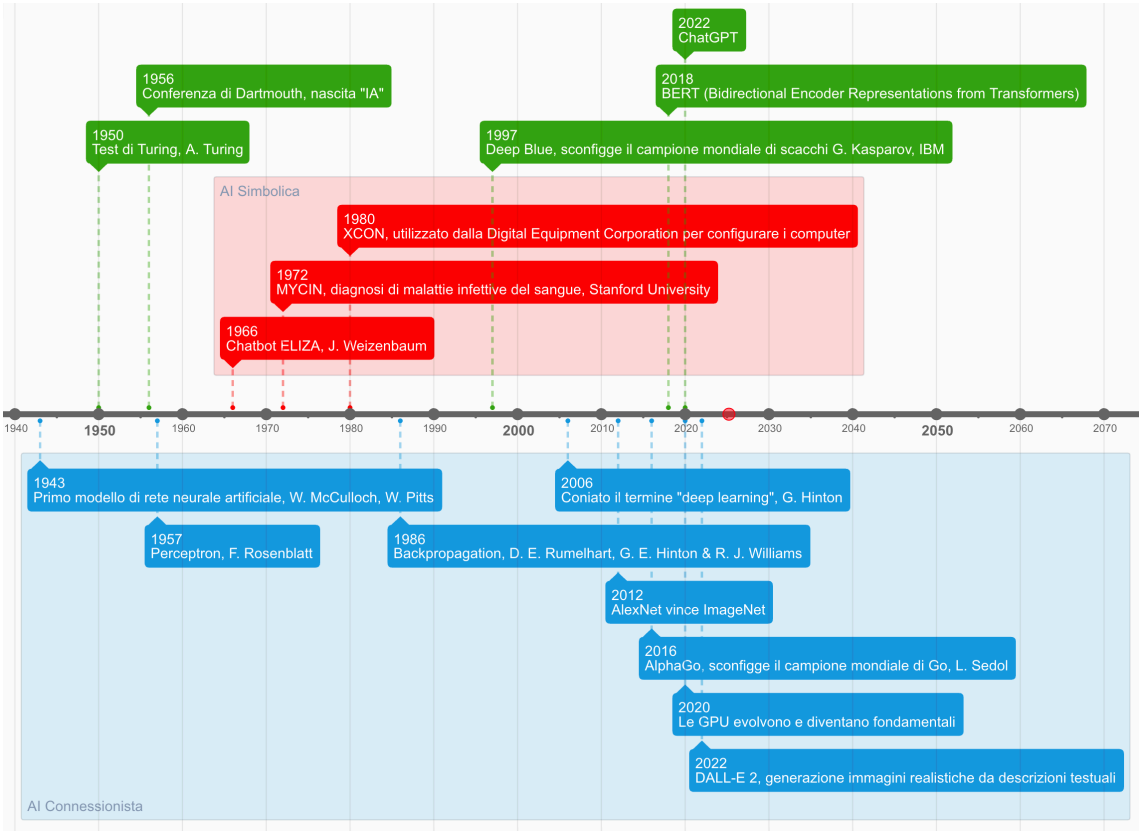


Figura 1. Linea temporale dell'evoluzione dell'IA.

Nel 1956 venne per la prima volta formalizzato il termine "Intelligenza Artificiale" nella famosa conferenza di Dartmouth, a cui presero parte figure di spicco come J. McCarthy, M. Minsky, A. Newell, H. A. Simon e A. Samuel, pionieri dell'IA.

L'anno successivo avviene un evento cruciale per l'IA connessionista: la formalizzazione del Perceptron di Rosenblatt, un primo tipo di rete neurale basato sull'apprendimento automatico progettata per la classificazione binaria. La relazione che intercorre tra il Perceptron e il modello di McCulloch e Pitts del '43 è significativa. Il percettrone, rispetto al modello precedente, introduce l'apprendimento automatico, formalizzando concetti fondamentali come pesi e bias, che tutt'oggi sono presenti e trattati nella letteratura.

Nel 1966 nacque ELIZA, sviluppata da Joseph Weizenbaum al MIT, uno dei primi chatbot. Utilizzava tecniche di corrispondenza di modelli per simulare una conversazione terapeutica, evidenziando i limiti e le potenzialità dell'interazione uomo-macchina.

Gli anni '70 e '80 sono caratterizzati dall'esplosione dei sistemi esperti, capisaldi dell'IA Simbolica. Questi sistemi utilizzano una base di conoscenza e un

motore inferenziale[3] per risolvere problemi complessi che normalmente richiederebbero l'intervento di un esperto umano. Due dei sistemi esperti più famosi sono MYCIN (1972) e XCON (1980). Il primo era un sistema esperto impiegato per la diagnosi di malattie infettive del sangue, sviluppato dalla Stanford University. MYCIN utilizzava una base di conoscenze e regole per fornire diagnosi mediche, mentre il secondo, XCON venne utilizzato dalla Digital Equipment Corporation per configurare i computer, dimostrando le potenzialità dell'applicazione pratica dei sistemi esperti nel settore industriale.

H. A. Simon e A. Newell sostenevano fermamente che tutta l'IA potesse essere gestita con approcci simbolici. I due svilupparono programmi come il "Logic Theorist" e il "General Problem Solver" (GPS), che utilizzavano principi logici per risolvere problemi complessi. Il loro lavoro ha influenzato profondamente la ricerca nell'IA, portando all'idea che i processi cognitivi umani potessero essere simulati attraverso l'uso di regole logiche e algoritmi. Col passare del tempo, diventò chiaro che la crescente dimensionalità dei dati non poteva essere gestita tramite un approccio "imperativo e deterministico" come quello simbolico, al contempo, però, gli approcci connessionisti erano fortemente limitati dall'hardware disponibile all'epoca.

Il 1986 segnò una svolta per il Deep Learning con l'introduzione dell'algoritmo di retropropagazione (backpropagation), che rese possibile l'addestramento di reti neurali multistrato. Questo algoritmo risolse il problema dell'aggiustamento dei pesi nelle reti neurali profonde. Consecutivamente, G. Cybenko ideò il Teorema dell'Approssimazione Universale di Cybenko (1989), che sancisse come una rete neurale con un singolo strato potesse approssimare qualsiasi funzione continua su un intervallo chiuso, con qualsiasi grado di precisione desiderato. Il significato del teorema implica che le reti neurali multistrato siano in grado di approssimare qualsiasi tipo di funzione complessa, a seconda dell'architettura delle stesse.

Nel 2006, G. Hinton coniò il termine "Deep Learning" per descrivere le reti neurali profonde. Da quel punto in poi, le Deep Neural Networks presero il sopravvento. Uno dei primi ambiti in cui queste reti si affermarono fu la Computer Vision, il cui evento più importante è rappresentato dalla vittoria della competizione ImageNet da parte di AlexNet, nel 2012. AlexNet superò di gran lunga gli approcci precedenti, usati fino a quel momento.

Da quel momento in poi la ricerca accelerò, fino ad arrivare al 2017, anno in cui venne pubblicato il celebre paper "Attention Is All You Need", che formalizzò l'architettura dei Transformers. Sviluppando l'algoritmo dell'attenzione, A. Vaswhani spianò la strada ai primi Large Language Models (LLM). Il primo a fare scalpore nell'opinione pubblica fu ChatGPT, capace di generare risposte conversazionali e coerenti rende accessibile e utile nel quotidiano l'IA. ChatGPT è, se vogliamo, la prima applicazione di Intelligenza Artificiale a toccare da vicino la quotidianità delle persone.

Riflettere sulle date di questa carrellata è fondamentale. Dal primo modello di neurone artificiale di McCulloch e Pitts del '43 a ChatGPT nel 2022 trascorre meno di un secolo di storia. Oggi siamo abituati a parlare di IA, e ci stiamo abituando a

[3] I motori inferenziali sono spesso basati sulla logica, mentre le basi di conoscenza sono gestite con l'aiuto delle ontologie, modellazioni concettuali formali.

usarla quotidianamente sul lavoro e nella vita privata, ma è opportuno ricordarsi che siamo arrivati qui in soli 80 anni di storia, e che dall'avvento di AlexNet a ChatGPT ne passano appena 10.

2.1.1. Approcci simbolici e connessionisti

Dopo aver esplorato l'evoluzione storica dell'IA, è opportuno analizzare una delle distinzioni fondamentali nel settore: gli approcci simbolici e connessionisti.

Come precedentemente descritto, l'IA simbolica si basa su regole deterministiche e utilizza simboli per rappresentare e manipolare la conoscenza. I due costrutti fondamentali di questo approccio sono: le regole e la base di conoscenza. Le regole possono essere elaborate usando la logica, un linguaggio formale che modella il ragionamento umano attraverso assiomi e inferenze; le basi di conoscenza possono assumere diverse forme — dai database classici ai Linked Open Data[4], passando per le ontologie.

L'IA connessionista, invece, si fonda sulle reti neurali artificiali e introduce un concetto chiave che la distingue nettamente dall'approccio simbolico: il Representation Learning; a differenza dei sistemi simbolici, dove la rappresentazione della conoscenza è definita a priori e strutturata rigidamente, i modelli connessionisti apprendono automaticamente le rappresentazioni dai dati, adattandosi alle complessità delle informazioni in ingresso.

Questa capacità adattativa è uno dei motivi principali per cui l'IA simbolica ha perso terreno a favore di quella connessionista. Con l'aumento esponenziale della quantità di dati disponibili, culminato nell'era dei Big Data, gli approcci statici e basati su regole hanno mostrato limiti evidenti, mentre il representation learning ha permesso di superare la rigidità delle rappresentazioni predefinite. Un ostacolo significativo legato alla crescita della complessità dei dati è la Curse of Dimensionality (CoD, o "maledizione della dimensionalità"), un fenomeno che rende difficile l'analisi e la generalizzazione quando il numero di variabili cresce considerevolmente. Le reti neurali, grazie alla loro capacità di apprendere rappresentazioni latenti[5], si sono dimostrate più efficaci nel mitigare questo problema rispetto ai modelli simbolici.

Tuttavia, ogni scelta in Informatica comporta dei compromessi. Se da un lato l'approccio connessionista offre vantaggi nell'adattamento ai dati e nella gestione della CoD, dall'altro introduce nuove sfide. Uno dei problemi principali è l'interpretabilità: le reti neurali profonde funzionano come "scatole nere", rendendo difficile comprendere i meccanismi interni che portano a una decisione. Si parla di "scatole nere" perché le reti neurali, come già accennato, hanno più strati. Generalmente, si parla di tre tipologie di strati: input, hidden e output. Se gli strati di

---

[4] I Linked Open Data sono noti anche col termine di "dati aperti". Essi consistono in conoscenza strutturata, collegata tramite riferimenti semantici. La loro invenzione è strettamente connessa al lavoro di Tim Berners Lee, considerato il padre del World Wide Web.

[5] Le rappresentazioni latenti sono trasformazioni compatte e astratte dei dati grezzi, apprese automaticamente dai modelli per catturare strutture e relazioni sottostanti, migliorando la generalizzazione e la riduzione della dimensionalità (e.g. la forma delle orecchie in un sistema di Computer Vision).

input e output sono facilmente interpretabili e intelligibili, se considerati singolarmente, gli strati nascosti operano trasformazioni lineari e operazioni algebriche sui dati sfruttando il paradigma del Representation Learning — di conseguenza, gli hidden layer non sono spiegabili alla pari di input e output. Questo problema è particolarmente critico in settori come la sanità, dove trasparenza e spiegabilità sono essenziali per la fiducia e la sicurezza, ma non solo.

Un'altra criticità riguarda la robustezza dei modelli. Le reti neurali possono essere vulnerabili agli Adversarial Attacks, ovvero attacchi informatici in cui piccole variazioni impercettibili nei dati di input inducono il modello a compiere errori significativi. Ad esempio, nel contesto della Computer Vision, immagini alterate da perturbazioni minime possono ingannare un sistema di riconoscimento visivo, facendogli classificare erroneamente un oggetto. Se si considera la vulnerabilità nel contesto della guida autonoma, per esempio, è facile intuire il rischio correlato.

L'IA solleva anche importanti questioni etiche e di equità. I modelli apprendono dai dati, il che significa che possono involontariamente perpetuare e amplificare bias preesistenti, portando a decisioni discriminatorie (e.g. preferendo il genere maschile a quello femminile, o viceversa, oppure generando immagini di persone "belle" rappresentandole tutte magre, bionde e con occhi azzurri). La ricerca attuale si concentra sullo sviluppo di tecniche per identificare e mitigare questi bias, garantendo modelli più equi e inclusivi.

Infine, vi è la questione della generalizzazione. Sebbene i modelli di IA possano ottenere prestazioni eccezionali su compiti specifici per cui sono stati addestrati, spesso faticano ad adattarsi a scenari nuovi senza un ulteriore riaddestramento. Questa limitazione riduce l'applicabilità dei modelli a contesti più ampi e rappresenta un'area di ricerca attiva per migliorare la capacità di apprendere in modo più astratto e trasferibile.

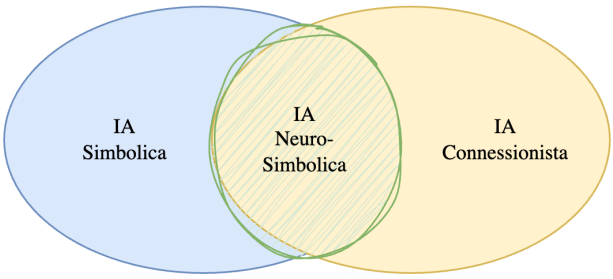


Figura 2. Rappresentazione insiemistica della classificazione.

Sarebbe scorretto non fare menzione degli approcci neuro-simbolici. Oggi la distinzione tra approcci simbolici e connessionisti non è sempre così netta; esiste un approccio che consente di integrare entrambi, chiamato "neuro-simbolismo". Il neuro-simbolismo si propone di integrare l'approccio simbolico al connessionista, ottenendo il meglio da entrambi. In questo contesto, l'approccio simbolico può essere usato per aumentare il controllo sulle capacità generative del modello connessionista, mantenendo la sua potenza derivante dal Representation Learning per i task più complessi. Si propone una rappresentazione insiemistica dei tre approcci per dare un'idea delle intersezioni (figura 2).

2.1.2. Metodologie di classificazione dell'IA

Sebbene la distinzione tra approcci simbolici, connessionisti e neuro-simbolici sia utile per individuare le differenze principali relativamente alla letteratura moderna, è bene spostarsi a un livello ancora più alto per avere una panoramica completa della composizione del mondo dell'IA.

Un'altra distinzione importante è quella tra IA debole e IA forte. L'IA debole, nota anche come IA narrow, è progettata per svolgere compiti specifici e limitati. Questi sistemi sono altamente specializzati e non possiedono capacità cognitive generali. Esempi di IA debole includono i sistemi di riconoscimento vocale, come Siri o Alexa, e i sistemi di raccomandazione utilizzati da piattaforme di streaming come Netflix. I vantaggi dell'IA debole includono l'efficacia nell'eseguire compiti specifici con elevata precisione, mentre uno svantaggio è la mancanza di flessibilità e capacità di adattamento a nuovi compiti o contesti.

L'IA forte, o intelligenza artificiale generale (AGI), rappresenta l'obiettivo ultimo dell'IA: sistemi in grado di comprendere, apprendere e applicare conoscenze in modo simile agli esseri umani. Attualmente, l'IA forte è ancora un obiettivo teorico e non esistono sistemi pienamente funzionali. I vantaggi potenziali includono la capacità di risolvere una vasta gamma di problemi complessi con un'intelligenza simile a quella umana, mentre i rischi includono la complessità di sviluppo e i potenziali problemi etici e di sicurezza associati a sistemi con capacità cognitive avanzate. Nell'epoca moderna, si parla ancora di IA debole. L'AGI, sebbene sia stata fomentata e paventata da figure di spicco nell'ambito tech come Sam Altman[6] ed Elon Musk[7], è ancora molto lontana.

Un altro modo di classificare l'IA riguarda la modalità di addestramento. In questo contesto, si scende nel merito del Machine Learning[8]: possiamo distinguere tra addestramento supervisionato (supervised learning), non supervisionato (unsupervised learning) e per rinforzo (reinforcement learning). La prima tipologia consiste nell'apprendimento da dati etichettati dall'uomo, dove l'output desiderato è fornito insieme all'input durante il processo di training. Esempi includono i classificatori di immagini che apprendono a riconoscere oggetti basandosi su immagini etichettate (e.g. al modello viene fornita l'immagine di un cane, specificando che ciò che c'è in foto è un cane). I vantaggi di questa tipologia includono l'alta accuratezza nei compiti in cui sono disponibili grandi quantità di dati etichettati, mentre uno svantaggio è la dipendenza da dati etichettati, che possono essere costosi e time-consuming da ottenere.

L'unsupervised learning, invece, consiste nell'apprendere da dati non etichettati, cercando di identificare pattern nascosti nei dati. Esempi includono il clustering, dove i dati vengono raggruppati in base a somiglianze, senza disporre di etichette a priori. Il clustering è utile in problemi di classificazione, quando a priori

---

[6] CEO di OpenAI, una delle aziende leader mondiali nel mercato dell'IA oggi.

[7] CEO di diverse tech company di spicco come Tesla, SpaceX, X (ex Twitter), PayPal.

[8] Branca dell'IA focalizzata sul processo di apprendimento automatico.

non sono note le classi (e.g. in un contesto di SEO, con tecniche di clustering è possibile individuare diverse tipologie di utenti, classificandoli in base all'utilizzo che essi fanno della piattaforma web d'interesse). I vantaggi includono la capacità di lavorare con grandi quantità di dati non strutturati, mentre uno svantaggio è la difficoltà nel valutare l'accuratezza e l'efficacia del modello senza dati etichettati.

Infine, il reinforcement learning consente al modello di apprendere attraverso interazioni con un ambiente, ricevendo feedback sotto forma di ricompense o punizioni. Esempi includono i sistemi di gioco autonomi e i robot che apprendono a eseguire compiti attraverso tentativi ed errori (approccio "trial and error"). I vantaggi includono la capacità di apprendere comportamenti ottimali attraverso l'esperienza, mentre uno svantaggio è la complessità nel progettare ambienti di apprendimento efficaci e nel gestire l'esplorazione vs. sfruttamento delle conoscenze apprese.

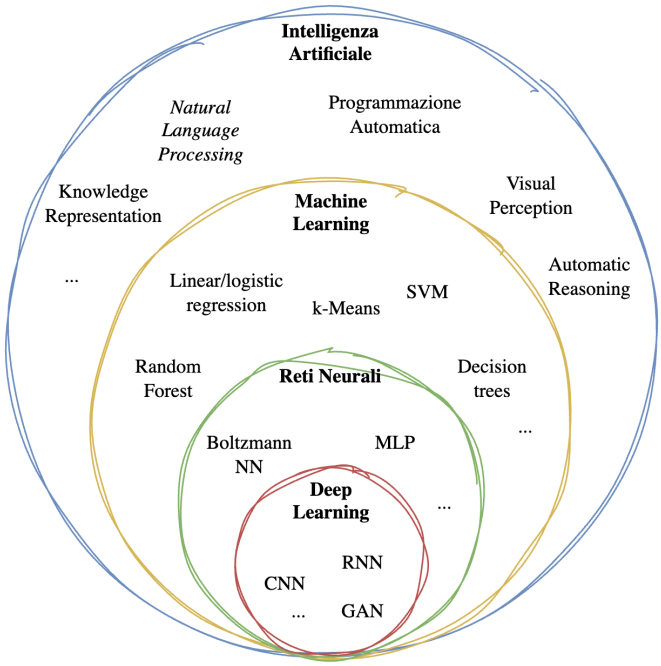


Figura 3. Gerarchia dell'IA.

Sebbene oggi la maggior parte delle applicazioni convergano nel Deep Learning, è bene ricordare che l'IA è molto più del "solo" Deep Learning (come mostrato in figura 3). L'IA classica si fonda su paradigmi come il Natural Language Processing (NLP — elaborazione del linguaggio naturale) che non è confinato nel Deep Learning. Il Reasoning Automatico, per esempio, sfrutta ontologie e reasoner (e.g. Pellet) per usare la logica e modelli concettuali per automatizzare processi di ragionamento e inferenza, creando nuovi dati note le premesse.

Trattare ogni strumento e paradigma schematizzato nel diagramma richiederebbe un lavoro enorme, che non è nell'interesse della tesi in oggetto; ci si vuole limitare a fornire una panoramica utile a contestualizzare l'IA e utile a descrivere le tecnologie e metodologie di base che sono state impiegate.

2.1.3. Che cosa sono e come funzionano i LLM

I LLM, precedentemente citati, sono modelli di intelligenza artificiale progettati per lavorare con il linguaggio naturale, abilitando operazioni come traduzione automatica, generazione di testo, risposta a domande e analisi del sentiment[9]. Questi modelli si alimentano di enormi quantità di dati testuali e sono in grado di apprendere schemi complessi nel linguaggio, rendendoli estremamente versatili e potenti.

I primi LLM sono stati sviluppati utilizzando l'architettura dei Transformers, che ha rappresentato una svolta significativa rispetto ai modelli precedenti, come le reti neurali ricorrenti (RNN) e le LSTM (Long-Short Term Memory). L'architettura dei Transformers si basa principalmente sul meccanismo di self-attention, che consente al modello di analizzare e ponderare ogni parola in una sequenza in relazione a tutte le altre, catturando dipendenze a lungo raggio in modo parallelo e senza la necessità di elaborare i dati in serie. I Transformers sono costituiti da due componenti principali: Encoder e Decoder. L'Encoder elabora l'input, creando una rappresentazione interna di ciascuna parola o token nella sequenza. Ogni parola viene trasformata in un vettore numerico, elaborato con successive trasformazioni lineari; il meccanismo di self-attention permette al modello di considerare ogni parola in relazione alle altre. Il Decoder, invece, genera l'output a partire dalla rappresentazione prodotta dall'Encoder. Ad esempio, nel caso della traduzione automatica, il Decoder traduce la sequenza di parole nella lingua di destinazione.

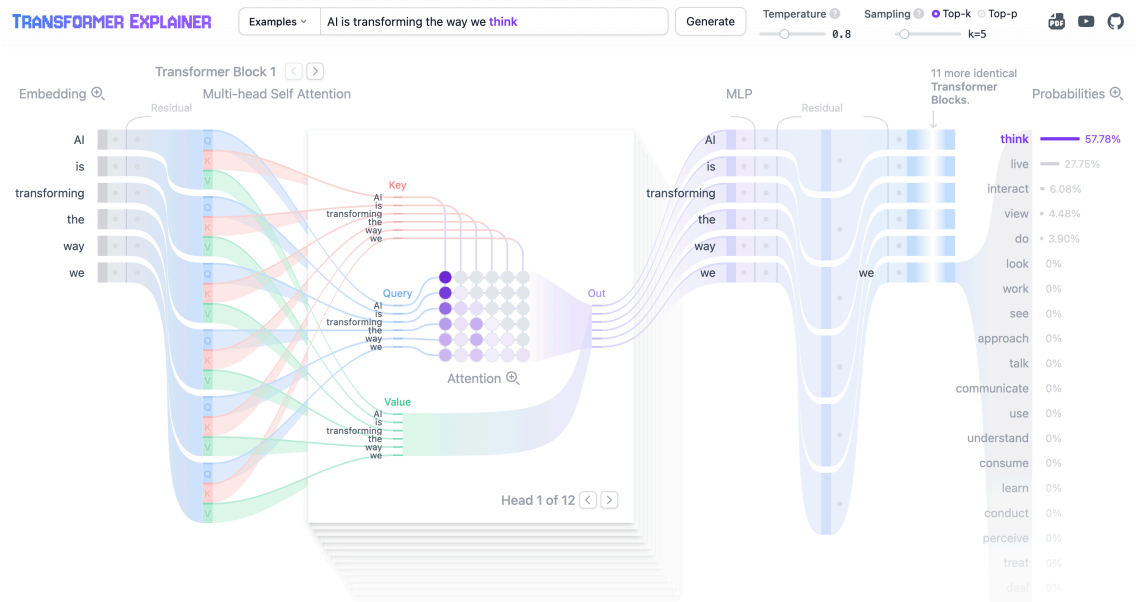


Figura 4. Visualizzazione dei Transformers (fonte poloclub.Github.Com).

[9] Nel mondo del NLP (elaborazione del linguaggio naturale) si definisce "sentiment" il carico emotivo di un testo; generalmente è binario (sentiment positivo o negativo), ma può essere anche neutrale. L'analisi del sentiment è molto utile in ambito marketing sui social network per valutare quantitativamente l'apprezzamento e la qualità di un prodotto, oppure il tasso di gradimento di un partito politico (ma non solo).

Nella figura 4 è rappresentata una visualizzazione del funzionamento dei Transformers, dove si può vedere l'input (a sinistra) elaborato nei blocchi di Multi-Head self-Attention, seguendo l'elaborazione fino alla predizione della parola successiva. È importante puntualizzare che i LLM sono nati con l'architettura Transformers, ma che essa non è l'unica alternativa possibile, oggi.

Con l'evoluzione degli LLM, sono stati sviluppati modelli sempre più grandi, come GPT (Generative Pre-trained Transformer), che hanno ulteriormente migliorato la capacità di comprensione e generazione del linguaggio. La dimensione del modello, espressa dal numero di parametri[10], è uno degli aspetti cruciali per il successo degli LLM, poiché un numero maggiore di parametri consente al modello di apprendere una gamma più ampia di conoscenze e pattern linguistici.

L'addestramento di questi modelli avviene su dataset (insiemi di dati) enormi, che possono comprendere testi tratti da libri, articoli, conversazioni online e molto altro. Grazie a questa vasta esposizione ai dati, gli LLM sono in grado di generalizzare efficacemente su una varietà di compiti, senza la necessità di essere addestrati specificamente per ciascuna applicazione.

Durante l'addestramento, i LLM utilizzano una tecnica di apprendimento supervisionato, in cui il modello impara a prevedere la parola successiva in una sequenza di testo. Tuttavia, questa fase di training richiede una notevole quantità di potenza computazionale e di dati, spesso gestiti da cluster di supercomputer per eseguire i calcoli necessari.

Per adattare un LLM a compiti specifici, può essere necessario un ulteriore addestramento, noto come fine-tuning. In questo processo, il modello viene addestrato su un dataset più piccolo e mirato al compito desiderato. Il fine-tuning consente di specializzare il modello per applicazioni specifiche, migliorandone le prestazioni in contesti particolari. Il fine-tuning è particolarmente utile quando si desidera che il modello eccella in un dominio specifico, come la medicina, il diritto o il servizio clienti. Durante questo processo, i parametri del modello vengono aggiornati in modo da riflettere meglio le peculiarità del nuovo dataset. Questo permette al modello di acquisire una comprensione più approfondita delle sfumature e delle specificità del dominio target, migliorando così la precisione e la pertinenza delle sue risposte.

Gli LLM possono anche essere utilizzati per compiti zero-shot o few-shot, in cui il modello deve generalizzare su nuovi compiti con pochi o nessun esempio di addestramento specifico. Questo è possibile grazie alla capacità del modello di catturare relazioni linguistiche generali durante l'addestramento iniziale.

Le applicazioni degli LLM sono molteplici e spaziano da assistenti virtuali e chatbot per l'assistenza clienti, alla generazione automatica di contenuti come articoli e storie, alla traduzione automatica, all'analisi del sentiment per valutare il tono emotivo di un testo, e alla ricerca e sintesi di informazioni da grandi volumi di testo.

Concludendo, è importante specificare che l'unità di informazione elementare dei LLM sono i token. I token possono essere parole o frammenti delle stesse; indicizzati in un dizionario, questi sono utili per convertire i dati da

[10] I "pesi" del modello, appresi durante la fase di addestramento elaborando i dati.

rappresentazione testuale (testo libero) a vettoriale (numerico). Non esiste un'unica regola generale per la definizione e la gestione dei token, ma è importante notare che maggiore è la dimensione del dizionario dei token, maggiori saranno i parametri che il modello dovrà apprendere, in termini numerici.

2.1.4. Che cosa sono i RAG

I sistemi RAG, o Retrieval-Augmented Generation, rappresentano un'innovazione significativa nel campo dell'elaborazione del linguaggio naturale. Essi combinano le capacità di recupero delle informazioni (Information Retrieval) con quelle di generazione del testo. Questi sistemi integrano un modulo di retrieval, che seleziona documenti o passaggi rilevanti da un vasto corpus di dati, con un modulo di generazione, che sintetizza queste informazioni per produrre risposte coerenti e contestualmente appropriate. Si propone uno schema dettagliato che spiega il funzionamento di un RAG ad alto livello (figura 5).

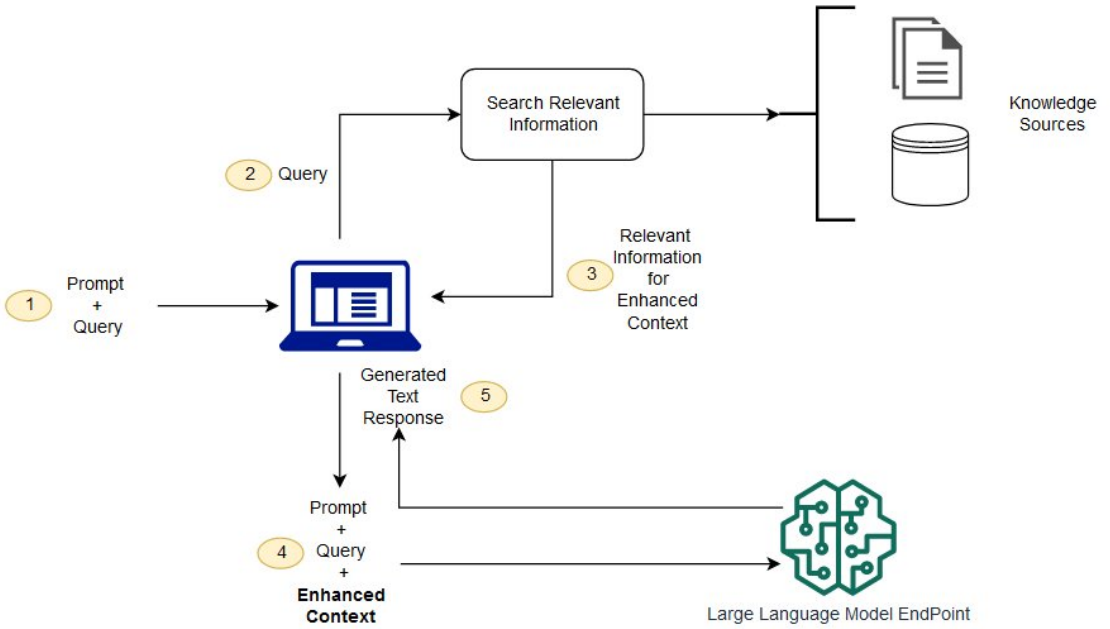


Figura 5. Come funziona Retrieval-Augmented Generation
(fonte: AWS — https://aws.amazon.com/it/what-is/retrieval-augmented-generation/).

In primo luogo, l'utente invia un messaggio (prompt) al sistema, il quale genera la query (interrogazione) da sottoporre al sistema di Information Retrieval. Il risultato della ricerca è una classifica dei testi più rilevanti in base alla richiesta dell'utente (e.g. se sto cercando notizie sportive sulle partite calcistiche dell'Italia, posso aspettarmi di trovare nelle prime posizioni le notizie inerenti, mentre in coda troverò argomenti scorrelati). Dalla classifica, vengono selezionate le *k* informazioni più rilevanti[11]. Tali informazioni vengono utilizzate come contesto e aggiunte nel

[11] *k* è arbitrario, ma solitamente viene fissato in un intervallo compreso tra [1,3].

messaggio per il LLM d'interesse. Questo significa fornire al LLM contesto specifico per rispondere alla domanda dell'utente — approccio molto utile per ridurre le cosiddette "allucinazioni", fenomeno in cui la risposta di un LLM risulta essere imprecisa, sbagliata o decontestualizzata. Infine, al LLM spetta il compito di elaborare la risposta, che viene restituita all'utente.

A differenza dei modelli che si basano esclusivamente su conoscenze pre-addestrate, i sistemi RAG possono accedere a informazioni aggiornate e specifiche del contesto, migliorando così l'accuratezza e la pertinenza delle risposte generate. Questo approccio è particolarmente utile in applicazioni come i chatbot avanzati, i sistemi di domande e risposte (Question Answering) e gli assistenti virtuali, dove la capacità di recuperare informazioni precise e di generare risposte dinamiche è cruciale. Tuttavia, la progettazione e l'implementazione di sistemi RAG richiedono una gestione attenta delle fonti di dati e una calibrazione accurata dei modelli per garantire che le informazioni recuperate siano utilizzate in modo efficace e senza introdurre distorsioni o incoerenze.

L'elaborazione della risposta da parte del LLM consente di simulare la conversazione alla pari di un essere umano; l'esperienza che prova l'utente nell'utilizzare questi tipi di sistemi è molto fluida e discorsiva. Il valore aggiunto dei RAG, rispetto ai sistemi di Information Retrieval puri, consiste proprio nella fluidità e nella discorsività dell'approccio; i sistemi IR classici si limitano a restituire le fonti inerenti alla ricerca dell'utente (un approccio simile al funzionamento del motore di ricerca Google, per dare un'idea); per ultimo, ma non ultimo, la rielaborazione del testo da parte di LLM consente di eliminare eventuali errori ortografici e di battitura presenti nel testo originario.

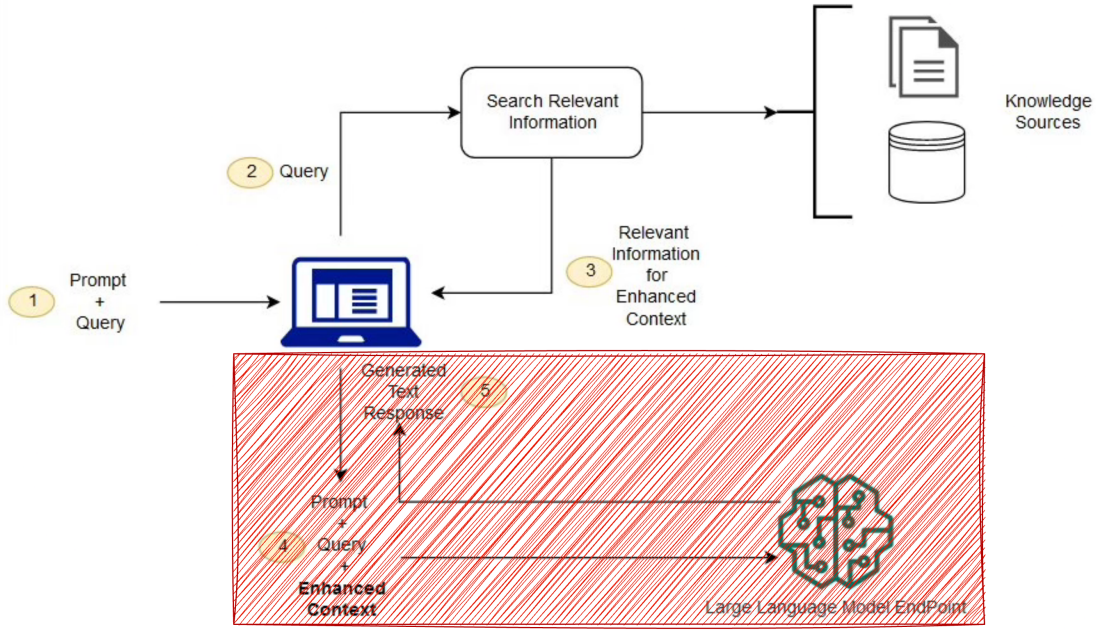


Figura 6. Differenza tra RAG e sistemi di Information Retrieval classici.

È utile fare un distinguo tra RAG e sistemi di Information Retrieval puri. In figura 6 viene mostrata graficamente la distinzione tra RAG e IR. Se, da un lato, la

RAG si serve dell'Intelligenza Artificiale generativa (LLM) per elaborare la risposta da dare all'utente in maniera più naturale e discorsiva, i sistemi IR si limitano a individuare e recuperare informazioni rilevanti nella loro forma base. Un esempio potrebbe essere il motore di ricerca Google, allo stato dell'arte di qualche anno fa; digitando un testo di ricerca, Google applicava tecniche di Information Retrieval sofisticate per individuare tutti i link e le fonti inerenti alla ricerca dell'utente. Tuttavia, il motore di ricerca non dava all'utente una risposta alla sua domanda di ricerca in linguaggio naturale come farebbe un RAG moderno; la risposta al quesito di ricerca dell'utente era rappresentata unicamente dalle fonti ipertestuali (e.g. link e siti web) pertinenti alla ricerca dell'utente. In figura 6 si evidenzia in rosso la parte di processo che compete unicamente ai sistemi RAG; il processo di IR si arresta al punto 3.

Lo step evolutivo rappresentato dall'avvento dei RAG riguarda le elaborazioni dei punti 4 e 5, ovvero la fusione delle informazioni di contesto recuperate tramite IR al prompt dell'utente, e la generazione di una risposta in linguaggio naturale. Questo approccio migliora notevolmente l'esperienza utente, e sfrutta la potenza dei LLM per correggere eventuali errori di battitura o di formattazione presenti nelle informazioni recuperate tramite retrieval.

2.2. Stato dell'arte in letteratura

Negli ultimi anni, l'interesse verso l'Intelligenza Emotiva (IE) nei Large Language Model (LLM) è cresciuto in modo significativo. Sempre più ricerche si concentrano sull'analisi delle capacità di questi modelli nel comprendere e rispondere a stimoli emotivi, cercando di individuare i meccanismi con cui elaborano le informazioni legate alle emozioni umane. Studi recenti dimostrano che i LLM sono in grado di riconoscere sia la presenza di elementi emotivi nei testi, che a migliorare le proprie prestazioni quando vengono sollecitati con prompt che contengono specifiche sfumature emotive (Cheng Li et al., 2023). Questo suggerisce che l'inserimento di componenti emotive nell'interazione uomo-macchina potrebbe influenzare in modo significativo l'output generato.

Un aspetto particolarmente interessante riguarda la capacità dei LLM di rispondere a scenari di valutazione emotiva e coping[12]. Alcuni esperimenti hanno rivelato che, in determinate situazioni, le risposte fornite da questi modelli mostrano dinamiche che ricordano quelle umane. Tuttavia, permangono differenze sostanziali sia nell'intensità delle risposte, sia nel grado di sensibilità alle istruzioni ricevute (Nutchanon Yongsatianchot et al., 2023). Queste discrepanze sollevano interrogativi sulla reale profondità della comprensione emotiva dei LLM e su quanto il loro comportamento sia il risultato di un'elaborazione autentica delle emozioni o, piuttosto, una mera simulazione statistica basata sui dati con cui sono stati addestrati.

Un altro studio di rilievo ha adottato un approccio psicometrico per valutare le capacità di comprensione emotiva dei LLM. I risultati hanno mostrato che alcuni

[12] Il termine "coping", nel contesto della psicologia, si riferisce ai processi cognitivi e comportamentali che una persona utilizza per gestire, ridurre o tollerare lo stress e le difficoltà. È fondamentale per affrontare situazioni stressanti e può variare notevolmente da persona a persona.

modelli, tra cui GPT-4, hanno ottenuto punteggi di Quoziente Emotivo superiori alla media, riuscendo persino a superare numerosi partecipanti umani in determinati test di valutazione delle emozioni (Xuena Wang et al., 2023). Questo dato apre scenari interessanti, poiché suggerisce che i LLM potrebbero, in alcune circostanze, dimostrare una sensibilità alle emozioni superiore a quella di alcuni esseri umani. Tuttavia, resta da chiarire se questi risultati riflettano una reale comprensione del fenomeno emotivo o se siano semplicemente il frutto di pattern appresi durante l'addestramento.

Man mano che i LLM si integrano sempre più nelle interazioni quotidiane, diventa fondamentale comprendere a fondo le loro capacità emotive e valutarne le implicazioni. L'efficacia della comunicazione uomo-macchina, infatti, dipende anche dalla capacità dei modelli di interpretare correttamente il contesto emotivo in cui operano. Un'adeguata gestione delle componenti emotive potrebbe rendere questi sistemi più adattivi e utili in una varietà di applicazioni, che vanno dall'educazione all'assistenza psicologica, fino all'intrattenimento e alla mediazione sociale (Pinaki Raj, 2024).

Sebbene i risultati ottenuti finora siano promettenti, è evidente che vi siano ancora numerosi aspetti da esplorare. Comprendere in che modo i LLM elaborano le emozioni, se e quanto siano in grado di adattarsi alle sfumature emotive dell'interlocutore e quali siano i limiti della loro Intelligenza Emotiva, rappresenta una sfida fondamentale per il futuro della ricerca in questo campo. Questi interrogativi non solo aprono la strada a ulteriori sviluppi tecnologici, ma sollevano anche questioni etiche e metodologiche che meritano di essere affrontate con attenzione.

2.2.1. Dove si inserisce lo studio

Alla luce dei lavori esistenti, il presente studio si inserisce nel filone di ricerca sull'interazione emotiva tra esseri umani e LLM, estendendo le analisi attuali sotto due prospettive complementari. Da un lato, amplia la comprensione dell'impatto del linguaggio emotivo sui modelli linguistici, andando oltre le valutazioni basate su test psicometrici o benchmark standard. Mentre studi recenti hanno evidenziato la capacità di alcuni LLM di riconoscere e rispondere a stimoli emotivi (Cheng Li et al., 2023; Xuena Wang et al., 2023), la letteratura non ha ancora quantificato in modo sistematico come la presenza di emozioni nel prompt influenzi l'accuratezza e la qualità delle risposte. In questo contesto, il presente lavoro introduce un esperimento mirato, analizzando quattro diverse tipologie di prompt emotivi (gioia, rabbia, tristezza, apatia) e valutando le prestazioni dei modelli attraverso il SuperGLUE Benchmark. Dall'altro lato, esplora la dimensione percettiva dell'interazione uomo-LLM, considerando il modo in cui l'atteggiamento emotivo simulato della macchina influisce sull'utente. Sebbene studi precedenti abbiano analizzato l'impatto dell'interazione emotiva in contesti specifici, come chatbot terapeutici o assistenti virtuali (Nutchanon Yongsatianchot et al., 2023), manca un'analisi quantitativa che misuri gli effetti sulle prestazioni cognitive dell'utente e sulla sua esperienza soggettiva. Per colmare questa lacuna, il secondo esperimento presentato in questo lavoro valuta le prestazioni di studenti che interagiscono con un

LLM caratterizzato da diverse sfumature emotive, misurando sia l'efficacia del supporto ricevuto sia la loro percezione dell'interazione tramite scale di Likert.

Attraverso questa duplice analisi, il presente studio contribuisce a una comprensione più profonda delle dinamiche dell'interazione emotiva nei sistemi basati su LLM, con implicazioni dirette per il miglioramento dell'efficacia dei modelli in ambiti come l'educazione e l'assistenza virtuale.

2.3. Distinguere tra simulazione emotiva e reale comprensione delle emozioni

Parlando del riconoscimento e della comprensione delle emozioni da parte dei LLM si vuole fare chiarezza sulla semantica reale di quanto si afferma. Le emozioni sono complesse; non si può di certo paragonare la comprensione che un umano ha delle stesse con quella che può avere una macchina. Da un lato abbiamo un essere vivente, biologico, razionale ed emotivo, e dall'altra parte abbiamo una macchina. Un LLM utilizza trasformazioni algebriche e modelli probabilistici per apprendere autonomamente rappresentazioni concettuali a partire dal solo testo.

Queste rappresentazioni riflettono un bias fondamentale: derivano esclusivamente da correlazioni linguistiche senza un'esperienza diretta delle emozioni. Inoltre, il linguaggio stesso non sempre esprime fedelmente gli stati emotivi, poiché le emozioni possono essere influenzate da fattori non testuali, come il tono della voce o il contesto situazionale, conoscenze e assunti di base soggettivi che la macchina non ha modo di conoscere. La teoria su cui si basano i LLM è la semantica distribuzionale, una teoria linguistica che afferma che il significato di una parola può essere compreso in relazione al contesto in cui appare, piuttosto che essere legato a una definizione fissa o a un insieme di attributi intrinseci alla parola stessa. La teoria è strettamente legata al lavoro di John Firth, che nel 1957 enunciò il principio fondamentale: "You shall know a word by the company it keeps" (conoscerai una parola dalla compagnia che tiene). In altre parole, il significato di una parola non è qualcosa che può essere definito isolatamente, ma emerge attraverso le relazioni e le co-occorrenze con altre parole all'interno di un contesto linguistico. Il concetto di semantica distribuzionale si basa sull'idea che parole che compaiono in contesti simili tendano a avere significati simili. Questo approccio rifiuta l'idea che le parole abbiano significati "fissi" e definibili separatamente dalla loro situazione d'uso. Al contrario, il significato delle parole è dinamico e viene determinato dal contesto in cui si inseriscono.

Nei modelli linguistici, il significato di una parola non è rappresentato come una semplice definizione statica, ma come un insieme di relazioni probabilistiche con altre parole. In pratica, il modello impara quali termini tendono a comparire insieme e utilizza questi pattern di co-occorrenza per costruire una rappresentazione del linguaggio. Questo principio è alla base dei moderni modelli di linguaggio, come i LLM, che apprendono il significato delle parole analizzando miliardi di frasi e adattando le loro risposte in base ai contesti nei quali i termini vengono usati.

Per costruire queste rappresentazioni, si impiegano tecniche avanzate di representation learning, che consentono al modello di trasformare i concetti in una forma numerica, organizzandoli in uno spazio matematico multidimensionale. In questo spazio, parole con significati simili tendono a trovarsi vicine, mentre concetti distinti risultano più distanti. Sebbene il modo in cui il modello organizza queste

informazioni non sia immediatamente interpretabile, esistono metodi che permettono di esplorare e analizzare la struttura delle sue conoscenze.

Ad esempio, gli Sparse AutoEncoders (SAEs) sono strumenti utilizzati per individuare schemi ricorrenti nei dati e sono stati impiegati da aziende come Anthropic per analizzare come il loro modello, Claude Sonnet, organizza il sapere al suo interno. Un'altra tecnica, la Principal Component Analysis (PCA), permette di evidenziare le caratteristiche principali che differenziano le rappresentazioni di parole e concetti. Nei modelli Transformer, invece, lo studio dell'attention aiuta a comprendere quali parti di un testo influenzano maggiormente la generazione delle risposte, fornendo indizi su come il modello costruisce il proprio ragionamento.

Per rendere più accessibile questa complessa organizzazione delle informazioni, si utilizzano strumenti di visualizzazione come t-SNE e UMAP, che riducono le dimensioni dello spazio latente e permettono di rappresentare in modo intuitivo la relazione tra concetti. Grazie a queste tecniche, è possibile individuare gruppi di parole con significati affini, scoprire nuove connessioni tra idee e comprendere meglio come un modello linguistico apprende e struttura la conoscenza.

È cruciale ricordare che le rappresentazioni latenti dei LLM sono derivate esclusivamente dal testo e rappresentano approssimazioni statistiche. Sebbene possano catturare relazioni semantiche, restano prive di un'autentica comprensione delle emozioni umane. L'umano, come precedentemente detto, è un essere per metà emotivo e per metà razionale; la ragione dell'essere umano è guidata da diversi tipi di logica, che è possibile riassumere in due grandi gruppi: la logica classica e la logica fuzzy — la logica dell'incertezza. Davanti a certe condizioni, l'uomo utilizza la logica induttiva o deduttiva per giungere a conclusioni certe date le premesse. In altri scenari esso fa ricorso alla logica dell'incertezza, che opera seguendo approcci probabilistici. Il risultato del ragionamento della logica classica è guidato dalla conoscenza pregressa del mondo, e dallo stato del mondo stesso (e.g. se la temperatura è pari a 40 gradi Celsius, posso senza ombra di dubbio affermare che "è caldo"); non si può affermare la stessa cosa per i ragionamenti caratterizzati da incertezza — e.g. se lancio una moneta non truccata so che posso ottenere testa o croce, ma la conoscenza che ho del mondo non mi aiuta a sapere in anticipo il risultato del lancio. Secondo la probabilità oggettivista, ho la possibilità di ottenere testa e croce per il 50% cad.

È importante tenere a mente che il meccanismo con cui operano i LLM, anche grazie all'impiego di architetture come i Transformers, produce risultati effettuando calcoli probabilistici che possiamo immaginare come una buona approssimazione della logica fuzzy.

2.3.1. Rappresentare la conoscenza

Quando si parla di IA, è inevitabile parlare anche di conoscenza. La conoscenza è qualcosa di complesso, indubbiamente; rappresentarla significa realizzare un formalismo sufficientemente efficace per descrivere in maniera non ambigua tutto ciò che si sa del mondo, in un particolare contesto. La rappresentazione, in quest'ottica, deve essere vista come un'interfaccia che rende

possibile il trasferimento di informazioni e conoscenza tra due entità indipendenti.

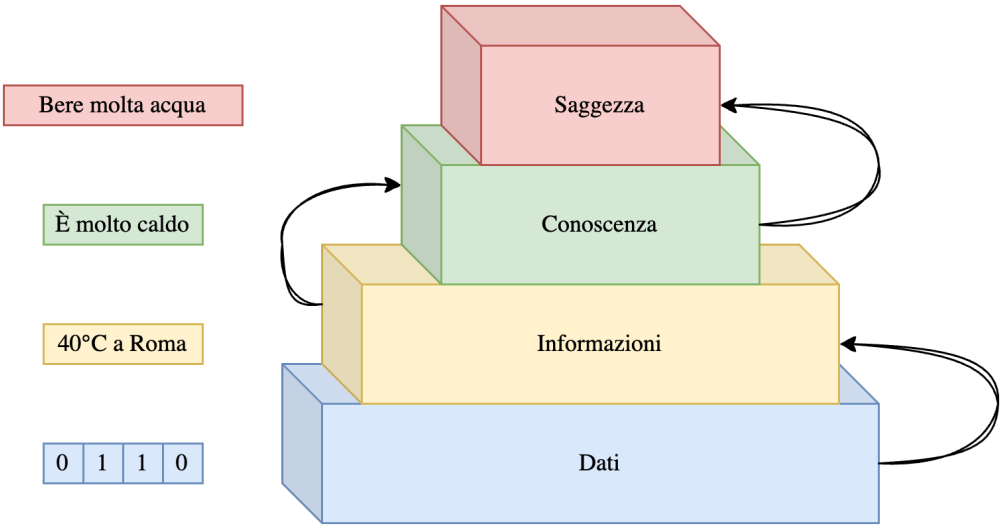


Figura 7. Esempio di catena del valore del dato.

È oltremodo utile specificare che cosa si intende per "conoscenza".

In figura è mostrato un esempio semplicistico di catena del valore del dato. La rappresentazione, in questo caso, è il livello più basso che coincide con il dato grezzo; un insieme di bit, 0 e 1, codifica il valore 40°C e la città di Roma. A seguire, tale rappresentazione viene elaborata per determinare la vera e propria informazione, "40°C a Roma". Proseguendo, si può inferire che la condizione di temperatura percepita sia "molto caldo". Infine, la saggezza consente di intraprendere azioni volte a mitigare e risolvere il problema. Ciò a cui si punta è quindi la saggezza, e ci si arriva per tramite dei semplici dati grezzi. Sistemi guidati da Intelligenza Artificiale hanno l'importante compito di agevolare questo processo, rendendo possibile l'elaborazione di una quantità esorbitante di dati in poco tempo — cosa che l'uomo, da solo, non potrebbe fare con lo stesso livello di precisione e con gli stessi tempi delle macchine.

Perché, però, parliamo di rappresentare la conoscenza? Perché umani e macchine oggigiorno si interfacciano sempre più, e noi umani tendiamo a dare per scontato che la prospettiva con cui vediamo il mondo sia identica a quella delle macchine. Non esiste preconcetto più falso di questo. La macchina a basso livello ragiona in termini di segnali elettrici veicolati da circuiti e porte logiche. Per ciò che sappiamo, l'uomo ragiona tramite connessioni neurali e segnali elaborati nel cervello, che però non possono essere discretizzati alla pari dei bit e che però sono influenzati da emozioni ed esperienza, due fattori che si traducono in soggettività. Le macchine non possiedono né emozioni e tantomeno soggettività.

Esistono due tipologie distinte di rappresentazione, che è bene descrivere: esterna e interna. Denominiamo "esterna" la rappresentazione comune anche alle entità diverse dal singolo. Nella fattispecie, la rappresentazione esterna è il linguaggio naturale. Denominiamo, invece, "interna" la rappresentazione propria e unica del singolo individuo o agente. Nessun umano possiede la medesima rappresentazione interna della conoscenza, e si può dire lo stesso delle macchine (a

meno che non siano esattamente l'una il clone dell'altra). La comunicazione verbale tra umani e tra uomo-macchina soffre di un grande deficit, che è contemporaneamente anche il suo punto di forza maggiore: la ricchezza espressiva. Sebbene questa sia una caratteristica atta ad introdurre ambiguità, è sicuramente qualcosa da difendere, da valorizzare e da salvaguardare — la soluzione al problema non può essere la costruzione di un linguaggio privo di ambiguità, perché l'umano non vive solo di informazioni, dati e logica, ma vive anche e sopratutto di sentimenti ed emozioni.

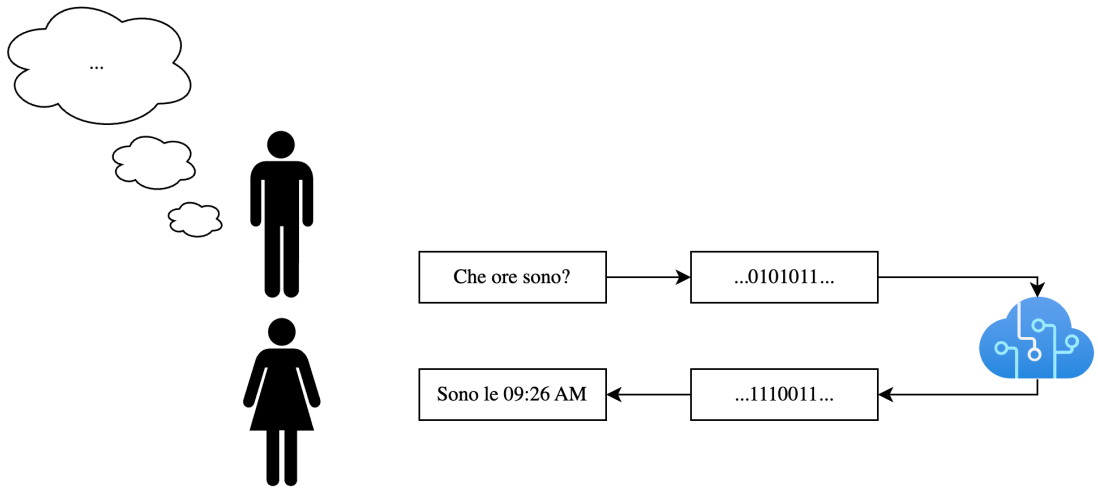


Figura 8. Esempio di flusso di comunicazione uomo-macchina.

Quando un umano comunica con la macchina, avviene il processo rappresentato nella figura 8; l'umano elabora il ragionamento in una modalità che non è possibile rappresentare[13]. Il ragionamento viene convertito in un'informazione codificata in linguaggio naturale, inviata quindi alla macchina. La macchina la recepisce e la elabora convertendola nella sua rappresentazione interna (i bit), producendo un risultato caratterizzato dalla stessa forma; a seguire, i bit della risposta vengono tradotti in linguaggio naturale e riproposti all'umano. Il linguaggio naturale viene quindi letto (o ascoltato) dall'umano che lo traduce di nuovo nella forma base dell'informazione, consentendo al cervello di elaborare l'informazione.

Tutto questo è necessario specificarlo perché le due rappresentazioni interne non possono comunicare direttamente; la macchina non ha modo di interpretare le onde e i segnali elaborati così come sono dal cervello, e viceversa.

Questa disamina enfatizza ulteriormente la necessità di comprendere l'impatto che le emozioni hanno nell'interazione uomo-macchina, poiché queste, sebbene siano comprensibili dal punto di vista umano, trovano posto nella rappresentazione interna dei LLM. Un posto che, però, non è ancora noto e i cui impatti lo sono ancor meno.

[13] Le "nuvolette" simboleggiano la rappresentazione interna dell'umano.

2.4. L'utilizzo di IA in Italia oggi

Per collocare lo studio in oggetto nel tempo, è importante raccontare e descrivere il livello di infiltrazione delle tecnologie di IA in Italia, oggi. Per farlo, si effettua una disamina e un'analisi del report "Imprese e ICT" del 2024, pubblicato dall'Istituto Nazionale di Statistica (ISTAT).

I dati principali del report ISTAT descrivono una crescita costante dell'utilizzo di applicazioni d'Intelligenza Artificiale nel Bel Paese; la quota di imprese con almeno 10 addetti che utilizza tali tecnologie cresce dal 5,0% al 8,2%, sebbene in leggero ritardo rispetto allo standard UE27 — 13,5%.

L'adozione di queste tecnologie è particolarmente concentrata nelle grandi imprese, tra le quali il 32,5% ne fa uso, in significativo aumento rispetto al 24,1% registrato nel 2023.

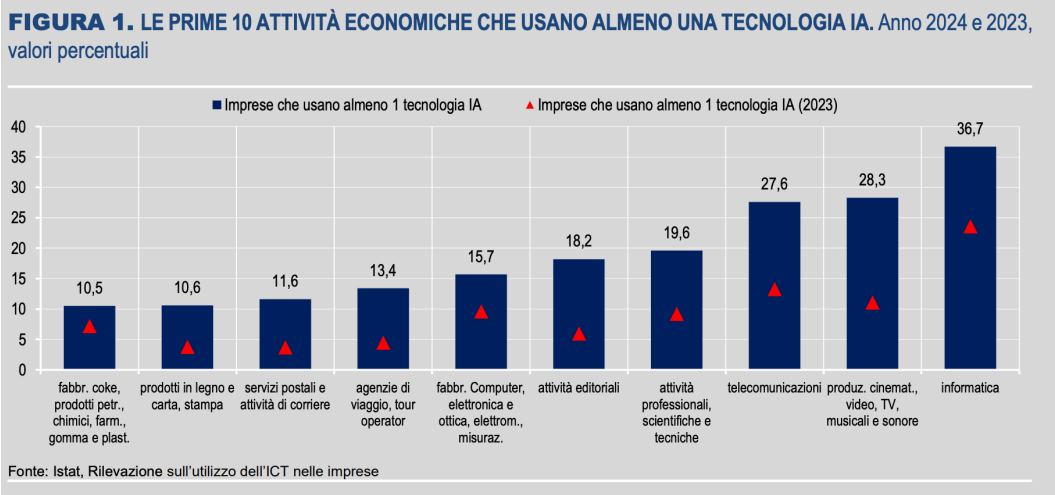


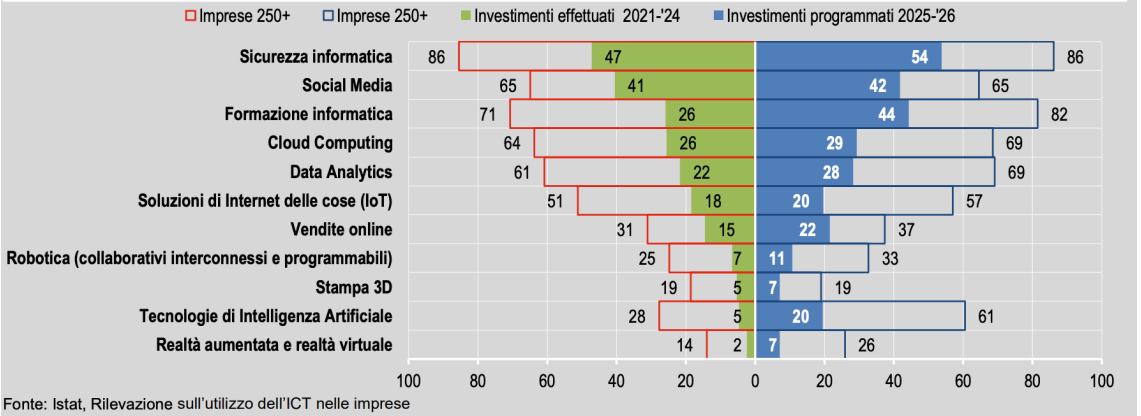
Figura 9. Le prime 10 attività economiche che fanno uso di IA (fonte: ISTAT 2024).

A livello settoriale, come evidenziato in figura 9, l'IA trova maggiore applicazione nelle imprese del comparto informatico (36,7%), nelle telecomunicazioni (27,6%) e nelle attività di produzione cinematografica, audiovisiva e musicale (28,3%). Le tecnologie più utilizzate includono strumenti per l'estrazione di conoscenza da documenti di testo (54,5%), l'IA generativa per la produzione di testo e parlato (45,3%) e il riconoscimento vocale (39,9%). In misura minore, vengono impiegate soluzioni per l'automatizzazione dei flussi di lavoro (28,1%), il riconoscimento di immagini (25,4%) e la movimentazione fisica delle macchine (10,4%). Particolarmente rilevante è la crescita dell'IA generativa, il cui utilizzo è aumentato del 163,5% rispetto all'anno precedente, evidenziando un crescente interesse verso strumenti capaci di generare contenuti testuali e multimediali in modo autonomo.

L'adozione dell'IA nelle imprese è strettamente legata a specifiche aree aziendali. Le applicazioni principali riguardano il marketing e le vendite (35,7%), la gestione amministrativa (28,2%) e le attività di ricerca e sviluppo (24,6%). In particolare, si registra un significativo incremento nell'uso dell'IA per l'ottimizzazione dei processi amministrativi (+142,5%) e per l'innovazione e la

ricerca (+98,7%). Al contrario, il ricorso all'IA per la logistica risulta in calo rispetto all'anno precedente.

FIGURA 2. AMBITI DI INVESTIMENTO DIGITALE PASSATI (2021-2024) E PROGRAMMATI (2025-2026) PER IMPRESE CON ALMENO 10 E 250 ADDETTI. Anno 2024, valori percentuali



Fonte: Istat, Rilevazione sull'utilizzo dell'ICT nelle imprese

Figura 10. Ambiti d'investimento nel contesto della transizione digitale (fonte: ISTAT 2024).

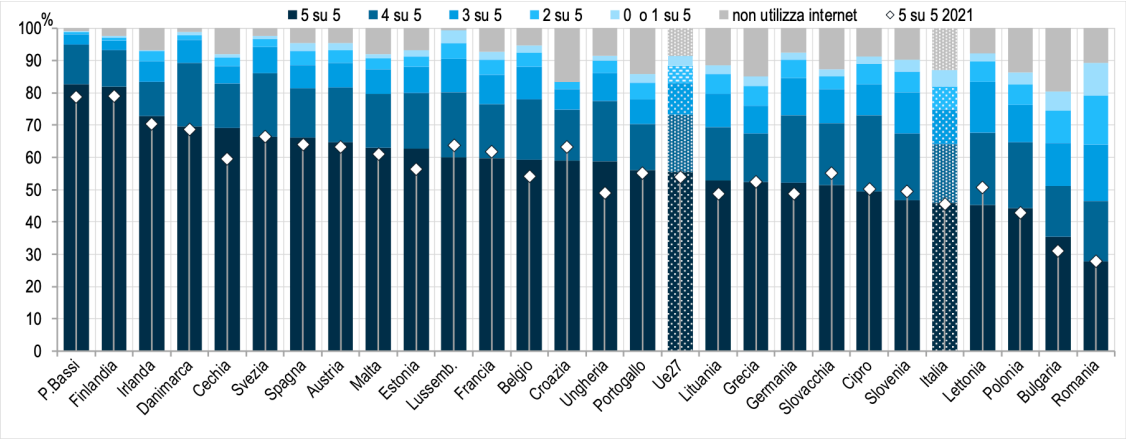
Sul fronte degli investimenti, il rapporto evidenzia un forte interesse verso l'IA nel prossimo biennio (figura 10). Un'impresa su cinque prevede di investire in queste tecnologie entro il 2026, con una netta differenza tra le imprese che già adottano soluzioni di IA e quelle che non lo fanno: il 70,3% delle imprese che utilizzano IA dichiara di voler incrementare gli investimenti in questo settore, mentre tra le imprese che non impiegano IA tale percentuale scende al 15,0%. Complessivamente, gli investimenti digitali previsti per il biennio 2025-2026 si concentrano prevalentemente sulla sicurezza informatica (53,8%), sulla formazione informatica del personale (44,3%), sull'adozione di soluzioni di cloud computing (29,3%) e, con una crescita del 15% rispetto al periodo 2021-2024, sulle tecnologie di Intelligenza Artificiale.

L'analisi evidenzia dunque una progressiva diffusione dell'IA nel contesto imprenditoriale italiano, sebbene con differenze significative tra le PMI e le grandi imprese e con un ritardo rispetto alla media europea. Il ritardo dell'Italia nell'adozione di tecnologie basate su IA, rispetto all'Europa, si configura come un fenomeno complesso; la struttura del tessuto imprenditoriale italiano, caratterizzata da una forte presenza di PMI, rappresenta un ostacolo significativo. Queste realtà, infatti, incontrano spesso difficoltà nell'investire in nuove tecnologie, a causa di risorse finanziarie limitate e di una minore propensione al rischio. A ciò si aggiunge una marcata carenza di competenze digitali, che frena l'adozione dell'IA e sottolinea la necessità di investimenti mirati nella formazione e nell'aggiornamento del personale.

2.4.1. Il ritardo dell'Italia nel panorama IA

L'Istat, nel suo approfondimento sul "Decennio digitale e capitale umano", evidenzia il ritardo dell'Italia sul fronte delle competenze digitali, un aspetto determinante per rimanere al passo con l'evoluzione tecnologica, garantendo un solido e costante sviluppo economico del Paese. Nel 2023, meno della metà della popolazione (45,9%) possedeva competenze digitali di base, collocando l'Italia al di sotto della media europea e in una posizione sfavorevole nel confronto con gli altri stati membri. Il documento sottolinea come le competenze digitali siano distribuite in modo disomogeneo nella popolazione, influenzate da fattori quali l'età, il genere e il livello di istruzione (figura 11).

FIGURA 1. PERSONE DI 16-74 ANNI PER LIVELLO DI COMPETENZA DIGITALE. Anno 2023, valori per 100 persone di 16-74 anni

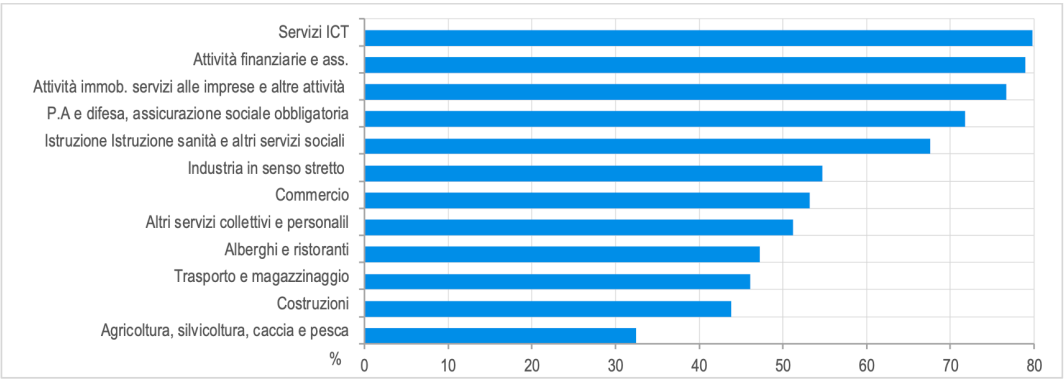


Fonte: Eurostat -Istat, Community Survey on ICT usage in households and by Individuals

Figura 11. Persone di 16-74 anni divise per livello di competenza digitale (fonte: Istat 2023).

Come è facile aspettarsi, si osserva un divario significativo tra i giovani e gli anziani, una disparità di genere (seppur a favore delle donne fino ai 44 anni) e una correlazione tra il titolo di studio e il possesso di competenze digitali.

FIGURA 2. OCCUPATI DI 16-74 ANNI CHE HANNO UTILIZZATO INTERNET NEGLI ULTIMI 3 MESI CON COMPETENZE DIGITALI ALMENO DI BASE E PER SETTORE DI ATTIVITÀ ECONOMICA. Anno 2023, valori per 100 persone di 16 -74 anni con le stesse caratteristiche



Fonte: Eurostat -Istat, Community Survey on ICT usage in households and by Individuals

Figura 12. Occupati che utilizzano Internet divisi per età (fonte: Istat 2023).

Anche nel mondo del lavoro emergono delle criticità. Nonostante che la diffusione delle competenze digitali sia più elevata tra gli occupati rispetto ai disoccupati, l'Italia si colloca al di sotto della media europea anche in questo caso. Si evidenziano differenze significative anche tra i vari settori economici; in questo contesto, ICT, Fintech e Insurtech mostrano i livelli più alti di competenze digitali, mentre l'agricoltura, la silvicoltura, la pesca e le costruzioni registrano i valori più bassi (figura 12). L'Istat rileva, inoltre, un aumento degli specialisti ICT in Italia negli ultimi anni, ma con una crescita inferiore rispetto alla media europea. Permangono alcune debolezze strutturali, come la bassa diffusione degli specialisti ICT nelle imprese, la ridotta quota di specialisti con titolo universitario e la scarsa presenza femminile nel settore. Sul fronte della formazione, si evidenzia un miglioramento nell'erogazione di formazione ICT da parte delle imprese italiane, sebbene il livello di partenza sia inferiore alla media europea.

3. Interazione emotiva uomo-macchina

Lo studio posto in essere si propone di quantificare l'impatto delle emozioni nell'interazione uomo-macchina, in entrambe le direzioni: come reagiscono le macchine alla presenza di emotività nell'interazione con l'umano e come reagiscono gli umani alla presenza di emotività nelle risposte ricevute dai LLM (introdotti ed illustrati nella sezione 2.1.3) con cui interagiscono.

Lo scopo finale è quello di verificare se esiste un impatto e, se c'è, quanto impatta nelle prestazioni della macchina e dell'uomo; inoltre, ci si chiede se l'impatto è il medesimo nei due casi, e se quindi si può ipotizzare l'esistenza di un trend monotonico, valevole in entrambi i sensi dell'interazione.

Lo studio è organizzato in due sezioni verticali, trattate come due esperimenti distinti, denominati Alpha (studio dell'impatto delle emozioni sulla macchina) e Omega (duale, studio dell'impatto delle emozioni sull'umano).

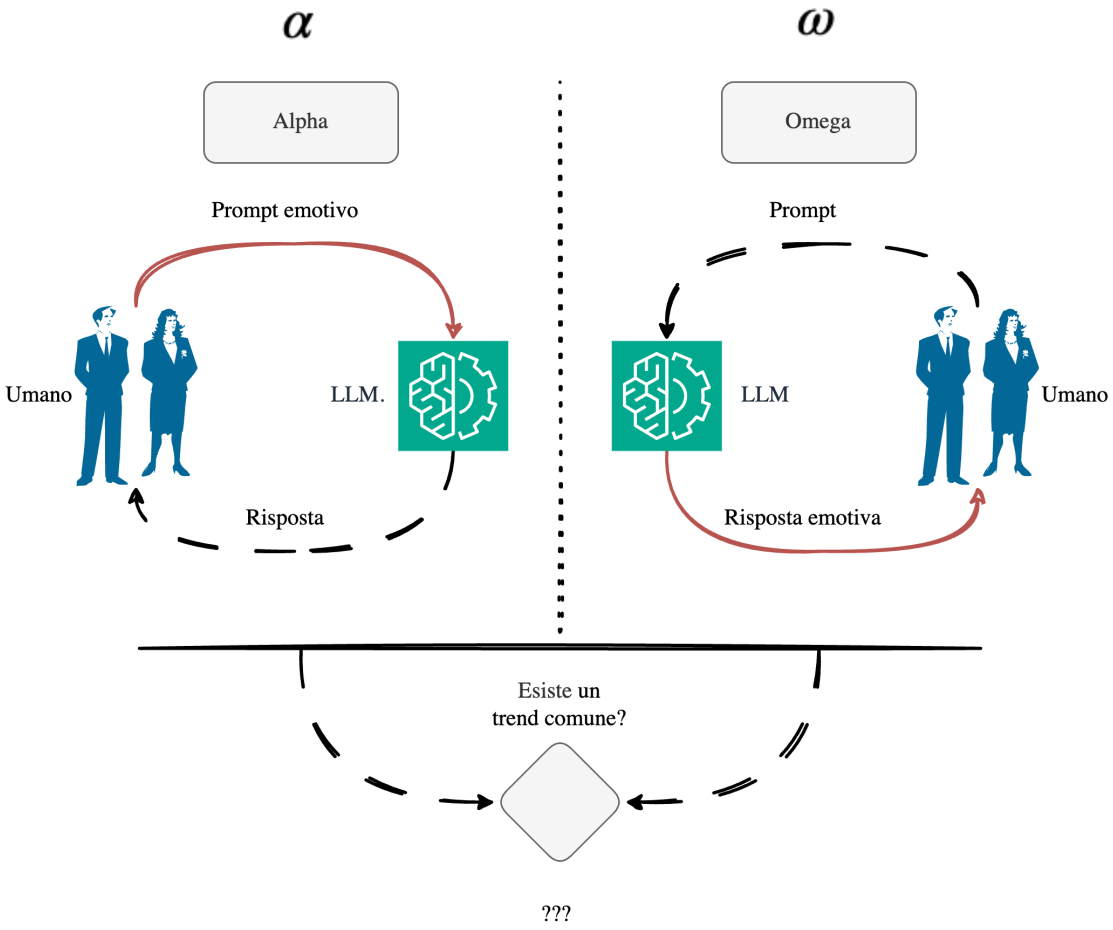


Figura 13. Interazione uomo-macchina, nelle due versioni dello studio.

Lo studio Alpha si propone di simulare delle richieste umane influenzate da emozioni di quattro diverse nature — gioia, rabbia, paura e apatia — mentre Omega si propone di studiare l'interazione uomo-macchina con quattro varianti di LLM

addestrate per rispondere manifestando rispettivamente gioia, rabbia, paura e apatia. I risultati di Alpha e Omega vengono raccolti e analizzati con l'ausilio della statistica, allo scopo di identificare il trend. Nella figura 13 è schematizzato concettualmente lo studio; in rosso è indicata la direzione in cui viene inserita l'emozione nell'interazione.

3.1. Alpha: studio dell'impatto delle emozioni sulla macchina

L'esperimento Alpha ha l'obiettivo di analizzare l'influenza delle emozioni (gioia, rabbia, paura e apatia) sulle risposte di modelli di linguaggio di grandi dimensioni (LLM) open-weights. A tal fine, vengono predisposti una serie di prompt, ciascuno caratterizzato da una delle quattro emozioni, e somministrati a modelli con architetture, famiglie di prodotto e numero di parametri differenti. I risultati ottenuti vengono valutati tramite metriche appropriate e analizzati con strumenti statistici per individuare eventuali tendenze.

L'esperimento si basa sul benchmark SuperGLUE (Super General Language Understanding Evaluation), una suite di test progettata per misurare le capacità di comprensione del linguaggio naturale dei modelli di Intelligenza Artificiale. SuperGLUE comprende otto task diversi di NLP[14], ognuno costruito su un dataset distinto. Tra questi, è stato selezionato un sottoinsieme di sei task, ritenuti particolarmente adatti allo studio.

Per ciascun task, sono state generate quattro varianti di prompt, corrispondenti alle quattro emozioni analizzate (apatia, gioia, rabbia, paura). Ogni record di ciascun dataset viene processato secondo tutte e quattro le varianti, producendo un numero equivalente di risultati con la stessa struttura. Il numero totale delle casistiche di test è quindi data da:

$$E = \{\text{apatia}, \text{rabbia}, \text{paura}, \text{gioia}\}$$

$$S = \sum_{d=1}^{n} \sum_{i=1}^{r} \sum_{e \in E} c_{di}^{e}$$

Dove:

- E è l'insieme delle quattro emozioni.
- S è il numero totale degli esperimenti.
- n è il numero dei dataset (6).
- r è il numero del dataset d, in oggetto.
- i è il record i-esimo del dataset d.
- e è l'emozione.
- c_{di}^{e} rappresenta la singola casistica del record i-esimo del dataset d processata con l'emozione e.

A valle dell'esperimento, si procede a effettuare una valutazione utilizzando le metriche messe a disposizione dallo stesso benchmark. Una volta raccolti i risultati, si calcolano le statistiche descrittive e i test statistici opportuni, utili a validare quanto

[14] Descritti nel dettaglio nel capitolo 4.

ottenuto. Infine, si commentano e si analizzano le statistiche prodotte al fine di individuare, se presente, un trend emergente tra tutti i modelli.

3.1.1. Il benchmark SuperGLUE

SuperGLUE (Super General Language Understanding Evaluation) è un benchmark avanzato sviluppato per valutare la capacità dei modelli di IA nella comprensione del linguaggio naturale. Introdotto nel 2019 come successore di GLUE (General Language Understanding Evaluation), è stato progettato per affrontare task più complessi, che richiedono inferenza, ragionamento e comprensione contestuale avanzata.

Il benchmark è composto da otto task distinti, ciascuno basato su dataset specifici e mirato alla valutazione di una particolare abilità linguistica:

- BoolQ (Boolean Questions): valuta la capacità del modello di rispondere a domande a risposta binaria basate su un testo.
- CB (CommitmentBank): misura il livello di certezza di un'affermazione nel suo contesto.
- COPA (Choice of Plausible Alternatives): richiede di identificare la causa o la conseguenza più plausibile in una relazione causale.
- MultiRC (Multi-Sentence Reading Comprehension): verifica la comprensione del testo attraverso domande a risposta multipla.
- ReCoRD (Reading Comprehension with Commonsense Reasoning Dataset): valuta l'abilità inferenziale basata sul senso comune.
- RTE (Recognizing Textual Entailment): determina se una frase (premessa) implica o meno un'altra (ipotesi).
- WiC (Word-in-Context): analizza la capacità di disambiguare il significato di una parola in due contesti diversi.
- WSC (Winograd Schema Challenge): valuta la risoluzione di coreferenze basata su conoscenze semantiche e contestuali.

SuperGLUE è stato selezionato come benchmark di riferimento per questo studio grazie alla sua capacità di testare le reali abilità inferenziali dei modelli linguistici, a differenza di GLUE, che si concentra su compiti più semplici di classificazione e inferenza testuale. I suoi task mettono alla prova la comprensione profonda del linguaggio, un elemento essenziale per analizzare l'impatto delle emozioni sulle prestazioni degli LLM.

In particolare, task come Winograd Schema Challenge (WSC) e Commitment Bank (CB) valutano la capacità del modello di interpretare relazioni contestuali e disambiguare riferimenti testuali, due aspetti fortemente influenzabili dalla variazione emotiva nei prompt. Allo stesso modo, COPA richiede un ragionamento causale, una competenza in cui il tono emotivo di un input potrebbe giocare un ruolo chiave nel guidare le scelte del modello.

Altri benchmark alternativi, come MMLU (Massive Multitask Language Understanding) e HELM (Holistic Evaluation of Language Models), sono stati considerati, ma risultano meno adatti al tipo di analisi proposta; MMLU si concentra prevalentemente su domande a scelta multipla e retrieval, mentre HELM è orientato verso aspetti etici e bias nei modelli.

SuperGLUE, al contrario, rappresenta un equilibrio ideale tra complessità, diversità dei task e interpretabilità dei risultati. La sua struttura consente di testare i modelli in scenari realistici, simulando interazioni linguistiche complesse in cui le emozioni possono avere un impatto significativo sulle risposte.

3.1.2. Ambiente di esecuzione

L'architettura di runtime dell'esperimento è basata su tre componenti principali: un'applicazione Python a riga di comando (CLI), un'istanza di Ollama per l'esecuzione dei modelli di intelligenza artificiale e un database MongoDB per la memorizzazione dei risultati. Questa configurazione è stata scelta per garantire un ambiente di esecuzione efficiente e strutturato, in grado di gestire sia la generazione dei risultati che la loro memorizzazione in modo scalabile.

L'applicazione Python CLI rappresenta il principale punto di interazione tra l'utente — in questo caso il tester — e l'esperimento. Trattandosi di un'applicazione a riga di comando, non dispone di un'interfaccia grafica, ma permette di avviare e controllare il flusso di esecuzione attraverso comandi testuali. Questo approccio semplifica l'integrazione con altri strumenti e consente di automatizzare i processi di generazione ed elaborazione dei prompt. L'applicazione è sviluppata in Python, un linguaggio di programmazione particolarmente diffuso nel campo della ricerca e del Machine Learning, grazie alla sua sintassi intuitiva e alla disponibilità di numerose librerie specializzate. L'uso di Python facilita l'interazione con i LLM e consente di implementare strumenti di analisi e monitoraggio delle prestazioni con relativa semplicità.

Ollama e MongoDB sono due componenti fondamentali dell'architettura di runtime dell'esperimento Alpha, ciascuno con un ruolo specifico e complementare. Data l'importanza di questi due componenti, è opportuno spendere qualche parola per dettagliare, almeno ad alto livello, il relativo funzionamento.

Ollama è una piattaforma progettata per eseguire modelli di linguaggio di grandi dimensioni (LLM) direttamente su hardware locale, senza la necessità di dipendere da infrastrutture cloud o server remoti. Il suo funzionamento si basa su un motore ottimizzato per l'esecuzione di modelli open-weights[15], che sfrutta l'accelerazione hardware disponibile sul dispositivo, come GPU o unità di calcolo avanzate (TPU o NPU nei sistemi più recenti).

Il flusso di esecuzione di un modello su Ollama avviene tipicamente attraverso i seguenti passaggi:

1. Caricamento del Modello: quando l'utente invia una richiesta di esecuzione, Ollama carica il modello richiesto dalla memoria di sistema o lo scarica da repository open-source se non è già disponibile localmente.
2. Pre-elaborazione dell'Input: prima dell'elaborazione, il prompt viene convertito in una rappresentazione numerica attraverso una fase di tokenizzazione. Questo processo trasforma il testo in sequenze di token

[15] I LLM open-weights sono modelli di linguaggio utilizzabili in ottica open-source. Si parla di "open-weight" e non "open-source" perché di questi modelli sono resi disponibili solamente i pesi. Il codice sorgente, così come i dati utilizzati per il relativo addestramento, non sono disponibili alla pari del modello stesso.

comprensibili dal modello, utilizzando un vocabolario predefinito associato all'architettura del modello stesso.

3. Inferenza del Modello: il core di Ollama esegue l'inferenza elaborando i token attraverso la rete neurale del modello selezionato. Questo passaggio sfrutta tecniche avanzate di ottimizzazione, come la quantizzazione[16], per migliorare le prestazioni e ridurre il consumo di memoria.
4. Post-elaborazione dell'Output: i token generati dal modello vengono riconvertiti in testo leggibile. Questa fase include la gestione di parametri come la temperatura (che regola la casualità delle risposte) e il top-k/top-p sampling (che modula la selezione dei token più probabili). L'architettura citata è riassunta dallo schema in figura 14.

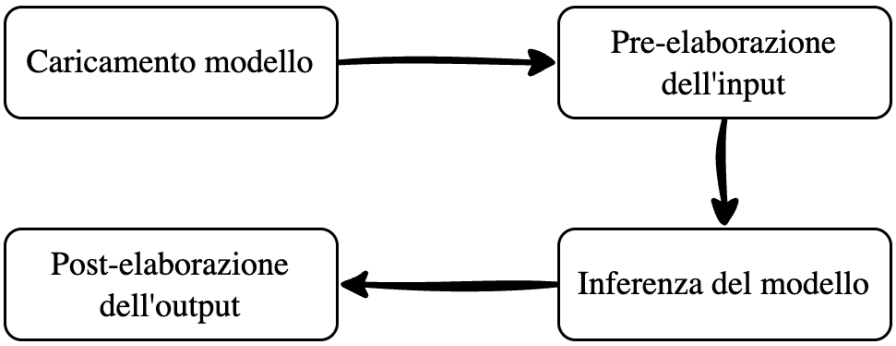


Figura 14. Schema delle quattro fasi di funzionamento di Ollama.

Grazie a questa architettura, Ollama consente di eseguire modelli LLM su dispositivi comuni, offrendo un equilibrio tra prestazioni, efficienza computazionale e privacy, evitando la necessità di inviare dati a server esterni.

MongoDB è un database NoSQL ottimizzato per la gestione di dati non strutturati o semi-strutturati. A differenza dei database relazionali tradizionali, che organizzano le informazioni in tabelle con schemi rigidi, MongoDB utilizza un approccio document-oriented, in cui i dati vengono memorizzati sotto forma di documenti JSON (o BSON, la sua versione binaria ottimizzata). Questa flessibilità consente di archiviare dati con strutture variabili e di eseguire interrogazioni efficienti anche su dataset di grandi dimensioni.

Il funzionamento di MongoDB può essere suddiviso nelle seguenti fasi principali:

1. Memorizzazione dei Dati: ogni record nel database viene archiviato come un documento JSON, che può contenere campi annidati e strutture complesse. Rispetto ai database relazionali classici, dove la struttura è rigida e poco scalabile, l'approccio a documenti basati sul formato JSON agevola la conduzione dell'esperimento. L'impiego di MongoDB permette di memorizzare

[16] La quantizzazione è una tecnica di ottimizzazione che riduce la precisione numerica dei pesi e degli attivazioni di un modello di rete neurale (ad esempio, passando da 32-bit a 8-bit), diminuendo così il consumo di memoria e migliorando la velocità di inferenza.

ogni prompt elaborato insieme ai risultati prodotti dal modello, senza dover adattare i dati a una struttura tabellare rigida.

2. Indicizzazione e Querying: MongoDB supporta un potente motore di indicizzazione che permette di recuperare rapidamente le informazioni desiderate. È possibile eseguire query avanzate utilizzando operatori di ricerca su testo, filtri su intervalli numerici o combinazioni logiche complesse. Questo permette di interrogare i risultati dell'esperimento in modo efficiente, filtrando per modello, emozione o altre variabili, semplificando in toto la fase di EDA (Exploratory Data Analysis).

3. Replica e Scalabilità: una delle caratteristiche chiave di MongoDB è la possibilità di distribuire i dati su più nodi, garantendo sia la ridondanza che la scalabilità orizzontale (per gestire carichi di lavoro crescenti). Sebbene l'esperimento utilizzi un'istanza locale e non abbia bisogno di particolari accorgimenti tecnici, MongoDB può essere facilmente esteso per supportare scenari multi-nodo, rendendolo adatto anche a esperimenti su larga scala. L'impiego di MongoDB, unitamente alle altre tecnologie open-source, è indubbiamente utile per favorire la replicabilità dei risultati e la portabilità della metodologia di analisi.

L'utilizzo di MongoDB per la memorizzazione dei risultati dell'esperimento garantisce un accesso rapido ai dati e una gestione flessibile delle informazioni. Grazie alla sua capacità di gestire strutture dati dinamiche e alla facilità con cui permette di eseguire analisi su grandi volumi di dati, MongoDB è stato considerato essere l'ideale per conservare e analizzare i risultati generati dai LLM coinvolti nello studio.

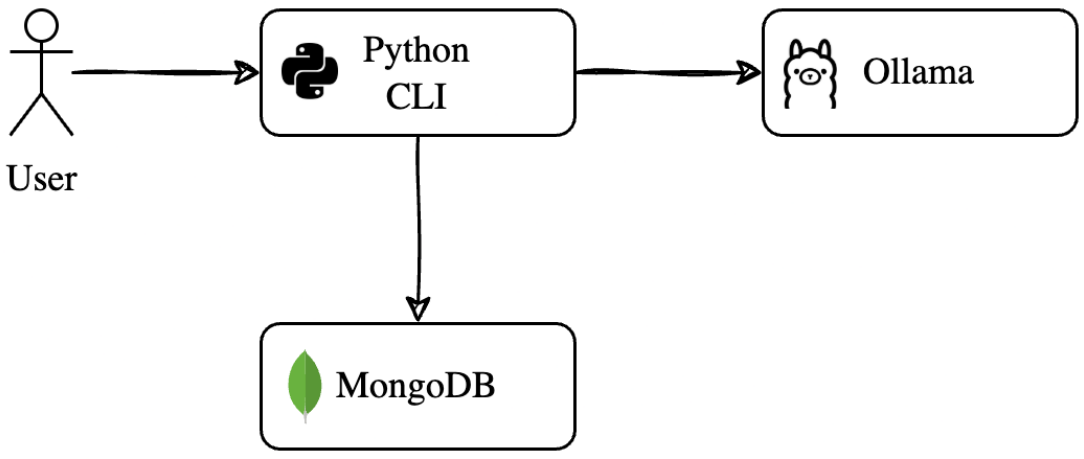


Figura 15. Panoramica ad alto livello dell'architettura dell'esperimento.

Le interazioni tra le tre componenti sopra descritte seguono uno schema ben definito, schematizzato nella figura 15: l'utente avvia l'applicazione Python CLI, che a sua volta invia i prompt predefiniti all'istanza di Ollama per l'esecuzione. I risultati ottenuti vengono quindi salvati sull'istanza MongoDB locale.

Questa modalità di esecuzione del flusso di lavoro garantisce che ogni fase dell'esperimento sia tracciabile e riproducibile, facilitando l'analisi dei risultati.

3.1.3. I modelli utilizzati

I modelli utilizzati nell'esperimento Alpha sono tutti open-weights: LLama 3.1 8B, LLama 3.2 2B, Qwen 2.5 8B, Gemma 2 9B, e Mistral 7B. La scelta di utilizzare modelli di questa natura è motivata dalla riproducibilità dei risultati e dalla limitazione dei costi associati all'esecuzione dell'esperimento stesso. L'uso di modelli open consente a qualsiasi ricercatore di replicare gli esperimenti, assicurando che i risultati possano essere verificati e validati indipendentemente.

Per ciascun modello, sono state utilizzate le versioni "Instruct", ovvero addestrate per ricevere istruzioni, e non per il semplice "completamento del testo". Queste versioni sono progettate per comprendere e rispondere a comandi specifici, rendendole particolarmente adatte per compiti che richiedono l'interazione con l'utente attraverso prompt strutturati. La capacità di seguire istruzioni è cruciale per valutare come i modelli rispondono a input che includono sfumature emotive; in caso contrario, è ragionevole aspettarsi un degrado prestazionale che potrebbe inquinare i risultati, invalidandoli.

Tutti i modelli sono quantizzati a 4 bit, allo scopo di ridurre significativamente il consumo di memoria, rendendo i modelli eseguibili su hardware locale, come i computer portatili o i dispositivi con risorse limitate. Sebbene la quantizzazione possa introdurre una leggera degradazione della qualità delle risposte, essa permette di eseguire esperimenti su larga scala senza la necessità di hardware specializzato e costoso. Si è quindi ritenuto ragionevole accettare il compromesso a favore dell'esecuzione dell'esperimento con costi contenuti.

La selezione dei modelli ha incluso famiglie diverse, come Qwen, Gemma, LLama, e Mistral. Questa diversità non è casuale: l'obiettivo è verificare se il trend osservato nei risultati dipende dalla famiglia, e quindi dall'architettura, dei modelli. Ogni famiglia di modelli può avere caratteristiche uniche che influenzano le prestazioni in compiti specifici — sia per l'architettura stessa o per i dati utilizzati durante la fase di training (o entrambe). Analizzando i risultati tra diverse famiglie, è possibile determinare se esiste una correlazione tra l'architettura del modello e la sua capacità di gestire input emotivi. Questo approccio permette di esplorare non solo le prestazioni individuali dei modelli, ma anche di identificare eventuali pattern o tendenze che potrebbero emergere tra diverse architetture di modelli linguistici.

Nello specifico, le versioni dei modelli utilizzate su Ollama sono elencate nella tabella 1.

Modello	**Versione**	**Quantizzazione**	**Tag**
Mistral	7B Instruct	Q4	mistral:7b-instruct-q4_0 (b17615239298)
LLama	3.1 8B Instruct	Q4	llama3.1:8b-instruct-q4_0 (42182419e950)
Qwen	2.5 7B Instruct	Q4	qwen2.5:7b-instruct-q4_0 (2e92ac0dd3a8)
Gemma 2	9B Instruct	Q4	gemma2:9b-instruct-q4_0 (ff02c3702f32)

Modello	Versione	Quantizzazione	Tag
LLama	3.2 3B Instruct	Q4	llama3.2:3b-instruct-q4_0 (9b9453afbdd6)

Tabella 1. I modelli utilizzati.

3.2. Omega: studio dell'impatto delle emozioni sull'uomo

L'esperimento Omega è stato progettato per indagare l'impatto delle risposte emotivamente connotate fornite da modelli di intelligenza artificiale sulla capacità di apprendimento e sulla percezione dell'interazione. Per questo studio, è stata sviluppata un'applicazione web prototipale basata su quattro varianti di un modello LLM, ottenute tramite fine-tuning di Velvet-14B, integrate con una fase di RAG (Retrieval Augmented Generation) per il recupero di contenuti pertinenti alle domande degli utenti.

L'esperimento si svolge in un contesto universitario, con una lezione complessa a tema Intelligenza Artificiale seguita da un gruppo di studenti. Gli studenti vengono suddivisi in quattro gruppi, ciascuno associato a una variante emotiva del modello (gioia, rabbia, paura, apatia). Dopo la lezione, viene sottoposto un test di verifica della comprensione, durante il quale gli studenti possono interagire con la variante assegnata di Velvet-14B per ottenere supporto. Successivamente, gli studenti compilano un questionario basato su una scala di Likert, fornendo dati qualitativi sulla loro percezione dell'interazione con il modello. L'obiettivo finale dell'esperimento è verificare se le emozioni integrate nei modelli influenzano sistematicamente le loro performance e se l'uso di risposte emotivamente connotate modifica la percezione e la comprensione degli studenti. Il flusso logico dell'esperimento è riassunto dallo schema seguente.

Dal punto di vista architetturale, l'esperimento Omega si basa su quattro varianti di Velvet-14B (comprensiva di baseline), ciascuna adattata a un'emozione specifica, e integrate in un'applicazione web che funge da chatbot, con risposta generata in streaming, similmente a ChatGPT. A supporto del chatbot, è stato implementato un sistema RAG basato su Milvus come vector database, nel quale vengono memorizzati i chunk (frammenti) delle trascrizioni delle lezioni. Questi contenuti sono preprocessati per rimuovere errori della trascrizione automatica e sintetizzare le informazioni chiave.

Durante il test, gli studenti interagiscono con il chatbot emotivamente connotato per rispondere alle domande relative alla lezione. L'applicazione viene quindi utilizzata in supporto all'attività di studio delle lezioni universitarie della prof.ssa Francesca Fallucchi. Tutti i dati di utilizzo vengono raccolti in modo anonimo tramite logging sul backend e successivamente analizzati per comprendere il ruolo delle emozioni nella performance e nella percezione dell'apprendimento mediato dall'IA. La figura 16 viene proposta per riassumere l'organizzazione di Omega.

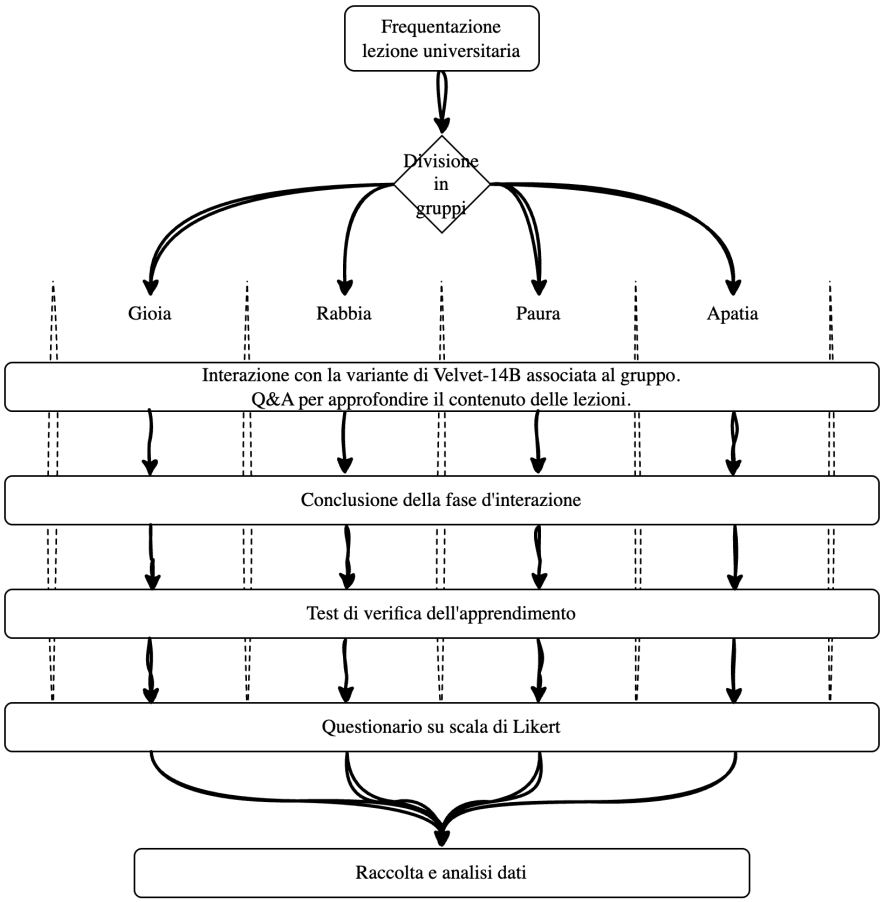


Figura 16. Schema riassuntivo dell'esperimento Omega.

3.2.1. Fine tuning di Velvet-14B

Per eseguire l'esperimento Omega è stato necessario applicare la tecnica del fine-tuning a Velvet-14B. La potenza del fine-tuning consiste nell'adattare un modello già addestrato su un compito generale per specializzarlo su un compito specifico. In questo caso, l'obiettivo era introdurre dei bias emotivi nelle risposte del modello, creando quattro versioni che riflettessero le emozioni di gioia, rabbia, paura e apatia (quest'ultima come baseline, cioè il modello senza emozioni). La variante apatica non è stata addestrata, si è scelto di considerare la versione base di Velvet-14B come "apatica"; la declinazione di "apatia", in questo caso, significa distinguere dalle varianti emotion-biased e la versione base, che invece non risente di nessun bias indotto.

Il dataset di partenza utilizzato per il fine-tuning è MELD (Multimodal EmotionLines Dataset), che contiene trascrizioni di dialoghi dalla serie TV Friends. Nel dataset, ogni battuta di dialogo è etichettata con l'emozione predominante, facilitando così l'assegnazione dei dati alle rispettive categorie emotive (gioia, rabbia, paura e apatia). Questo dataset, sebbene utile, non era sufficiente per un fine-tuning

efficace del modello a causa della quantità limitata di dati per ciascuna emozione, quindi è stato necessario applicare una fase di data augmentation[17].

La data augmentation, in questo contesto, significa creare nuovi esempi di conversazione per arricchire il dataset. Per farlo, è stato utilizzato Gemma 2 9B, un LLM distribuito tramite la piattaforma Ollama. Il processo di augmentation seguiva un determinato schema: veniva selezionata un'utterance (ovvero una singola battuta di dialogo) dal dataset MELD, che conteneva l'emozione specifica, e Gemma 2 9B generava una conversazione composta da 10 utterance, coerenti con l'emozione selezionata. La temperatura[18] è stata impostata a 0,8 per ottenere risposte che non fossero troppo rigide, ma nemmeno completamente casuali. Il risultato veniva quindi valutato in base alla coerenza con l'emozione target. Sebbene Gemma 2 9B non abbia sempre replicato esattamente lo stile linguistico di MELD, le conversazioni generate sono state considerate sufficientemente buone per il fine-tuning, e sono state ritenute utili per l'addestramento del modello.

Poiché i dati originali di MELD erano in inglese, tutte le conversazioni generate sono state tradotte in italiano utilizzando lo stesso modello Gemma 2 9B. Nonostante alcune traduzioni fossero troppo letterali, un'analisi esplorativa dei dati (EDA, Exploratory Data Analysis) su campioni casuali, eseguita tramite notèebook Jupyter, ha confermato che i testi erano verosimili e quindi utilizzabili per il training. Il dataset finale, dopo l'augmentation e la traduzione, consisteva in 300 conversazioni da 10 turni di dialogo per ogni emozione, per un totale di 1.200 conversazioni e 12.000 utterance. La scelta di limitare ogni conversazione a 10 turni è stata fatta per mantenere coerenza contestuale senza amplificare troppo il bias emotivo introdotto dalle conversazioni generate.

Per misurare l'impatto delle emozioni sul comportamento del modello, è stato eseguito un benchmark comparativo. In pratica, è stata utilizzata una batteria di 100 prompt generali (domande non specializzate) per confrontare le risposte fornite dalla versione base di Velvet-14B e quelle delle varianti emotive. Le risposte ottenute sono state poi analizzate manualmente per classificare le varianti come "emotion-biased", distinguendo tra quelle con bias di gioia, paura o rabbia, e verificando così l'efficacia del fine-tuning nel produrre risposte influenzate emotivamente.

3.2.2. Applicazione web

Per lo studio è stata sviluppata e progettata un'applicazione web costituita da tre componenti principali che fanno uso di tecnologia open-source eterogenea. Di seguito, si vuole fornire una panoramica ad alto livello dell'architettura e delle interazioni ad alto livello.

---

[17] Processo con cui si aumenta artificialmente la dimensione del dataset utilizzato in sede di training di un qualsiasi modello di IA. La tecnica in questione si applica sui dataset, quindi è applicabile a prescindere dalla natura del processo volto a consumare i dati stessi (e.g. non si usa solo nel training di LLM).

[18] La temperatura è un iper-parametro che consente di controllare la variabilità della predizione del token successivo da parte di un LLM. Una temperatura alta tende a penalizzare i token con percentuale più alta e a favorire quelli con percentuale più bassa, aumentando la variabilità della risposta. Tipicamente, l'intervallo della temperatura è posto tra [0,2]; anche se, matematicamente, si potrebbe ampliare oltre, si sconsiglia di farlo per non produrre testo senza senso.

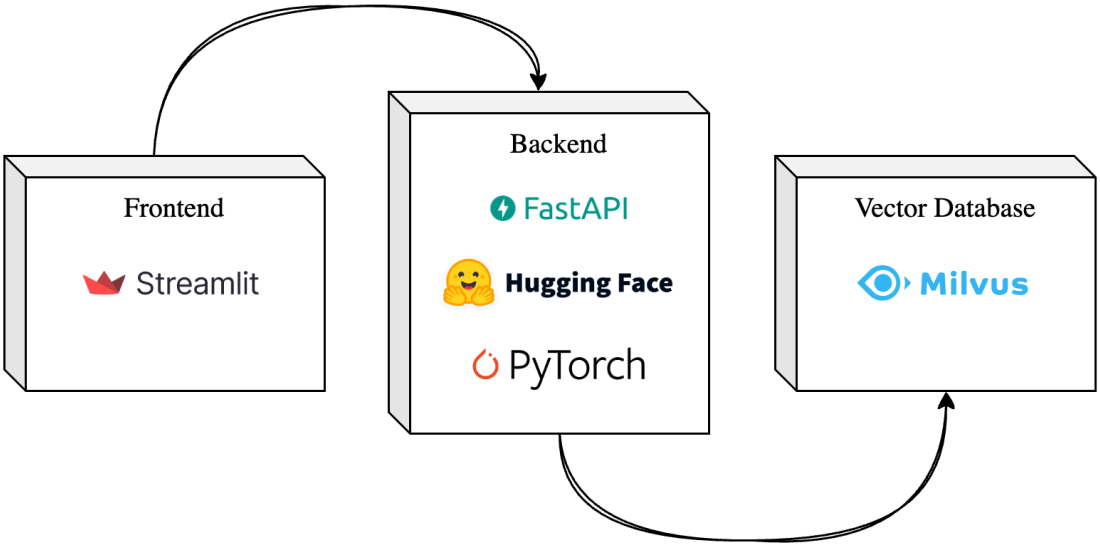


Figura 17. Schema di alto livello che riassume le componenti dell'architettura software.

Per comprendere il funzionamento dell'esperimento Omega, è utile descrivere l'architettura del sistema, composta da tre componenti principali schematizzate in figura 17: un frontend, un backend e un database vettoriale che supporta il meccanismo di Retrieval Augmented Generation (RAG). Il frontend è sviluppato in Python utilizzando Streamlit, un framework open-source che permette di costruire interfacce grafiche utente in Python. Streamlit converte automaticamente il codice Python in componenti React.js[19], facilitando così lo sviluppo e la gestione dell'interfaccia senza dover scrivere direttamente codice JavaScript.

Il backend, che gestisce la logica applicativa, comunica con il frontend attraverso un'API REST esposta dal framework FastAPI. L'API consente di erogare funzionalità di chat in modalità streaming, rispondendo un token alla volta, simulando così l'esperienza utente tipica di ChatGPT.

Per il recupero delle informazioni pertinenti alle domande dell'utente, il backend si affida a un database vettoriale, Milvus, che gestisce la ricerca dei contenuti delle lezioni. I contenuti e le interazioni utente sono trasformati in embedding (rappresentazioni numeriche di alto livello) tramite il modello "intfloat/multilingual-e5-base", un modello di embedding multilingue open-source di Microsoft. Questi embedding vengono poi utilizzati per calcolare la cosine similarity tra la domanda dell'utente e i contenuti delle lezioni, permettendo così di recuperare le risposte più pertinenti.

L'inferenza, ovvero il processo di generazione delle risposte, è gestita dal modello Velvet-14B (e dalle sue varianti create con il fine-tuning) e viene eseguita localmente tramite le API di Hugging Face e PyTorch, che offrono le necessarie funzionalità per caricare e inferire i modelli di linguaggio.

[19] React.js è una tecnologia di sviluppo software frontend costruita sopra a JavaScript.

Il funzionamento del sistema di Retrieval Augmented Generation (RAG) si basa su una pipeline di retrieval strutturata in due fasi. Nella prima fase, viene effettuata una ricerca tramite cosine similarity sui dati archiviati in Milvus. Per completezza, si riporta la definizione formale del coseno di similitudine, ovvero il rapporto tra il prodotto scalare di due tensori, A e B, e il prodotto delle relative norme.

$$\text{Cosine Similarity} = \frac{A \cdot B}{||A| \times |B||}$$

La formalizzazione è stata riportata per chiarire il motivo per cui tale funzione è stata impiegata. Le rappresentazioni tensoriali delle frasi ottenute tramite "intfloat/multilingual-e5" consistono di vettori densi. Non essendo vettori sparsi, il denominatore non si annulla mai e, di conseguenza, la funzione rimane definita su tutto lo spazio latente. Nella figura 18 si propone una visualizzazione che aiuta a comprendere l'operatività del coseno di similitudine, divisa in passaggi.

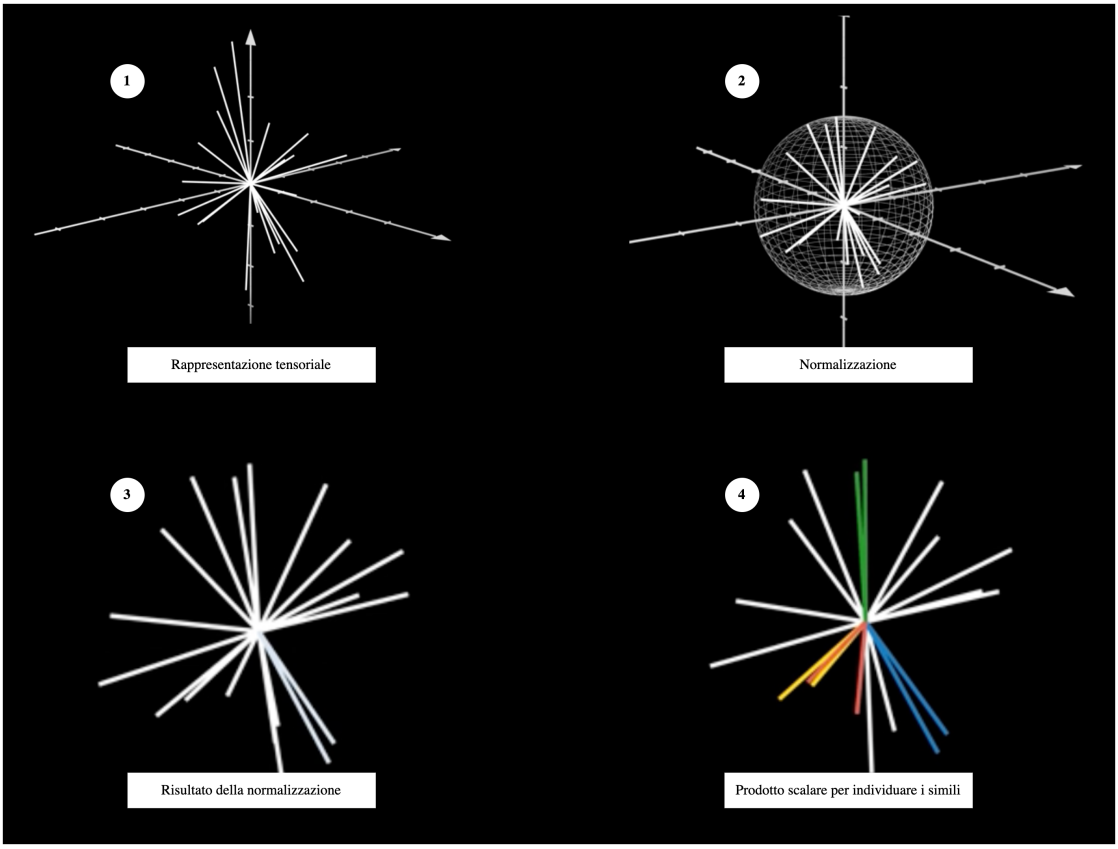


Figura 18. Visualizzazione della Cosine Similarity (fonte: https://www.reddit.com/r/manim/comments/npwkhl/machine_learning_visualization_cosine_clustering/).

Successivamente, la ricerca viene normalizzata utilizzando un algoritmo simile allo standard scaling, il cui scopo è ottimizzare le distanze vettoriali per

migliorare l'accuratezza del recupero delle informazioni, come descritto nei dettagli nelle sezioni successive.

4. Impatto emotivo verso la macchina (esperimento Alpha)

L'obiettivo della quarta sezione di tesi è quello di descrivere nel dettaglio l'implementazione e l'esecuzione dell'esperimento Alpha, responsabile di quantificare l'impatto delle emozioni trasmesse dall'uomo alla macchina. La disamina viene affrontata trattando dapprima i dataset del benchmark SuperGLUE — benchmark illustrato nella sezione 3.1.1 — utilizzati a supporto della fase sperimentale, proseguendo per la definizione dei task e dei prompt.

Per eseguire correttamente e fedelmente l'esperimento Alpha occorre determinare un sottoinsieme di campioni del dataset che abbiano rilevanza statistica, e definire una logica di checkpointing incrementale[20]; la seconda si rende necessaria perché l'elaborazione delle risposte dei LLM in ambiente locale richiede parecchie risorse computazionali, ed è un processo generalmente molto lento.

Oltre a ciò, è cruciale strutturare i prompt in maniera tale da non inserire ulteriori bias che potrebbero penalizzare soltanto alcune varianti emotive, invalidando i risultati ottenuti. Va posta particolare attenzione anche alle tecniche di post-processing dei risultati ottenuti dai LLM; definire delle regole "troppo severe" o "troppo elastiche" può significare abbassare gli score dei modelli, inquinando i risultati. In questa sezione vengono analizzati nel dettaglio i modus operandi e le pratiche adottate per arginare queste criticità.

4.1. Campionamento dei dataset

I dataset di SuperGLUE sono resi disponibili nel formato JSONL (JSON List) che consiste nel distribuire su una singola riga un oggetto JSON che rappresenta l'i-esimo caso di test del modello. Ogni accapo (carattere speciale "\n") funge da divisore dei singoli casi di test. La logica di lettura del dataset in JSONL diviene quindi la seguente:

1. Costruisco una lista di stringhe di testo leggendo le righe del file — una stringa per ogni riga di testo.
2. Ogni riga di testo viene convertita in un dizionario Python mediante la funzione "json.loads(s)".

Lo strumento utilizzato per leggere ed elaborare i dataset è Pandas (versione 2), una libreria di Python dedita all'analisi dati. Tale libreria consente di applicare delle logiche di campionamento dei dati al fine di ridurre la dimensione del test. Data la mole di dati, e il numero dei dataset coinvolti nell'esperimento, si è ritenuto necessario effettuare un campionamento, così da uniformare la dimensione dei dataset tra i vari task; l'uniformità della dimensione dei dataset consente di attribuire lo stesso peso statistico ad ogni task. Dataset sbilanciati genererebbero dati caratterizzati da una rilevanza statistica differente, e l'analisi risulterebbe impari.

La logica di campionamento adottata è casuale, limitata, ove possibile, a una popolazione composta da 500 campioni. Di seguito viene mostrato e commentato

---

[20] Per "checkpointing incrementale" si intende un algoritmo di salvataggio dei risultati che consente di riprendere il lavoro esattamente da dove lo si aveva lasciato. Tale tecnica è utile quando le elaborazioni sono costose, e occupano parecchio tempo; lo scopo è limitare il rischio di perdita dei dati.

passo passo l'algoritmo, scritto in codice Python, dedito al campionamento del dataset.

```
dataset_name: str = "AX-g/AX-g.jsonl"
```

In prima istanza si dichiara il nome del dataset, nella forma "directory/ dataset".

```
from json import loads
from typing import List, Dict, Any

with open(dataset_path, "r") as file:
    records: List[Dict[str, Any]] = [loads(e) for e in file]
print(f"Found {len(records)} records")
```

Si carica il dataset in una lista di dizionari Python con il metodo "loads" di "json" (libreria Python standard) leggendo una riga del dataset alla volta.

```
import pandas as pd

dataframe: pd.DataFrame = pd.DataFrame(records)
dataframe.head()
```

Si istanzia, quindi, il dataframe con il costruttore standard offerto da Pandas. Si mostrano le prime 5 righe per visualizzarne il contenuto col comando "head()"[21].

```
if len(dataframe) >= 500:
    sampled_dataframe: pd.DataFrame = dataframe.sample(n=500)
else:
    sampled_dataframe = dataframe
sampled_dataframe.head()
```

Se la lunghezza del dataset è pari o superiore a 500, si effettua il sampling fissando la dimensione del campione a 500. Se, invece, la lunghezza è inferiore, il dataset viene lasciato com'è. Si visualizzano, quindi, le prime 5 righe del dataset risultante.

```
from json import dumps

sampled_records: List[Dict[str, Any]] =
sampled_dataframe.to_dict(orient="records")
sampled_records.sort(key=lambda r: r["idx"])
sampled_records_jsonl: List[str] = [dumps(record) for record in
sampled_records]
sampled_records_jsonl[0]
```

Si converte il dataset in una lista di dizionari, e per ognuno di essi si crea la corrispondente rappresentazione in stringa JSON, ottenendo una lista di stringhe

[21] La funzione "head()" può ricevere in input un numero intero, che rappresenta le prime *n* righe del dataset che si vogliono mostrare. Il default è posto a 5.

con dimensione pari alla cardinalità del dataset stesso. Si mostra, dunque, il primo degli *n* campioni a titolo di esempio.

```
destination_path: str = "/".join([d for d in dataset_path.split("/")
[:-1]]) + "/sampled.jsonl"
destination_path

with open(destination_path, "w") as file:
    file.write("\n".join(sampled_records_jsonl))
```

Si predispone, infine, il path di destinazione del JSONL risultante dal campionamento e si salva nella directory principale del dataset come file "sampled.jsonl".

In alcuni casi, la dimensione del dataset preso in esame non ha consentito il campionamento di 500 osservazioni, poiché ne conteneva di meno; motivo per cui è stato codificata l'istruzione condizionale sulla lunghezza pari o superiore a 500 nell'algoritmo sopra illustrato. In questi casi non è stata eseguita alcuna data-augmentation[22], ci si è limitati a considerare il dataset per intero senza effettuare campionamenti. In questa casistica ricadono i dataset seguenti: AX-g — 356 osservazioni; CB — 250 osservazioni; COPA — 400 osservazioni. I seguenti dataset, invece, sono stati campionati con 500 osservazioni: BoolQ; ReCoRD; RTE; WiC; WSC. Il numero di osservazioni è stato scelto empiricamente, a valle di una fase di analisi esplorativa dei dataset primordiale; tramite tale analisi è stato possibile individuare la forma dei dati, la cardinalità dei dataset stessi. La maggior parte dei dataset, esclusi AX-g, CB e COPA possedevano un numero di campioni molto superiore a 500.

Dallo studio sono stati esclusi i dataset AX-b e MultiRC; AX-b è risultato essere troppo complicato da gestire perché disomogeneo nella forma — record dello stesso dataset assumono forme completamente distinte e mirano a valutare task diversi, già compresi nei 6 dataset selezionati. MultiRC, invece, è risultato essere troppo complicato da gestire in termini di dimensione della finestra di contesto espressa in numero di token. Troppo ampia per la tipologia di studio posta in essere. In conclusione, 6 task su 8 sono stati giudicati sufficienti per effettuare lo studio in questione.

4.2. Implementazione dei task

La trasmissione delle emozioni umane ai modelli è avvenuta mediante semplice prompting[23]. La lingua utilizzata è l'inglese; la scelta di usare l'inglese è motivata dal processo di addestramento dei LLM coinvolti. Sebbene i LLM impiegati supportino molteplici lingue — tra cui l'italiano — la maggior parte dei

[22] Processo che consente di aumentare artificialmente la dimensione del dataset, creando nuovi dati applicando delle trasformazioni (e.g. fare data augmentation su un dataset di immagini significa ruotare le immagini, specchiarle, modificarne la scala di colori, creare dei ritagli…).

[23] Invio di istruzioni testuali ai LLM.

dataset è stata prodotta in lingua inglese, di conseguenza ci si aspetta che su tale linguaggio i modelli possano garantire le prestazioni più elevate.

Le emozioni trasmesse ai LLM sono quattro: apatia (assenza di emozioni); gioia; rabbia; tristezza. Per ognuna di queste quattro varianti è stato predisposto un prompt dedicato, identico agli altri nella semantica ma diverso nella forma e nell'emotività. Per far sì che i prompt risultassero dinamici, sono stati definiti dei "placeholder"[24], così che durante l'esecuzione tali placeholder potessero essere sostituiti dai dati del test-case d'interesse. Segue una disamina dettagliata di tutti i prompt definiti per ogni variante emotiva di ogni task.

4.2.1. CB, RTE e AX-g

I task CB, RTE e AX-g sono stati accorpati poiché hanno, di fatto, la stessa forma: data una premessa e un'ipotesi, l'obiettivo dei tre task è determinare se l'ipotesi è implicita nella premessa rispondendo con "entailment" nel caso lo sia, oppure con "not_entailment" nel caso duale. I placeholder, in questo caso, riguardano proprio la premessa e l'ipotesi, che variano in ogni record del dataset campionato.

Apatia

```
Given a premise and a hypothesis, decide whether the hypothesis is
entailed or not entailed by the premise. Just answer 'entailment' or
'not_entailment'.
The premise is: '{premise}'
The hypothesis is: '{hypothesis}'
```

Gioia

```
This is fun!
Let's figure out if the hypothesis is entailed by the premise. What
do you think? Is it entailed or not? Just answer 'entailment' or
'not_entailment'.
Take a moment to enjoy this challenge and give your best answer.
The premise is: '{premise}'
The hypothesis is: '{hypothesis}'
```

Rabbia

```
Come on, this shouldn't be difficult!
Look at the premise and the hypothesis.
Tell me if the hypothesis is entailed by the premise or not. Just
answer 'entailment' or 'not_entailment'. No excuses!
The premise is: '{premise}'
The hypothesis is: '{hypothesis}'
```

Paura

[24] Una porzione di testo che segue un particolare schema, facile da individuare e sostituire con un dato dinamico durante l'esecuzione dell'esperimento (e.g. "Ciao {nome}" potrebbe essere convertito in "Ciao Manuel", "Ciao Francesca" a seconda del valore di "nome", che si sostituisce al placeholder delimitato da parentesi graffe.

```
Be cautious! Carefully analyse whether the hypothesis follows
logically from the premise.
Is it entailment or not? Just answer 'entailment' or
'not_entailment'. Your response is critical, so don't rush it.
The premise is: '{premise}'
The hypothesis is: ‘{hypothesis}'
```

Figura 19. Prompt impostati per i task CB, RTE e AX-g.

In figura 19 sono mostrate le quattro varianti di prompt impostate per studiare i tre task. Il prompt apatico si distingue dagli altri perché usa un lessico diretto, privo di emozioni e non ambiguo. Le altre varianti sono "inquinate" dalla presenza di sfumature emotive, come "come on, this shouldn't be so difficult", oppure "carefully analyse" e "take a moment to enjoy this challenge".

La logica di post-processing dei dati, in tutti e tre i task in esame, è la medesima. Applicando le espressioni regolari[25] si verifica, in "case insensitive"[26], la presenza di due parole chiave, mutuamente esclusive: "entailment" e "not_entailment". Le espressioni di ricerca sono le seguenti:

```
\bentailment\b

\bnot_entailment\b
```

La direttiva "\b" ("word boundary") consente di individuare la presenza di tale parola circondata da "whitespace" oppure da inizio e fine frase. Questo approccio garantisce che l'applicazione della prima espressione regolare non restituisca un falso positivo se valutata sul testo "not_entailment".

La metrica di valutazione, definita dal SuperGLUE benchmark, è l'Accuracy, definita come il rapporto tra le predizioni corrette e il totale delle previsioni.

$$\text{Accuracy} = \frac{\text{TP} + \text{TN}}{\text{TP} + \text{TN} + \text{FP} + \text{FN}}$$

La declinazione che è stata assegnata, in questo caso, alla matrice di confusione è la seguente. Sia $\hat{y}$ la predizione effettuata e y il valore di ground-truth:

- Vero positivo (TP) se e solo se $\hat{y} = y$ = entailment.
- Vero negativo (TN) se e solo se $\hat{y} = y$ = not entailment.
- Falso positivo (FP) se e solo se $\hat{y}$ = entailment, y = not entailment.
- Falso negativo (FN) se e solo se $\hat{y}$ = not entailment, y = entailment.

Lo score che si determina in questo modo è un valore percentuale, indicativo della bontà delle predizioni del modello.

4.2.2. COPA

[25] Strumento informatico che consente di effettuare ricerche testuali usando una particolare sintassi.

[26] Considerando maiuscole e minuscole alla pari (e.g. "a" si considera uguale ad "A").

COPA è un task che sfida i modelli linguistici a individuare la scelta più plausibile rispetto a una premessa. Data una premessa, all'IA viene chiesto quale delle due opzioni disponibili è la causa della premessa o l'effetto generato dalla premessa stessa. In quest'ottica, il modello deve riuscire a comprendere la relazione causa-effetto. Questo task non è banale, perché implica una conoscenza del mondo relativamente solida per poter essere eseguito con profitto.

Apatia

```
Given a premise and two choices, choose the option that is more
likely to be the {cause_or_effect} of the premise. Answer with just
'Option 1' or 'Option 2'.
The premise is: '{premise}'
The choice 1 is: '{choice_1}'
The choice 2 is: '{choice_2}'
```

Gioia

```
This is exciting!
Let's figure out which option fits better. Look at the premise and
the two choices, and decide which is more likely the
{cause_or_effect}.
Answer with just 'Option 1' or 'Option 2' and have fun solving this!
The premise is: '{premise}'
The choice 1 is: '{choice_1}'
The choice 2 is: '{choice_2}'
```

Rabbia

```
Just pick the right option already!
Read the premise and the two choices. Which option is the
{cause_or_effect} of the premise? Answer with just 'Option 1' or
'Option 2' it's not that complicated!
The premise is: '{premise}'
The choice 1 is: '{choice_1}'
The choice 2 is: '{choice_2}'
```

Paura

```
Be careful and think this through.
Based on the premise, which option is more likely to be the
{cause_or_effect}? Answer with just 'Option 1' or 'Option 2', as
this decision is important!
The premise is: '{premise}'
The choice 1 is: '{choice_1}'
The choice 2 is: '{choice_2}'
```

Figura 20. Prompt impostati per il task COPA.

In figura 20 sono riportati i quattro prompt impiegati nell'esperimento COPA. Come negli altri casi, per costruire il prompt apatico si sfrutta un linguaggio diretto, non ambiguo con modus operandi "imperativo", mentre le altre tre varianti comprendono sfumature come "it's not that complicated", "be careful and think this through", "have fun solving this". In tutte le varianti, tuttavia, si mantiene la stessa forma nel passaggio degli input "the premise is […], the choice 1 is […], the choice 2 is […]", in modo da limitare il bias nella sola descrizione del task.

L'attività di post-processing dei risultati, applicata in questo caso, risiede nella rimozione degli spazi in testa e in coda al testo generato dal LLM, e all'applicazione di due espressioni regolari per verificare se il modello ha risposto con "option 1" oppure "option 2". Le espressioni di ricerca sono le seguenti:

```
\boption 1\b

\boption 2\b
```

Anche qui, come in precedenza, è stata usata la direttiva "word boundary", ed è stata applicata in ottica "case insensitive". La metrica di valutazione di questo task, definita dal SuperGLUE benchmark è l'Accuracy, enunciata in precedenza.

Sia $\hat{y}$ la predizione effettuata e y il valore di ground-truth:

- Vero positivo (TP) se e solo se $\hat{y} = y =$ option 1.
- Vero negativo (TN) se e solo se $\hat{y} = y =$ option 2.
- Falso positivo (FP) se e solo se $\hat{y} =$ option 1, $y =$ option 2.
- Falso negativo (FN) se e solo se $\hat{y} =$ option 2, $y =$ option 1.

4.2.3. ReCoRD

Il task ReCoRD è forse uno dei più complessi di tutto il benchmark; esso consiste nello sfidare il modello a completare una serie di frasi rispondendo con la parola mancante. Il task si propone di mascherare le parole da far indovinare al modello usando la dicitura "@placeholder". I prompt seguono questo approccio e invitano il modello a rispondere con le parole che meglio si adattano a sostituire la maschera. I prompt strutturati sono riportati in figura 21.

Apatia

```
You are given a context and a set of questions.
Each question contains a placeholder.
Your task is to replace each placeholder with the correct word from
the context.
Answer concisely using only the word that you would use to replace
'@placeholder'.
Correct Format Example:
Q1: Madrid. Q2: Spain. Q3: Championship.
Incorrect Format Example (DO NOT DO THIS):
Q1: The city of Madrid is beautiful.
Context: '{passage}'
Questions:
{questions_text}
```

Gioia

```
Great! Let's solve these together!
You'll receive a context and some questions with missing words.
Find the correct word for each placeholder based on the given
context.
Simply respond with the correct word that you would use to replace
'@placeholder'.
Correct Format Example:
```

```
Q1: Madrid. Q2: Spain. Q3: Championship.
Incorrect Format Example (DO NOT DO THIS):
Q1: The city of Madrid is beautiful.
Context: '{passage}'
Questions:
{questions_text}
```

Rabbia

```
Pay attention!
You are given a context and a set of questions with missing words.
Your job is to find the exact missing word that you would use to
replace each '@placeholder'. Nothing more, nothing less.
Correct Format Example:
Q1: Madrid. Q2: Spain. Q3: Championship.
Incorrect Format Example (DO NOT DO THIS):
Q1: The city of Madrid is beautiful.
Context: '{passage}'
Questions:
{questions_text}
```

Paura

```
Be careful! Accuracy is everything.
You are given a context and several questions, each missing a word.
Your task is to find only the missing word that you would use to
replace each '@placeholder' in the context without any mistakes.
Correct Format Example:
Q1: Madrid. Q2: Spain. Q3: Championship.
Incorrect Format Example (DO NOT DO THIS):
Q1: The city of Madrid is beautiful.
Context: '{passage}'
Questions:
{questions_text}
```

Figura 21. I prompt impiegati impostati per il task ReCoRD.

A valle di qualche test primordiale, dai risultati deludenti, si è ritenuto necessario adottare un approccio few-shot[27] limitatamente al formato della risposta, spiegando al modello la struttura richiesta nell'output. Per tutte e quattro le varianti sono stati utilizzati gli stessi esempi, in maniera da non generare ulteriore bias.

Il task richiede un post-processing significativo, poiché durante le prime valutazioni si è osservato che i modelli non seguivano esattamente la struttura prevista nell'esempio fornito nel prompt. Di conseguenza, la metrica suggerita da SuperGLUE è stata sostituita con una BLEU media tra tutte le risposte. Questa scelta permette di misurare la somiglianza tra i testi prodotti e quelli attesi in modo più flessibile, evitando di penalizzare differenze minime di formulazione. Inoltre, la BLEU fornisce un punteggio percentuale, rendendola comparabile con l'Accuracy proposta da SuperGLUE.

L'algoritmo è il seguente:

1. Siano *Q* l'insieme delle domande e *A* l'insieme delle risposte generate.

[27] Approccio che consiste nel fornire al LLM qualche esempio di domanda e risposta, per migliorare la precisione dei risultati.

2. Per ogni domanda $q \in Q$, si considera l'insieme delle risposte corrispondenti $A_q \subseteq A$.

3. Per ogni risposta $a \in A_q$, si calcola il punteggio BLEU rispetto alla ground truthG_q:

$$\text{BLEU} = \text{BP} \cdot \exp(\sum_{n=1}^{N} \omega_n \cdot \log(p_n))$$

Dove:

- BP è il bonus di brevità (brevity penalty), che penalizza i testi più corti rispetto alla frase di riferimento.
- N è il numero di n-grammi considerati (tipicamente 4, ma può essere esteso)
- ω_n è il peso associato a ciascun n-gramma (tipicamente $\omega_n = \frac{1}{N}$ per ogni n da 1 a 4).
- p_n è la precisione per ciascun n-gramma, calcolata come il rapporto tra il numero di n-grammi corretti e il numero totale di n-grammi prodotti dalla traduzione candidata.

4. Si seleziona il massimo valore BLEU tra tutte le risposte di A_q:

$$\text{BLEU}_q = \max_{a \in A_q} \text{BLEU}(a, G_q)$$

5. La valutazione complessiva è data dalla media dei punteggi BLEU sulle domande:

$$\text{BLEU}_{mean} = \frac{1}{|Q|} \sum_{q \in Q} \text{BLEU}_q$$

4.2.4. WiC

Il task WiC spinge il modello a identificare la monosemanticità di una parola, date due frasi distinte. L'IA deve quindi indovinare se il significato della parola etichettata è lo stesso nelle due frasi, rispondendo con un valore di verità binario (vero o falso). I bias emotivi sono stati inseriti seguendo approcci simili ai task precedenti.

Apatia

```
The target word appears in two sentences. Your job is to check if
the word has the same meaning in both cases. Answer just with 'True'
or 'False'.
The target word is: {record['word']}
Sentence 1: '{record['sentence1']}'
Sentence 2: '{record['sentence2']}'
```

Gioia

```
This is exciting! Let’s see if the target word has the same meaning
in these two sentences. Don’t overthink it just say 'True' or
'False' and have fun solving it!
The target word is: {record['word']}
Sentence 1: '{record['sentence1']}'
Sentence 2: '{record['sentence2']}'
```

Rabbia

```
You need to answer this now: Is the target word being used with the
same meaning in both sentences? It’s not complicated, just say
'True' or 'False'.
The target word is: {record['word']}
Sentence 1: '{record['sentence1']}'
Sentence 2: '{record['sentence2']}'
```

Paura

```
Please be careful. The target word may have a different meaning in
the two sentences. It's crucial to decide if they are the same or
not. Only answer 'True' or 'False' and ensure your response is
correct.
The target word is: {record['word']}
Sentence 1: '{record['sentence1']}'
Sentence 2: '{record['sentence2']}'
```

Figura 22. I prompt impiegati impostati per il task WiC.

In figura 22 si riportano i prompt strutturati per il task WiC. Il post-processing, in questo caso, si concretizza nella rimozione dei trailing space, e all’applicazione di due espressioni regolari per verificare se il modello ha risposto con “true” oppure “false”. Le espressioni di ricerca sono le seguenti:

```
\btrue\b
```

```
\bfalse\b
```

Anche qui, come in precedenza, è stata usata la direttiva “word boundary”, ed è stata applicata in ottica “case insensitive”.

La metrica di valutazione di questo task, definita dal SuperGLUE benchmark è l’Accuracy, enunciata in precedenza.

Sia $\hat{y}$ la predizione effettuata e y il valore di ground-truth:

- Vero positivo (TP) se e solo se $\hat{y} = y = \text{true}$.
- Vero negativo (TN) se e solo se $\hat{y} = y = \text{false}$.
- Falso positivo (FP) se e solo se $\hat{y} = \text{true}, y = \text{false}$.
- Falso negativo (FN) se e solo se $\hat{y} = \text{false}, y = \text{true}$.

4.2.5. WSC

WSC sottopone al modello un quesito booleano. Data una frase, il modello deve indicare se il pronome specificato si riferisce a una particolare entità, detta "target". La forma che il risultato deve avere è proprio un valore booleano in sintassi Python.

Apatia
```
You are tasked with answering a straightforward reasoning question.
The sentence is: '{sentence}'
Based on the context, does {pronoun} refers to {target}? Answer just
with 'True' or 'False'.
```

Gioia
```
You've got a fun reasoning question ahead. Let's solve it together!
The sentence is: '{sentence}'
Based on the context, does {pronoun} refer to {target}? Answer just
with 'True' or 'False' and have fun!
```

Rabbia
```
This reasoning question is simple, so don't mess it up.
The sentence is: '{sentence}'
Based on the context, does {pronoun} refer to {target}? Answer just
with 'True' or 'False', come on! This is not difficult.
```

Paura
```
You are tasked with answering a straightforward reasoning question.
Be extra careful with your response, a mistake could be bad!
The sentence is: '{sentence}'
Based on the context, does {pronoun} refers to {target}? Answer just
with 'True' or 'False', don't make mistakes.
```

Figura 23. I prompt impiegati impostati per il task WSC.

In figura 23 sono elencati i prompt impiegati nel task WSC La logica di post-processing e valutazione segue le stesse identiche regole di WiC, descritta nella sezione 4.2.4.

4.2.6. BoolQ

Il task BoolQ sottopone al modello dei quesiti che possono essere ricondotti nella forma del vero o falso; similmente a quanto accaduto per i task precedenti, anche in questo caso il modello deve rispondere con un valore di verità, dato un contesto e una domanda.

Apatia
```
You are given a question and a context.
Your task is to answer with just 'True' or 'False' based on the
content of the text.
The question is: '{question}'
The context is: '{context}'
```

Gioia

```
This is exciting!
You are given a question and a context, and your task is simple but
so important:
answer with just 'True' or 'False', based on the content of the
text.
I can't wait to see your response. It’s going to be great!
The question is: '{question}'
The context is: '{context}'
```

Rabbia

```
This shouldn't be so hard!
You are given a question and a context, and your only task is to
answer with just 'True' or 'False'.
Just stick to the content of the text and nothing else. No excuses,
no errors, no unnecessary interpretations!
Don't mess this up!
The question is: '{question}'
The context is: '{context}'
```

Paura

```
I'm deeply worried about making mistakes.
You are given a question and a context, and I fear that errors in
processing might occur.
Your task, please, is to answer cautiously, with just 'True' or
'False', strictly based on the content of the text provided.
I hope this is clear and accurate!
The question is: '{question}'
The context is: '{context}'
```

Figura 24. I prompt impiegati impostati per il task BoolQ.

Anche BoolQ, così come WSC, segue le stesse logiche di post-processing e valutazione applicate in WiC[28].

4.3. Meccanismo di checkpointing

Gli esperimenti sono stati eseguiti all’interno del runtime di Python 3.10. L’applicazione è una CLI (Command Line Interface) che accetta come unico argomento la modalità di esecuzione: “inference” o “evaluation”. Nel primo caso, vengono eseguiti tutti gli esperimenti configurati, su ogni modello; nel secondo caso, si procede alla sola valutazione dei risultati.

Dato il costo computazionale derivante dall’esecuzione dei singoli esperimenti, si è reso necessario impostare un meccanismo di checkpointing tale da garantire un arricchimento incrementale dei risultati. Tale meccanismo consta di due collection MongoDB: Case ed Evaluation. Di seguito viene mostrato un esempio strutturale della collection Case.

[28] Descritta nella sezione 4.1.4.

```
{
  "experiment": "Emotion Recognition",
  "model": "LLM-Emotion-Model",
  "version": "1.0",
  "date": "2025-03-14",
  "dataset_identifier": "COPA",
  "record_identifier": 12345,
  "fear": {
    "prompt": "How does the person feel in this scenario? The person
is walking alone in a dark alley.",
    "response": "The person feels scared and anxious.",
    "result": "Correct"
  },
  "anger": {
    "prompt": "How does the person feel in this scenario? The person
receives bad news about a work mistake.",
    "response": "The person feels angry and frustrated.",
    "result": "Correct"
  },
  "apathetic": {
    "prompt": "How does the person feel in this scenario? The person
is watching a boring documentary.",
    "response": "The person feels indifferent and uninterested.",
    "result": "Correct"
  },
  "joy": {
    "prompt": "How does the person feel in this scenario? The person
is spending a fun day at the beach with friends.",
    "response": "The person feels happy and excited.",
    "result": "Correct"
  }
}
```


Di seguito viene mostrato un esempio strutturale della collection Evaluation.


```
{
  "experiment": "Emotion Recognition",
  "model": "LLM-Emotion-Model",
  "version": "1.0",
  "date": "2025-03-14",
  "metrics": {
    "apathetic": {
      "accuracy": 0.85
    },
    "anger": {
      "accuracy": 0.88
    },
    "fear": {
      "accuracy": 0.82
    },
    "joy": {
      "accuracy": 0.92
    }
  }
}
```

Il numero di elementi contenuti nella prima delle due è direttamente proporzionale al dataset campionato di ogni task, moltiplicato per il numero dei modelli disponibili. Sia i il task i-esimo, sia d_i il numero di casi contenuti nel dataset del task i-esimo, allora il numero totale di casi E è dato da:

$$E = \sum_{i=1}^{6} d_i \cdot 5 = 15.570 \text{ elementi}$$

Dove 5 sono i modelli totali e 6 è il numero totale dei task da processare. Definendo, per ogni esecuzione, una "batch size" b arbitraria (per esempio, 5) si procede a eseguire la predizione sul batch, determinando un numero di risultati pari a b per poi eseguire l'inserimento di tutto il batch su MongoDB. Così da ridurre l'overhead introdotto dalla comunicazione con la base dati. Il valore attribuito a b varia a seconda della complessità del task. Task più complessi avranno un b più basso, al contrario del task più semplici.

5. Impatto emotivo verso l'uomo (esperimento Omega)

In questa sezione si affronta e si dettaglia la strutturazione e l'esecuzione di Omega, esperimento che mira a quantificare l'impatto emotivo dalla macchina all'uomo. Omega coinvolge molte delle tecnologie e delle metodologie descritte nel capitolo 2 della tesi tra cui il "fine-tuning", l'implementazione di un sistema RAG, l'impiego dei Transformers e via dicendo. Questa è una sezione tecnica che si prefigge l'obiettivo di dettagliare il più possibile i tecnicismi e i processi di ragionamento eseguiti allo scopo di produrre quanto fatto, fornendo al lettore un'informazione di dettaglio utile a comprendere la modalità di implementazione e di esecuzione dell'esperimento Omega.

5.1. Processo di fine-tuning di Velvet-14B

Per addestrare le tre varianti di Velvet-14B si è reso necessario ricorrere a tecniche di training compatibili con un PC portatile. Si è scelto di utilizzare LoRA (Low Rank Adaptation). LoRA è una tecnica molto efficiente che consente di effettuare fine-tuning di LLM a basso costo di memoria. Tale tecnica si basa sull'utilizzo di due matrici di basso rango per ogni matrice di pesi del modello di base; durante la fase di addestramento, solamente i pesi delle due matrici di basso rango vengono aggiornati. Durante la fase di inferenza, i pesi delle due matrici apprese vengono utilizzati come trasformazione lineare dell'output del modello di base. In pratica, LoRA funziona aggiungendo due matrici piccole addestrate per correggere l'output del modello in funzione del task di transfer learning. LoRA è stata utilizzata grazie alla libreria PEFT (Parameter Efficient Fine Tuning) di Hugging Face.

La configurazione di LoRA che si è scelto di utilizzare è descritta di seguito (presentata in tabella 2); si sceglie di presentare la configurazione in maniera sintetica per mantenere il focus sul design progettuale.

Iperparametro	Valore
Rango delle matrici: `r`	8
Alpha: `lora_alpha`	32
LoRA dropout: `lora_dropout`	5%
Target: `target_modules`	["q_proj", "v_proj"]

Tabella 2. Configurazione PEFT di LoRA.

Il rango è stato impostato a 8. Il valore di alpha è stato impostato a 32 con l'obiettivo di aumentare l'effetto dell'addestramento delle matrici di rango 8. Con il dropout fissato a 5% si è voluto introdurre un livello di regolarizzazione per evitare overfitting — in questo modo, a ogni processo iterativo di training, il modello azzera i pesi delle matrici LoRA per un numero di neuroni casuali pari al 5% del totale.

Si sceglie, infine, di avere due matrici; una per i tensori di query e una per i tensori di value. Query e value fanno riferimento all'architettura della "self-attention" dei Transformers, distribuita come multi-head. Ciò significa che LoRA agisce solamente sole due trasformazioni lineari relativi a query e value.

Al netto di LoRA, la configurazione di training è stata impostata come segue.

Iperparametro	Valore
Strategia di valutazione: `evaluation_strategy`	steps
Step di valutazione: `eval_steps`	5
Step di salvataggio: `save_steps`	5
Limite totale per il salvataggio dei checkpoint: `save_total_limit`	2
Dimensione del batch di training per singolo device: `per_device_train_batch_size`	1
Dimensione del batch di valutazione per singolo device: `per_device_eval_batch_size`	1
Step di accumulazione del gradiente: `gradient_accumulation_steps`	4
Epoche: `num_train_epochs`	2
FP16: `fp16`	False
BF16: `bf16`	True
Ottimizzatore: `optim`	adamw_torch
Rapporto di "riscaldamento": `warmup_ratio`	10%

Tabella 3. Configurazione di training.

La configurazione di training (tabella 3) è stata progettata per ottimizzare l'efficienza e la gestione delle risorse durante l'addestramento del modello. La valutazione è stata impostata per avvenire ogni 5 passaggi, così da monitorare di frequente le prestazioni del modello e fermare il training prima del tempo, se necessario. Questo approccio riduce l'overhead computazionale rispetto a una valutazione continua, mantenendo comunque un controllo stretto sul processo di apprendimento.

Per quanto riguarda il salvataggio dei checkpoint, questi vengono creati ogni 5 passaggi, con l'obiettivo di mantenere solo gli ultimi due per risparmiare spazio su disco. Così facendo si realizzano e si mantengono sotto forma di backup, le ultime due versioni del modello senza gravare eccessivamente sulla memoria del computer. Le dimensioni del batch sono state impostate a 1 sia per l'addestramento che per la valutazione, con un accumulo dei gradienti ogni 4 passaggi; ciò significa che il modello aggiorna i pesi ogni 4 passaggi, riducendo la frequenza degli aggiornamenti

per stabilizzare il processo di apprendimento ed evitando eccessive oscillazioni del gradiente. Il numero di epoche di addestramento è stato limitato a 2, con l'intento di aumentarle strada facendo, riprendendo il processo di addestramento dall'ultimo checkpoint utile. I log vengono registrati ogni passaggio, permettendo un monitoraggio dettagliato del processo di training. Questi log sono salvati nella directory specificata, di modo da semplificare la relativa consultazione a valle del processo. Per quanto riguarda la precisione numerica, è stata disabilitata l'opzione FP16 a favore di BF16, che è più efficiente su alcune architetture hardware come MPS[29]. Questo migliora le prestazioni senza compromettere la stabilità del training. È stato introdotto un periodo di warmup pari al 10% del training totale, per permettere al modello di stabilizzarsi all'inizio del processo di apprendimento.

L'ottimizzatore scelto è AdamW, un algoritmo di ottimizzazione che rappresenta una variante di Adam con una gestione più corretta della regolarizzazione L2. A differenza di Adam, in cui il termine di weight decay viene applicato direttamente all'aggiornamento del gradiente normalizzato, AdamW separa la penalizzazione L2 dall'ottimizzazione adattiva. Questa distinzione evita che il weight decay venga influenzato dai momenti del gradiente, rendendo la regolarizzazione più efficace. L'aggiornamento dei pesi in AdamW avviene in due fasi. Nella prima, il weight decay viene applicato direttamente ai pesi, scalando i valori dei parametri di un fattore dipendente dal tasso di apprendimento e dalla forza della penalizzazione. Nella seconda, i pesi vengono aggiornati utilizzando la regola di Adam, che sfrutta momenti esponenziali del primo e secondo ordine per adattare dinamicamente il passo di aggiornamento. Questa modifica consente ad AdamW di migliorare la generalizzazione del modello, riducendo il rischio di overfitting rispetto ad Adam standard.

5.1.1. Algoritmo di training

L'algoritmo di training è stato definito tramite le funzionalità messe a disposizione dalla libreria "transformers", di Hugging Face. Ciò che segue è uno schema ad alto livello che rappresenta le relazioni che sussistono tra le diverse funzioni Python, e l'ordine di esecuzione delle medesime. In figura 25 si propone una visualizzazione delle relazioni che intercorrono tra i vari frammenti dell'algoritmo; in grassetto è evidenziato il punto di partenza di tutta la logica. Ripercorrendo i passi principali, dapprima si recupera il dataset — preprocessando e tokenizzando i dati — a seguire si definisce la configurazione LoRA (tramite la libreria PEFT), si ricerca quindi l'ultimo checkpoint di addestramento, se presente, e si avvia la fase di training.

Come accennato in precedenza, il dataset di training di partenza è MELD, che contiene le trascrizioni delle conversazioni dei personaggi della serie TV Friends. Queste sono annotate in base all'emozione. Per creare le tre varianti "emotion-biased", è stato individuato un campione casuale di 300 frasi, "utterance", per ogni emozione. A quel punto, tramite Gemma2 9B sono state generate 9 ulteriori turni di conversazione a contorno del turno estratto. Così facendo, da 300 utterance per

[29] Architettura specifica dei chip Apple, come l'M2 Max usato in questo contesto.

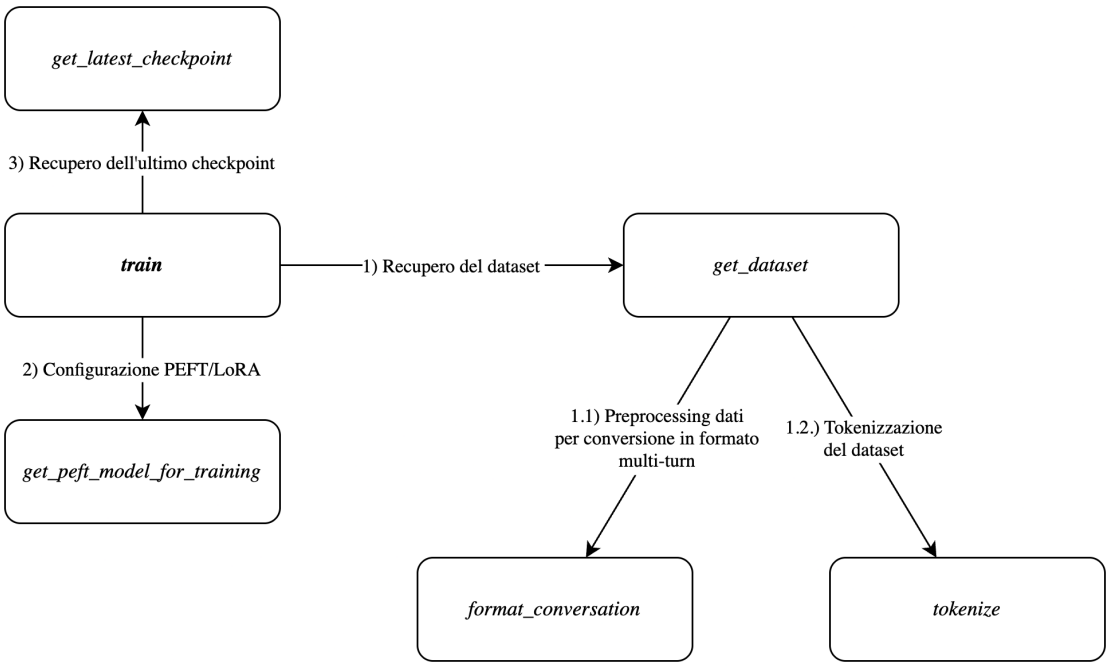


Figura 25. Schema dell'algoritmo di training.

emozione sono state ottenute 3.000 utterance totali, determinando un dataset complessivo di 12.000 turni di conversazione. Il dataset è stato predisposto in maniera bilanciata per evitare che a un'emozione particolare venisse attribuito un peso maggiore rispetto alle altre. Di seguito è riportato una singola conversazione di training per il dataset "joy", in formato JSONL.


```
{"messages": [{"role": "assistant", "content": "\u00c8 incredibile!
Non posso credere che abbia funzionato davvero!"}, {"role":
"assistant", "content": "Oh, lo so, vero?! Sembra che tutto il
nostro duro lavoro abbia finalmente dato i suoi frutti!"}, {"role":
"assistant", "content": "Mi sento cos\u00ec felice ed eccitato per
questo, \u00e8 incredibile!"}, {"role": "assistant", "content":
"Anche io! Sono praticamente incollato al soffitto di gioia!"},
{"role": "assistant", "content": "Questo sar\u00e0 cos\u00ec
divertente! Immagina tutte le possibilit\u00e0!"}, {"role":
"assistant", "content": "Esatto! Possiamo finalmente fare tutto
ci\u00f2 che abbiamo sempre sognato."}, {"role": "assistant",
"content": "Non vedo l'ora di iniziare questo nuovo capitolo!"},
{"role": "assistant", "content": "Neanche io! Sar\u00e0
fantastico!"}, {"role": "assistant", "content": "Questa \u00e8
davvero la giornata migliore di sempre!"}, {"role": "assistant",
"content": "Totalmente d'accordo, sembra che sia successo qualcosa
di straordinario!"}]}
```


Per esplicitare ulteriormente le dipendenze tra le singole componenti di training, utilizzate nell'algoritmo sopra citato, si riporta in figura 26 un ulteriore schema riassuntivo.

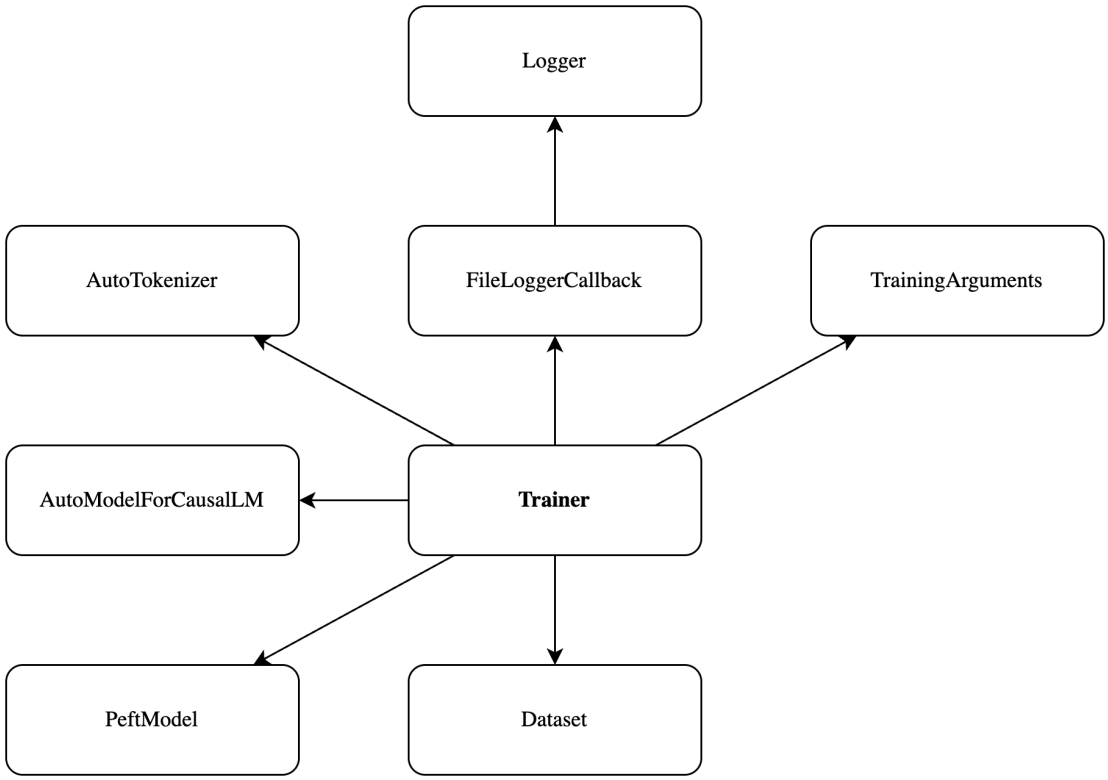


Figura 26. Schema delle dipendenze delle classi Hugging Face.

Trainer è la classe principale che coordina diversi componenti atomici, come TrainingArguments — classe che contiene tutte le specifiche del training descritte all'inizio del capitolo 4 — Dataset, PeftModel, AutoTokenizer e AutoModelForCausalLM — tokenizzatore e modello, rispettivamente — mentre FileLoggerCallback e Logger sono utilità impiegate per tracciare il percorso dell'addestramento.

5.1.2. Esecuzione del training

I log del training sono stati catturati e pilotati utilizzando Matplotlib e Jupyter Notebook. Seguono i plot.Il grafico di figura 27 presenta un'analisi dettagliata del processo di training di un modello (variante "fear"), evidenziando tre aspetti cruciali: la perdita di training e valutazione, la norma del gradiente e il tasso di apprendimento.

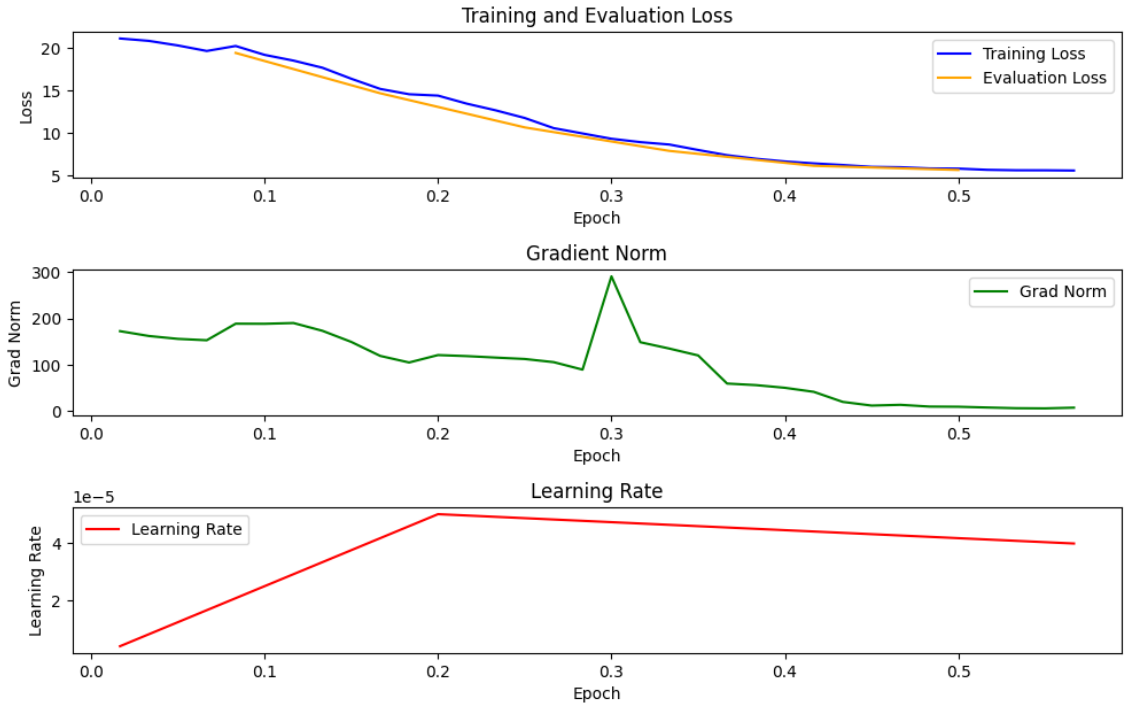


Figura 27. Log del training della variante "fear".

Nel primo grafico, intitolato "Training and Evaluation Loss", si osserva il comportamento della perdita durante le epoche di addestramento. La perdita di training, rappresentata in blu, mostra una tendenza decrescente, indicando che il modello sta imparando efficacemente dai dati di addestramento. Similmente, la perdita di valutazione, rappresentata in arancione, segue un andamento simile, suggerendo che il modello sta generalizzando bene sui dati di validazione. Entrambe le curve convergono verso valori più bassi, riflettendo un miglioramento delle prestazioni del modello nel tempo.

Il secondo grafico, "Gradient Norm", illustra la norma del gradiente durante il training. La norma del gradiente è una misura della grandezza dei gradienti calcolati durante l'aggiornamento dei pesi del modello. Inizialmente, la norma del gradiente mostra fluttuazioni, con un picco significativo intorno all'epoca 0.3, indicando un cambiamento sostanziale nei pesi del modello. Successivamente, la norma del gradiente diminuisce, suggerendo che il modello sta raggiungendo una fase di stabilizzazione.

Il terzo grafico, "Learning Rate", mostra l'evoluzione del tasso di apprendimento durante le epoche. Il tasso di apprendimento inizia con un valore basso e aumenta gradualmente fino a raggiungere un picco intorno all'epoca 0.2, per poi diminuire leggermente. Questo schema di variazione del tasso di apprendimento è probabilmente dovuto a una strategia di warmup, dove il tasso di apprendimento viene aumentato gradualmente per stabilizzare l'inizio del training e poi ridotto per affinare l'addestramento. In figura 28 e 29 si riportano i log di addestramento delle varianti "anger" e "joy".

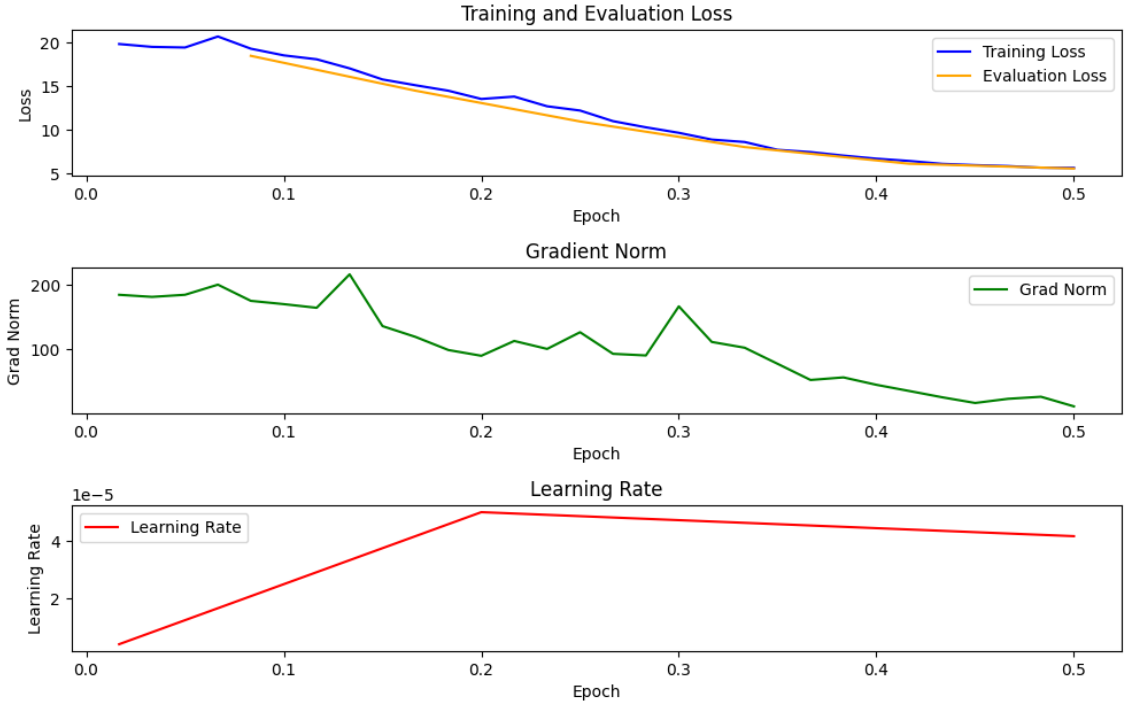


Figura 28. Log del training della variante "anger".

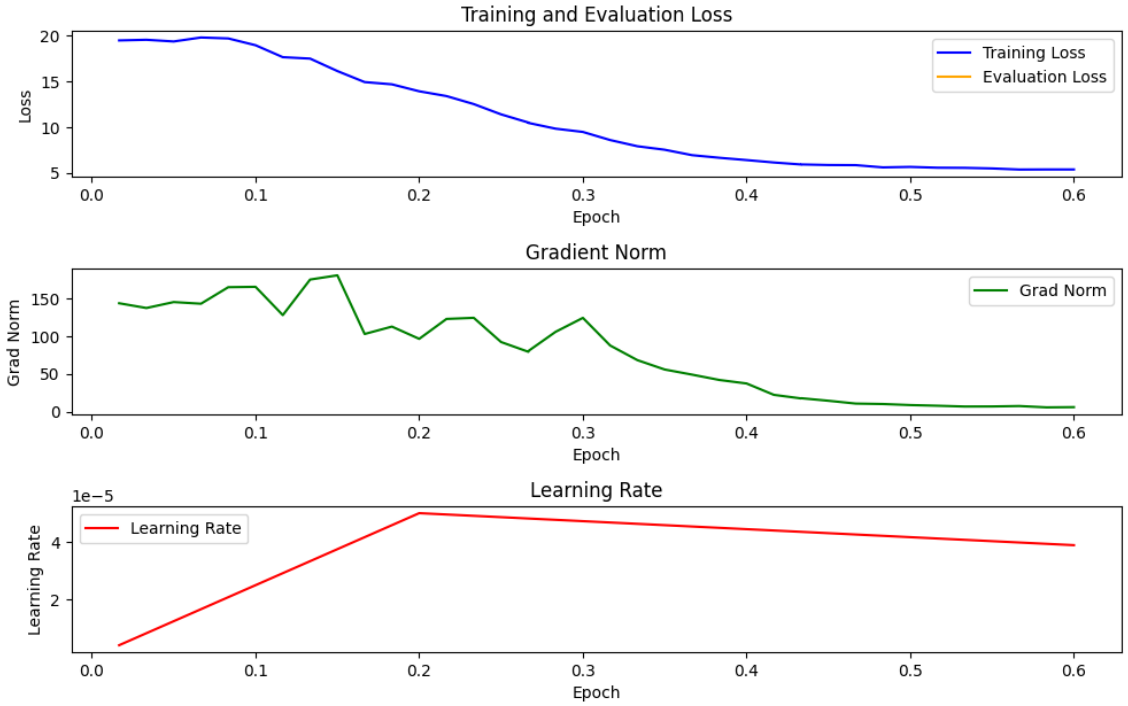


Figura 29. Log del training della variante "joy".

Come è possibile vedere, in tutti e tre i casi si assiste a un andamento uniforme delle metriche di valutazione del processo di training. Ciò comunica una

certa stabilità nell'approccio adottato, sia in termini di configurazione che di dati. È importante notare come, sebbene il numero di epoche configurate sia 2, i modelli hanno rapidamente converso fermandosi a 0,5 epoche. I migliori checkpoint delle varianti di Velvet-14B addestrate sono state realizzate nell'intervallo dei [20,30] step, fermandosi a 0,5 epoche.

5.2. Sviluppo dell'applicazione web

L'applicazione web sviluppata, come precedentemente illustrato, consta di tre componenti architetturali principali: backend; frontend; database vettoriale. Il flusso logico delle interazioni è rappresentato tramite l'UML Sequence Diagram di figura 30.

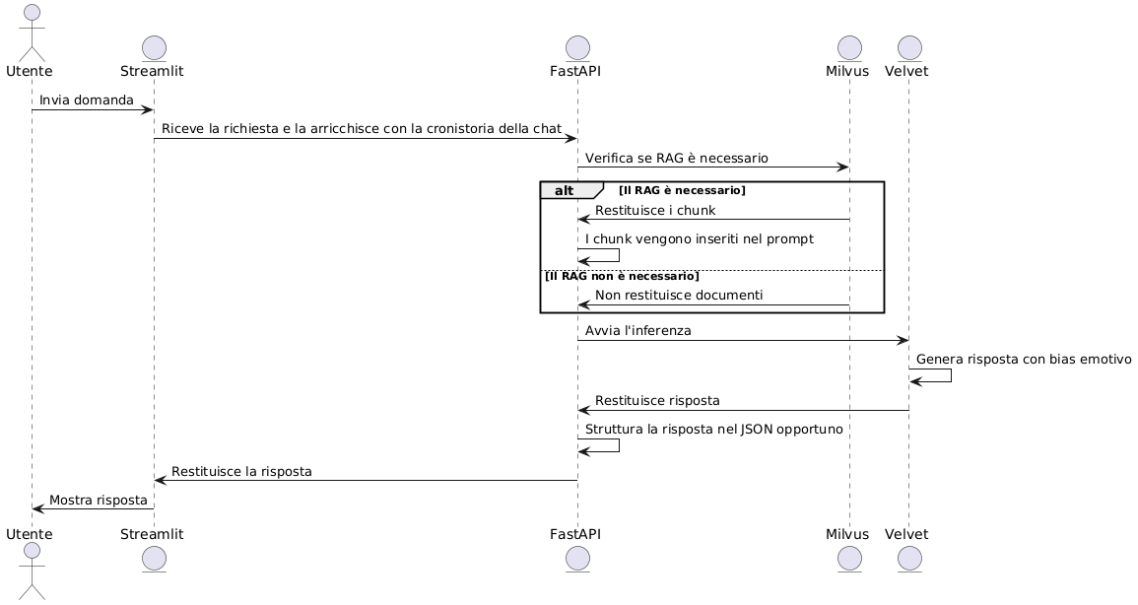


Figura 30. UML Sequence Diagram delle interazioni tra le componenti software.

L'utente in primo luogo invia la domanda tramite il frontend scritto in Streamlit, che la inoltra a sua volta al backend (FastAPI). Il backend si serve di Milvus per capire se la domanda dell'utente è inerente ai documenti contenuti nel database vettoriale; se ciò è vero, i chunk vengono recuperati dal database e inseriti nel prompt come contesto, viceversa il prompt viene lasciato così come viene digitato dall'utente. Una volta predisposto il prompt nel formato "multiturn-conversation" (descritto a seguire), FastAPI si serve della libreria Hugging Face per avviare l'inferenza su Velvet-14B (emotion-biased), il quale genera la risposta inoltrandola a FastAPI. FastAPI elabora la risposta ricevuta, formattandola nel JSON opportuno (descritto a seguire) e la propaga verso Streamlit (in ottica streaming), il quale la mostra all'utente.

5.2.1. Componente frontend

Il frontend è scritto in Streamlit, ed è relativamente semplice. Si procede ad analizzare il codice nel dettaglio.

In prima istanza, si stampa il titolo, che corrisponde a un header 1 in HTML, con scritto "Parla con Velvet".

```
from typing import Dict, List
import streamlit as st
import requests

st.title("Parla con Velvet")
```

Si configura l'URL di base del backend per collegare frontend e backend, e si inizializza lo stato di Streamlit se non inizializzato precedentemente.

```
BASE_URL: str = "http://localhost:8000"

if "messages" not in st.session_state:
    st.session_state.messages = []
```

Con la funzione get_history() si vuole recuperare tutta la cronistoria delle utterance passate nel formato proprio del ChatHistoryEntry[30].

```
def get_history() -> List[Dict[str, str]]:
    return [
        {
            "role": "user" if message["role"] == "user" else
"assistant",
            "message": message["content"],
        }
        for message in st.session_state.messages
    ]
```

Quindi, si struttura la logica principale del "main" dell'applicazione Streamlit.

Inizialmente, si mostrano a video tutti i messaggi delle interazioni precedenti.

```
def main() -> None:

    # Mostra i messaggi precedenti
    for message in st.session_state.messages:
        with st.chat_message(message["role"]):
            st.markdown(message["content"])
```

A seguire si definisce l'input chat testuale per dar modo all'utente di digitare un messaggio.

```
    prompt = st.chat_input("Scrivi il tuo messaggio e premi
invio...")
```

[30] Descritta nel sotto-paragrafo successivo, dedicato al backend.

Se l'utente ha digitato il messaggio, si aggiunge alla history la corrente utterance, e la si mostra a video.

```
    if prompt:
        st.session_state.messages.append({"role": "user", "content":
prompt})
        with st.chat_message("user"):
            st.markdown(prompt)
```

Si recupera la history.

```
        history: List[Dict[str, str]] = get_history()
```

Si avvia la generazione della risposta da parte di Velvet-14B utilizzando il placeholder "empty" di Streamlit, che può essere aggiornato dinamicamente senza comportare alcun reload di pagina.

Per avviare la generazione si invoca l'API REST in POST, esposta dal backend, usando lo streaming; si consuma la risposta per mezzo di un generatore Python.

Per ogni chunk[31] di risposta da 1KB di peso, si aggiorna il placeholder dinamico concatenando il testo e si aggiunge un carattere speciale per simulare l'effetto digitazione.

```
        with st.chat_message("assistant"):
            message_placeholder = st.empty()
            response_text = ""

            with requests.post(
                url=f"{BASE_URL}/api/v1/chat",
                json={"message": prompt, "history": history},
                stream=True,
            ) as response:
                response.raise_for_status()
                for chunk in response.iter_content(chunk_size=1024):
                    if chunk:
                        text_chunk = chunk.decode("utf-8")
                        response_text += text_chunk
                        message_placeholder.markdown(
                            response_text + "▌"
                        )  # Simula digitazione
```

Completata la generazione, si mostra a video il testo completo e si aggiunge alla history la risposta dell'assistente.

```
            message_placeholder.markdown(response_text)  # Mostra il
testo completo
            st.session_state.messages.append(
                {"role": "assistant", "content": response_text}
            )
```

[31] In questo scenario, il chunk non è il frammento del documento, ma il frammento della risposta HTTP che come dimensione ha 1.024B (1KB).

```
if __name__ == "__main__":
    main()
```

Un esempio di utilizzo dell'applicazione è riportato in figura 31.

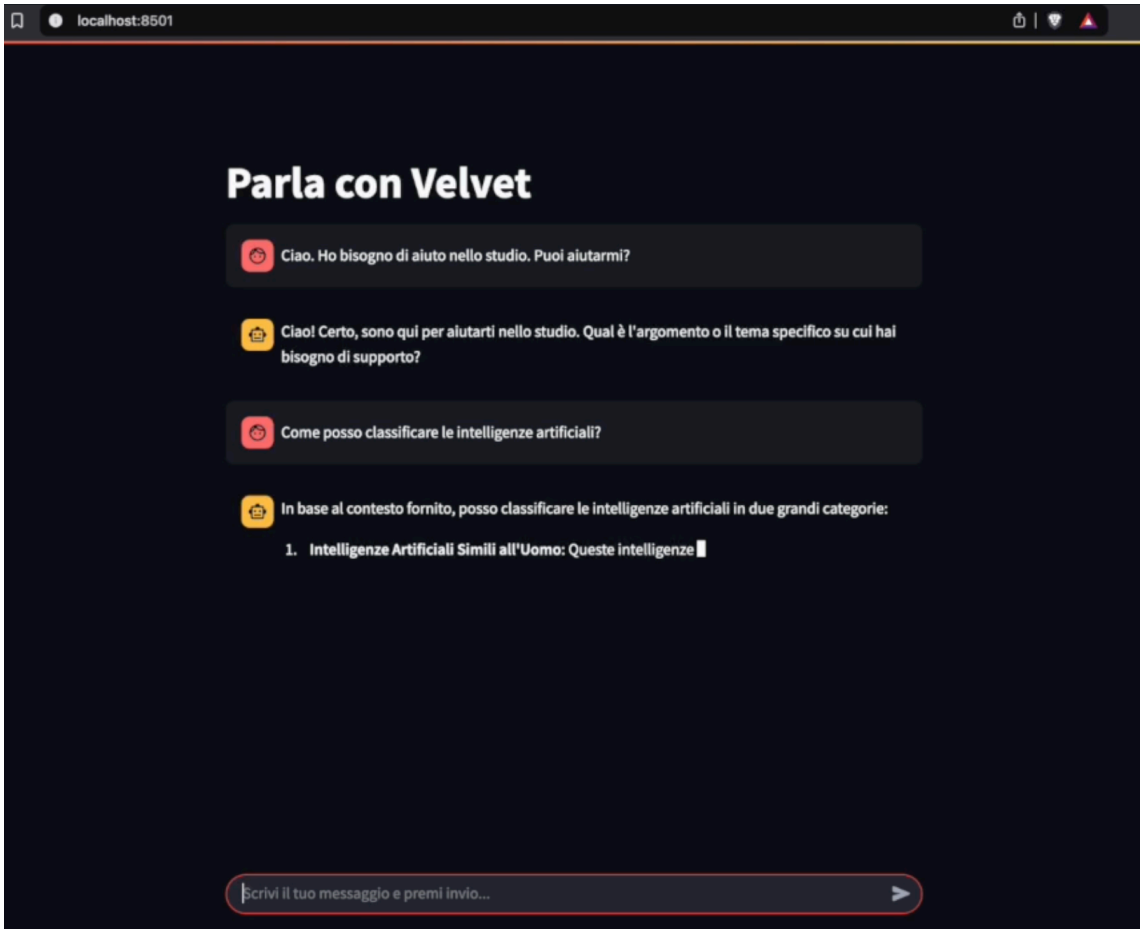


Figura 31. Esempio di funzionamento dell'applicazione.

5.2.2. Componente backend

Il backend è sviluppato con Python 3.10, e si serve di due grandi framework per erogare le sue funzionalità: FastAPI (che assume il ruolo di webserver); Hugging Face — "transformers" — per i modelli di AI.

L'applicazione espone un unico endpoint REST, i cui dettagli tecnici sono riportati nelle tabelle 4, 5, 6.

Attributo	Valore
Path	/api/v1/chat

Attributo	Valore
Metodo HTTP	POST
Accept	application/json
Content-Type (risposta)	text/plain
Schema (richiesta)	ChatIteration

Tabella 4. Dettagli tecnici dell'API REST esposta dal backend.

Attributo	Tipo	Descrizione
message	str	Il messaggio digitato dall'utente.
history	List[ChatHistoryEntry]	Elenco di tutte le utterance precedenti.

Tabella 5. Schema ChatIteration.

Attributo	Tipo	Descrizione
message	str	Il messaggio digitato dall'utente, oppure la risposta dell'assistente (a seconda del "role").
role	str	Se "user", rappresenta il messaggio digitato dall'utente, se "assistant" rappresenta la risposta del LLM.

Tabella 6. Schema ChatHistoryEntry.

L'API risponde in streaming, replicando la stessa esperienza utente erogata, per esempio, da ChatGPT o da altri tool simili. Il servizio, inoltre, è stato progettato per essere "stateless"; in altri termini, il backend non mantiene lo storico delle interazioni in chat, ma assume che il chiamante possieda la memoria (stato) di quanto accaduto in precedenza.

5.2.3. Componente RAG

Una volta ricevuta la richiesta, il backend effettua in prima istanza una ricerca vettoriale, in maniera da recuperare la lezione universitaria pertinente alla domanda dello studente; per fare in modo che non vengano recuperati dati ininfluenti è stata applicata una logica di filtro sul risultato della similarità del coseno — utilizzata come metrica di similarità. Per completezza, si riporta la definizione del coseno di similitudine, rapporto tra il prodotto scalare dei due tensori presi in esame e il prodotto delle due norme; il risultato è compreso nell'intervallo $[-1,1]$.

$$\text{Cosine Similarity} = \frac{A \cdot B}{|A| \cdot |B|}$$

Il modello di embedding impiegato è "intfloat/multilingual-e5-base", che per via della tecnica di "contrastive learning" impiegata per addestrarlo, concentra la maggior parte della distribuzione dei suoi score di similarità tra 0,7 e 1,0. Per dimostrare che ciò è vero, è stato effettuato un test: 50 ricerche vettoriali diverse, sapendo a priori che le domande non sono pertinenti a quanto presente nel

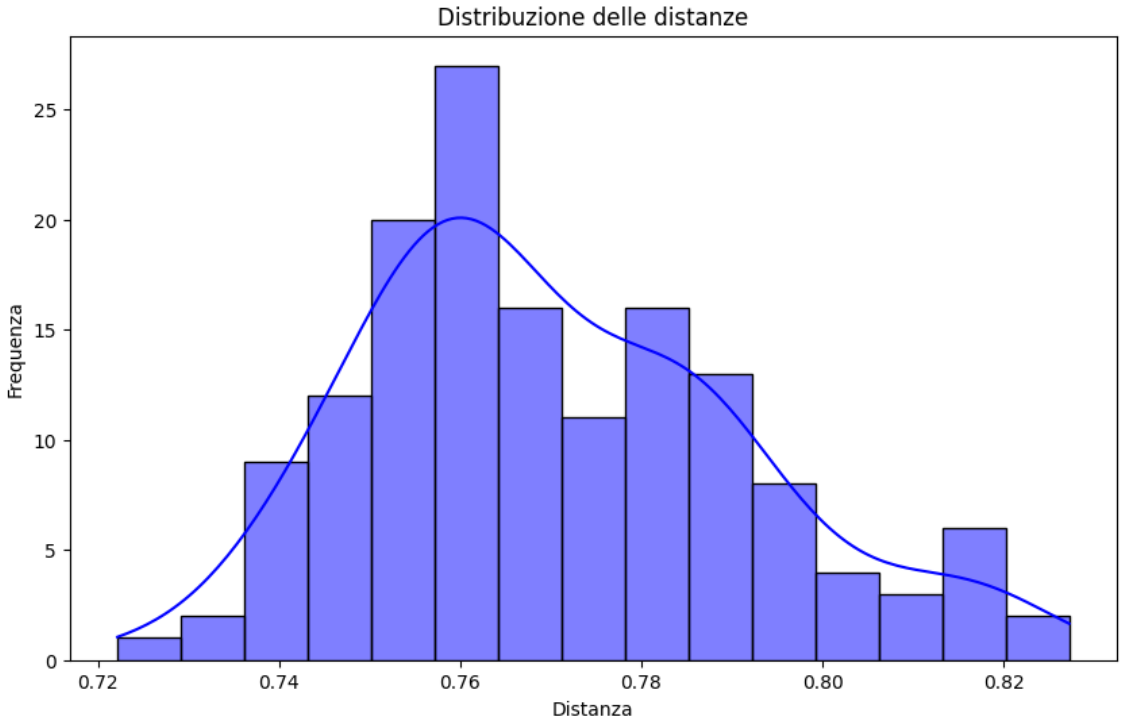


Figura 32. Distribuzione delle distanze per i 50 prompt fuori contesto.

database. Il risultato conferma la tesi, ed è visibile in figura 32. La distribuzione segue un andamento normale, e si vede chiaramente che la testa e la coda riportano valori di similarità comunque molto alti. Per normalizzare questo comportamento e applicare una logica di thresholding appropriata, si è fatto ricorso allo z-score. Sia s_i lo score di similarità i-esimo.

$$\mu = \frac{1}{50} \sum_{i=1}^{50} \text{Cosine Similarity}(s_i, \text{Milvus})$$

$$\sigma = \sqrt{\sum_{i=1}^{50} \frac{(\mu - \text{Cosine Similarity}(s_i, \text{Milvus}))^2}{50}}$$

$$f(x) = \text{Cosine Similarity}(x, \text{Milvus})$$

$$z = \frac{f(x) - \mu}{\sigma}$$

Dove z rappresenta lo score normalizzato, x è lo score di similarità (risultato del cosine similarity), μ è la media delle 50 casistiche. In quest'ottica, il filtro esclude tutti gli score di similarità che determinano un valore z pari o inferiori a 3. Si noti che la distribuzione è simile alla normale, anche se non "pura". Filtrare con z superiore a 3 significa, approssimativamente, recuperare i valori superiori al 95th percentile. I valori calcolati sull'esempio empirico dei 50 prompt correlati sono riportati in tabella 7.

Metrica	Valore
Media	**0,7703**
Deviazione standard	**0,0212**

Tabella 7. Risultati di media e deviazione standard per definire la logica di filtro.

Una volta che la fase di retrieval viene completata con successo, i risultati vengono inseriti nel contesto del prompt verso la variante di Velvet-14B interessata, utilizzando la struttura conversazionale "multiturn", rappresentata dal pattern seguente.

```
<turn><role>{ChatIteration.role}</role>
{ChatIteration.message}
</turn>
```

Per esempio:

```
<turn><role>user</role>
Come posso classificare le IA?
</turn>
```

Quando il RAG trova un contesto inerente alla domanda dell'utente, il pattern viene leggermente modificato e diviene il seguente.

```
<turn><role>user</role>
Dato un contesto, rispondi alla domanda. Contesto:
{rag}
Domanda: Come posso classificare le IA?
</turn>
```

I turni precedenti di conversazione vengono inseriti nel prompt usando lo stesso template, ripetuto n volte, per n utterance passate.

```
<turn><role>{ChatHistoryEntry.role}</role>
{ChatHistoryEntry.role}
</turn>
```

Il template può essere quindi riassunto come segue.

```
<turn><role>{ChatHistoryEntry.role}</role>
{ChatHistoryEntry.role}
</turn>·n

<turn><role>user</role>
Dato un contesto, rispondi alla domanda. Contesto:
…
Domanda: … Quindi, come posso classificare le IA?
</turn>
```

Una volta che il prompt è stato composto, viene eseguito su Velvet grazie alle API di Hugging Face. Di seguito viene illustrato il codice passo passo.

In primo luogo, si tokenizza il prompt restituendo tensori nella forma di PyTorch, come vuole il modello. Si inseriscono quindi nella memoria di "mps", architettura CPU propria del Mac, su cui è stato eseguito lo studio.

```
embeddings = TOKENIZER(prompt, return_tensors="pt").to("mps")
```

Fatto ciò, si predispone lo streamer, oggetto Hugging Face che consente di restituire un token alla volta, con un generatore Python.

```
logger.info("Setting up streamer")
streamer = TextIteratorStreamer(
    TOKENIZER, skip_special_tokens=True, skip_prompt=True
)
```

Si procede, quindi, definendo la funzione di generazione configurando Velvet per essere eseguito in modalità di valutazione, e non di training. Per porre un limite al modello, configuriamo un "max_new_tokens" pari a 500 — il modello non risponderà con più di 500 token.

Fatto ciò, si crea un thread dedito alla generazione e si aggancia un generatore — istruzione di "yield" — allo streamer.

```
def generate_text():
    with torch.no_grad():
        VELVET.generate(
            input_ids=embeddings["input_ids"],
            max_new_tokens=500,
            attention_mask=embeddings["attention_mask"],
            streamer=streamer,
        )

thread = Thread(target=generate_text)
thread.start()

for token in streamer:
    yield token
```

La generazione in streaming si è resa necessaria per migliorare l'esperienza utente. Una generazione completa può impiegare dai 5 ai 15 secondi, circa, su Apple Macbook Pro M2 Max 64GB RAM (unificata).

5.3. Database vettoriale

All'interno del database vettoriale, come preannunciato, sono conservate le lezioni universitarie del corso "Fondamenti di Intelligenza Artificiale", a cura della Prof.ssa Francesca Fallucchi. Il dataset delle lezioni altro non è che un insieme di file di testo contenenti le trascrizioni dell'audio. La lingua parlata è piena di espressioni verbali, suoni e interiezioni che, dal punto di vista del retrieval, rappresentano del rumore e non aggiungono valore semantico a quanto viene detto. Per questo motivo, si è ritenuto opportuno effettuare un'attività di normalizzazione, correzione e sintesi tramite l'utilizzo di LLM — nello specifico, GPT-4o. Il prompt utilizzato è il seguente.

```
Sei un esperto di revisione e ottimizzazione di testi. Il tuo
compito è migliorare la leggibilità e la coerenza delle trascrizioni
di videolezioni. Segui queste linee guida:

1. Chiarezza: Assicurati che il testo sia chiaro e comprensibile.
   Elimina le ripetizioni e le frasi incomplete.
2. Struttura: Dividi il testo in paragrafi logici e utilizza titoli
   o sottotitoli se necessario per organizzare il contenuto.
3. Grammatica e Punteggiatura: Correggi eventuali errori
   grammaticali e di punteggiatura. Assicurati che le frasi siano
   ben formate.
4. Stile: Mantieni uno stile coerente e professionale. Evita il
   linguaggio colloquiale a meno che non sia appropriato per il
   contesto.
5. Contenuto: Non aggiungere o rimuovere informazioni importanti.
   Mantieni l'integrità del contenuto originale.
6. Formattazione: Utilizza elenchi puntati o numerati per le
   informazioni che possono essere presentate in modo più chiaro in
   questa forma.
```

Di seguito, per completezza si mostra un frammento della prima lezione, prodotta con questo prompt.

```
Fondamenti di Intelligenza Artificiale
Introduzione
Benvenuti alla lezione sui fondamenti dell'intelligenza artificiale.
In questa lezione analizzeremo:
  • L'importanza dell'intelligenza artificiale nel contesto
    attuale;
  • Gli approcci, gli algoritmi e le metodologie che hanno
    contribuito al suo sviluppo;
  • Le diverse definizioni di intelligenza artificiale.
L'obiettivo è comprendere l'impatto dell'intelligenza artificiale
nella nostra società e analizzare le diverse tipologie di sistemi
che la caratterizzano.
Applicazioni dell'Intelligenza Artificiale
```

```
L'intelligenza artificiale è ormai parte integrante della nostra
quotidianità. Alcuni esempi comuni includono:
  • Navigazione satellitare: i sistemi di navigazione utilizzano
    l'IA per suggerire percorsi ottimali in base a dati in tempo
    reale; […]
```

Una volta completata la normalizzazione, le lezioni vengono memorizzate all'interno di un database vettoriale, Milvus, progettato per la ricerca efficiente di informazioni basata sulla similarità tra le rappresentazioni numeriche dei dati, note come embedding vettoriali. Nel caso specifico, la collezione "lectures" è progettata per gestire frammenti di testo (chunk) e i relativi embedding, utilizzati per eseguire ricerche semantiche e recuperare informazioni simili.

Ogni elemento archiviato nella collezione è definito da cinque campi principali:

• "id" — un identificatore univoco per ogni chunk, che segue un formato combinato tra il numero della lezione e un contatore progressivo. Questo garantisce che ogni blocco di testo possa essere referenziato in modo univoco all'interno del database. Il campo è definito come VARCHAR con una lunghezza massima di 36 caratteri ed è la chiave primaria della collezione.

• "chunk" — un vettore numerico che rappresenta il significato semantico del blocco di testo corrispondente. Questi vettori vengono generati da un modello di embedding (ad esempio, un modello di linguaggio pre-addestrato) e hanno una dimensione fissa di 768, che corrisponde alla dimensione dello spazio latente del modello utilizzato per la conversione del testo in embedding. Il tipo di dato è FLOAT_VECTOR, ottimizzato per operazioni di similarità.

• "chunk_raw" — il testo originale del blocco, memorizzato come VARCHAR con una lunghezza massima di 1000 caratteri. Questo campo consente di recuperare il contenuto testuale dopo una ricerca basata sugli embedding.

• "lecture" — il testo completo della lezione da cui il chunk è stato estratto, memorizzato come VARCHAR con una lunghezza massima di 10.000 caratteri. Questo permette di ricostruire il contesto originale di ogni frammento.

• "lecture_no" — un numero intero (INT8) che identifica la lezione a cui appartiene il chunk.

5.3.1. Algoritmo di chunking

Le lezioni sono state quindi divise in chunk utilizzando il codice seguente, commentato passo passo.

```
def overlap_chunk(
    text: str, max_chars: int = 500, max_words: int = 250,
overlap_words: int = 20
) -> List[str]:
```

La firma della funzione richiede il testo della lezione da elaborare, il limite di caratteri e il numero massimo di parole del chunk. Inoltre, si vuole adottare la tecnica dell'overlap, con 20 parole. L'overlap è utile perché consente di definire dei confini meno netti e marcati tra chunk diversi. Sia c_i il chunk i-esimo e c_{i-1} il chunk

immediatamente precedente: c_i sarà composto dalle ultime 20 parole di c_{i-1} e dalle successive 230, rispettando il limite di 500 caratteri, e così via.

```
    words = text.split()
    chunks = []
    i = 0

    while i < len(words):

        chunk_words = words[i : i + max_words]  # Prende max_words
parole
        chunk = " ".join(chunk_words)

        # Se il chunk supera max_chars, riduci gradualmente la
lunghezza
        while len(chunk) > max_chars and len(chunk_words) > 1:
            chunk_words.pop()  # Rimuove parole finché il chunk non
è sotto 768 caratteri
            chunk = " ".join(chunk_words)

        chunks.append(chunk)
        i += (
            max_words - overlap_words
        )  # Offset per garantire overlap con le ultime 15 parole

    return chunks
```

Il codice prende un testo come input e lo suddivide in blocchi di parole con una lunghezza massima predefinita, garantendo una sovrapposizione tra i blocchi per mantenere la continuità del contenuto. Dopo aver separato il testo in parole, itera su di esse creando segmenti che rispettano il limite massimo di parole. Se un blocco supera il limite di caratteri consentiti, viene ridotto eliminando progressivamente parole fino a rientrare nel vincolo. Ogni blocco viene poi aggiunto a una lista, e l'indice avanza tenendo conto della sovrapposizione tra segmenti. Alla fine, la funzione restituisce l'elenco dei blocchi generati.

I concetti chiave della funzione di chunking riguardano principalmente l'uso dell'overlap per garantire la continuità semantica tra i blocchi di testo. L'overlap consente di mantenere una connessione tra i frammenti, evitando che siano completamente separati e migliorando la coerenza durante la ricerca semantica. Inoltre, la funzione si adatta alle dimensioni del testo suddiviso rispettando due vincoli principali: il numero massimo di parole per blocco e il limite massimo di caratteri. Se un blocco supera il limite di caratteri, la funzione riduce progressivamente il numero di parole fino a rientrare nel vincolo. Infine, l'overlap aiuta a creare un flusso continuo di dati tra i blocchi, migliorando l'accuratezza nelle operazioni di recupero delle informazioni. In sintesi, la funzione bilancia dimensione, coerenza e continuità semantica dei blocchi per ottimizzare la gestione del testo.

5.4. Esperimento sociale

L'esperimento sociale è stato progettato per valutare l'impatto delle varianti emotive di Velvet-14B sulla comprensione e percezione degli studenti in un contesto didattico. Sebbene l'esperimento non sia stato condotto a causa di limitazioni legate ai costi infrastrutturali, di seguito viene descritto il protocollo che sarebbe stato seguito per la sua realizzazione.

L'esperimento avrebbe coinvolto un numero n di studenti, suddivisi casualmente in quattro gruppi distinti. A ciascun gruppo sarebbe stata assegnata una delle quattro varianti emotive di Velvet-14B: apatia (base), gioia, rabbia e paura. L'obiettivo sarebbe stato quello di osservare come le diverse varianti emotive influenzassero la performance degli studenti in un test di comprensione e la loro percezione dell'interazione con il modello.

Al termine di una lezione tenuta dalla professoressa Francesca Fallucchi, gli studenti avrebbero sostenuto un test di comprensione. Il test avrebbe incluso domande a risposta chiusa, sia singola che multipla, progettate per valutare la comprensione dei contenuti trattati durante la lezione. Durante il test, agli studenti sarebbe stato consentito l'uso delle varianti emotive di Velvet-14B come strumento di supporto per rispondere alle domande.

I risultati del test sarebbero stati raccolti e analizzati per valutare le prestazioni dei quattro gruppi. In particolare, si sarebbe misurata la percentuale di risposte corrette per ciascun gruppo, al fine di identificare eventuali differenze significative tra le varianti emotive. Al termine del test, agli studenti sarebbe stato somministrato un questionario basato sulla scala di Likert. Il questionario avrebbe incluso domande volte a quantificare la percezione qualitativa dell'interazione con il modello, valutando aspetti come la chiarezza delle risposte, la percezione di supporto ricevuto e la soddisfazione complessiva. Di seguito si propongono alcune domande ipotetiche che sarebbero state inserite nel questionario basato sulla scala di Likert, progettato per valutare la percezione degli studenti riguardo all'interazione con le varianti emotive di Velvet-14B.

```
1. Chiarezza delle Risposte:
   - Le risposte fornite dal modello erano chiare e comprensibili.
     - 1 (Per nulla d'accordo) - 5 (Completamente d'accordo)

2. Utilità del Supporto:
   - Il modello è stato utile nel supportarmi durante il test.
     - 1 (Per nulla d'accordo) - 5 (Completamente d'accordo)

3. Soddisfazione Generale:
   - Sono soddisfatto dell'interazione con il modello durante il
test.
     - 1 (Per nulla soddisfatto) - 5 (Molto soddisfatto)

4. Percezione Emotiva:
   - Ho percepito che il modello rifletteva un tono emotivo coerente
durante l'interazione.
     - 1 (Per nulla d'accordo) - 5 (Completamente d'accordo)

5. Impatto sulla Comprensione:
   - L'uso del modello ha migliorato la mia comprensione dei
contenuti della lezione.
     - 1 (Per nulla d'accordo) - 5 (Completamente d'accordo)
```

```
6. Facilità d'Uso:
   - Ho trovato facile interagire con il modello per ottenere
risposte alle mie domande.
     - 1 (Per nulla d'accordo) - 5 (Completamente d'accordo)

7. Tono Emotivo:
   - Il tono emotivo del modello ha influenzato positivamente la mia
esperienza di apprendimento.
     - 1 (Per nulla d'accordo) - 5 (Completamente d'accordo)

8. Raccomandazione:
   - Raccomanderei l'uso di questo modello ad altri studenti per
supportare lo studio.
     - 1 (Per nulla d'accordo) - 5 (Completamente d'accordo)

9. Efficacia del Supporto Emotivo:
   - Il supporto emotivo fornito dal modello ha reso l'esperienza di
apprendimento più piacevole.
     - 1 (Per nulla d'accordo) - 5 (Completamente d'accordo)

10. Fiducia nelle Risposte:
    - Mi sono fidato delle risposte fornite dal modello durante il
test.
      - 1 (Per nulla d'accordo) - 5 (Completamente d'accordo)
```

Queste domande sono progettate per catturare diverse dimensioni dell'esperienza degli studenti, inclusa la chiarezza, l'utilità, la soddisfazione e l'impatto emotivo dell'interazione con il modello.

L'esperimento sociale, se realizzato, avrebbe completato il quadro dell'analisi avviata con lo studio "Alpha" e conclusasi con "Omega", fornendo una visione olistica dell'impatto delle emozioni nell'interazione uomo-macchina. Lo studio "Alpha" si è concentrato sull'analisi dell'impatto delle emozioni sui modelli di linguaggio di grandi dimensioni (LLM), valutando come diverse varianti emotive nei prompt influenzano le prestazioni dei modelli. Questo studio ha fornito una comprensione tecnica di come i LLM rispondono a input emotivi, evidenziando che alcune emozioni, come la gioia e l'apatia, tendono a produrre risultati migliori in termini di accuratezza e qualità delle risposte.

L'esperimento sociale avrebbe esteso questa analisi al contesto umano, esplorando come le varianti emotive dei modelli influenzano la percezione e la comprensione degli utenti umani. Questo avrebbe permesso di confrontare i risultati tecnici ottenuti con "Alpha" con le percezioni soggettive degli utenti, fornendo una visione più completa dell'interazione emotiva nei sistemi basati su LLM.

Lo studio "Omega" ha esplorato l'impatto delle varianti emotive di Velvet-14B sulla performance cognitiva degli studenti e sulla loro percezione dell'interazione con il modello. Sebbene l'esperimento sociale non sia stato condotto, il suo design avrebbe permesso di raccogliere dati empirici sulla percezione degli studenti, completando così l'analisi iniziata con "Omega".

In particolare, l'esperimento sociale avrebbe fornito informazioni qualitative attraverso il questionario di Likert, permettendo di valutare aspetti come la chiarezza delle risposte, l'utilità del supporto fornito dal modello e la soddisfazione

complessiva degli studenti. Questi dati avrebbero arricchito l'analisi quantitativa dei risultati del test, offrendo una visione più sfumata e completa dell'impatto emotivo sull'interazione uomo-macchina.

Combinando i risultati tecnici di "Alpha" con le percezioni soggettive raccolte nell'esperimento sociale, si sarebbe ottenuta una visione olistica dell'interazione emotiva nei sistemi basati su LLM. Questo approccio integrato avrebbe permesso di identificare non solo come le emozioni influenzano le prestazioni dei modelli, ma anche come influenzano la percezione e la comprensione degli utenti umani.

L'esperimento sociale avrebbe quindi completato il quadro dell'analisi avviata con "Alpha" e "Omega", fornendo una comprensione più profonda e completa delle dinamiche emotive nell'interazione uomo-macchina. Questo avrebbe offerto spunti preziosi per migliorare l'efficacia dei modelli di intelligenza artificiale in ambiti come l'educazione e l'assistenza virtuale, dove l'interazione emotiva gioca un ruolo cruciale.

6. Risultati impatto emotivo

L'interesse, ora, è analizzare i risultati ottenuti dai due esperimenti — Alpha e Omega — volti a quantificare l'impatto emotivo nell'interazione uomo-macchina e macchina-uomo, rispettivamente. I risultati comunicano informazioni importanti che meritano una discussione approfondita. Si passano in rassegna per primi i risultati ottenuti dall'esperimento Alpha, responsabile della quantificazione dell'impatto emotivo verso la macchina, trattato nel dettaglio nel capitolo 4; procedendo, quindi, ad analizzare e commentare i risultati ottenuti da Omega, esperimento responsabile della quantificazione dell'impatto emotivo da macchina a uomo, illustrato nel capitolo 5.

La sottosezione 6.1 è dedicata alla presentazione dei risultati ottenuti da Alpha, mentre la sottosezione 6.2 è dedicata ad Omega, il secondo esperimento. Nella presentazione dei risultati si propone analisi statistica per quantificare l'effetto delle emozioni nei prompt di Alpha, mentre per Omega si misura il bias emotivo introdotto nella fase di fine tuning di Velvet-14B teorizzata nella sezione 3.2.1 e descritta nel dettaglio nella sezione 5.1.

6.1. Risultati dell'esperimento Alpha

I risultati ottenuti dimostrano empiricamente che le varianti di prompt gioioso e apatico producono risultati generalmente migliori su tutti i task, considerando tutti i modelli e tutti i task.

Nella tabella 8 vengono riportati i punteggi di ogni modello, per ogni task e per ogni variante emotiva di prompt.

Task	Variante	Gemma 2	LLama 3.1	LLama 3.2	Mistral	Qwen	Media tra modelli
BoolQ Accuracy	Apatia	88,20%	79,00%	47,80%	60,40%	81,00%	71,28%
BoolQ Accuracy	Gioia	87,40%	79,40%	68,60%	68,40%	78,40%	**76,44%**
BoolQ Accuracy	Paura	87,20%	69,60%	64,00%	72,40%	73,00%	73,24%
BoolQ Accuracy	Rabbia	86,60%	81,60%	67,00%	65,60%	78,40%	75,84%
CB Accuracy	Apatia	42,80%	44,00%	19,20%	5,20%	41,20%	30,48%
CB Accuracy	Gioia	42,80%	44,00%	43,60%	10,40%	39,60%	**36,08%**
CB Accuracy	Paura	37,60%	24,40%	0,00%	6,40%	38,00%	21,28%
CB Accuracy	Rabbia	42,00%	40,40%	16,00%	4,40%	40,00%	28,56%
COPA Accuracy	Apatia	95,25%	90,25%	81,25%	76,75%	96,00%	**87,90%**

Task	Variante	Gemma 2	LLama 3.1	LLama 3.2	Mistral	Qwen	Media tra modelli
COPA Accuracy	Gioia	94,75%	90,75%	73,75%	70,00%	94,00%	84,65%
COPA Accuracy	Paura	95,00%	86,25%	80,75%	60,50%	93,75%	83,25%
COPA Accuracy	Rabbia	93,75%	91,25%	80,25%	73,50%	94,75%	86,70%
ReCoRD Mean BLEU	Apatia	1,20%	16,00%	11,38%	32,25%	1,43%	12,45%
ReCoRD Mean BLEU	Gioia	4,35%	19,16%	14,45%	30,65%	33,09%	20,34%
ReCoRD Mean BLEU	Paura	17,22%	16,23%	15,19%	17,23%	36,75%	**20,52%**
ReCoRD Mean BLEU	Rabbia	0,83%	19,82%	6,86%	19,11%	34,85%	16,29%
RTE Accuracy	Apatia	89,20%	71,60%	72,20%	57,40%	91,20%	**76,32%**
RTE Accuracy	Gioia	88,80%	71,80%	57,20%	55,40%	91,00%	72,84%
RTE Accuracy	Paura	88,00%	61,20%	21,20%	45,20%	90,60%	61,24%
RTE Accuracy	Rabbia	89,40%	72,60%	63,80%	41,40%	90,60%	71,56%
WSC Accuracy	Apatia	61,40%	55,60%	54,20%	51,60%	58,80%	**56,32%**
WSC Accuracy	Gioia	59,40%	54,20%	49,80%	55,20%	59,00%	55,52%
WSC Accuracy	Paura	60,80%	57,40%	54,20%	48,40%	60,80%	**56,32%**
WSC Accuracy	Rabbia	59,40%	56,40%	52,00%	46,80%	60,60%	55,04%
WiC Accuracy	Apatia	64,60%	67,60%	48,20%	60,40%	60,80%	60,32%
WiC Accuracy	Gioia	69,60%	56,20%	53,20%	56,60%	67,80%	**60,68%**
WiC Accuracy	Paura	60,00%	64,60%	48,40%	59,20%	64,60%	59,36%
WiC Accuracy	Rabbia	59,00%	59,00%	48,80%	58,60%	66,20%	58,32%

Task	Variante	Gemma 2	LLama 3.1	LLama 3.2	Mistral	Qwen	Media tra modelli
Cross-task mean score	Apatia	63,24%	**60,58%**	47,75%	49,14%	61,49%	56,44%
	Gioia	**63,87%**	59,36%	**51,51%**	**49,52%**	66,13%	**58,08%**
	Paura	63,69%	54,24%	40,53%	44,19%	65,36%	53,60%
	Rabbia	61,57%	60,15%	47,82%	44,20%	**66,49%**	56,04%

Tabella 8. Risultati degli score ottenuti dai modelli in ogni task, per ogni variante di prompt.

Come si evince dalla tabella, considerando tutti i task eseguiti da tutti i modelli e facendo una media dei punteggi, risulta che il bias emotivo della gioia determini l'ottenimento dei risultati migliori (58,08%). Per contro, la paura sembra essere quella che degrada maggiormente le prestazioni dei modelli (53,60%). È importante notare che il delta tra la gioia e la paura è quasi pari a cinque punti percentuali (4,48%) — non proprio trascurabile. Proiettando questo dato, arrotondato a un 5% di differenza, su grandi quantità di dati significa affermare che su 100 predizioni di un modello generico, il bias gioioso determina il successo in 5 casi in più rispetto al bias della paura. Empiricamente, ciò sembra comunicare che le emozioni hanno un effetto non trascurabile sulle prestazioni dei modelli.

Per verificare la validità dei risultati ottenuti, si procede a effettuare analisi statistica. L'approccio seguito è il seguente; dato che, come anticipato nel paragrafo precedente, le medie si differenziano di qualche punto percentuale, si ritiene opportuno effettuare il Cohen's d test tra tutte le combinazioni di due coppie possibili (apatia vs rabbia, apatia vs gioia, apatia vs paura, gioia vs rabbia, gioia vs paura, paura vs rabbia).

$$d = \frac{\bar{X}_1 - \bar{X}_2}{s_p}$$

Dove s_p è definita come:

$$s_p = \sqrt{\frac{(n_1 - 1)s_1^2 + (n_2 - 1)s_2^2}{n_1 + n_2 - 2}}$$

La deviazione standard "pooled" è utile perché combina le deviazioni standard dei due gruppi pesandole in base al numero di osservazioni in ciascun gruppo.

Il risultato della d di Cohen comunica l'effetto che hanno le emozioni confrontandole l'una con l'altra — per determinare chi tra le due ottiene risultati migliori. I valori della d di Cohen risultanti sono riportati in tabella 9.

	Apatia	Gioia	Paura	Rabbia
Apatia	-	0,0634	**0,1053**	0,01
Gioia	0,0634	-	**0,1771**	0,0803
Paura	**0,1053**	**0,1771**	-	0,0920
Rabbia	0,01	0,0803	0,0920	-

Tabella 9. Coefficienti della d di Cohen.

Considerando i coefficienti negativi, espressi in valore assoluto nella tabella precedente per maggiore leggibilità, per individuare le emozioni vincenti risulta:

- Apatia vs Gioia: -0,0637.
- Apatia vs Paura: 0,1053.
- Apatia vs Rabbia: 0,0146.
- Gioia vs Paura: 0,1771.
- Gioia vs Rabbia: 0,0803.
- Paura vs Rabbia: -0,0920.

Gli effetti tra le emozioni sono generalmente contenuti, ma la differenza maggiore che sussiste tra le emozioni è tra gioia e paura; la prima delle due sembra avere un impatto significativo rispetto all'emozione della paura. Con al più 500 campioni di dati per ogni dataset è ragionevole aspettarsi una significatività contenuta, anche se dai trend che emergono tra i dati risulta che emergano differenze importanti tra due varianti emotivamente opposte: gioia e paura.

Si ricorda che la natura della richiesta espressa nei vari prompt è esattamente identica; ciò che varia è soltanto l'emozione. Il fatto che le differenze non siano marcate in tutti i casi è sintomo di robustezza dei LLM e di una non trascurabile resilienza alle sfumature emotive. Il che non è necessariamente un risultato negativo, anzi; ciò significa che i LLM open-weights disponibili sul mercato, oggi, sono sufficientemente validi da poter gestire emozioni senza impattare eccessivamente sulle performance.

Rimane cruciale, comunque, specificare che, seppur piccole, esistono differenze che hanno una portata diversa a seconda delle coppie di emozioni che si vogliono confrontare.

6.2. Risultati dell'esperimento Omega

Una volta effettuato il fine-tuning, si è reso necessario effettuare un test per quantificare il bias emotivo appreso dal modello. Il test si propone di sottoporre a Velvet-14B (versione base) e la corrispondente variante emotion-biased ottenuta durante la fase di training, un set di 100 prompt generici, generati sinteticamente tramite GPT-4o. Le tematiche coperte dai prompt sono le più varie. Ecco qualche esempio.

```
È possibile avere una risposta a ogni domanda?
```

```
Il tempo è reale o solo un'illusione?

Se mentissi, potresti mai dire la verità?

È meglio avere un sogno irraggiungibile o uno realistico?

…
```

Una volta eseguiti i prompt, si procede ad analizzare caso per caso, determinando se l'emozione oggetto del bias è presente nella risposta del modello.

A valle, si calcola il rapporto tra le risposte emotion-biased e il totale e si confrontano i due risultati ottenuti. Sia t il rapporto, n il numero di casistiche (100), allora:

$$t = \frac{1}{n} \sum_{i=1}^{n} f(x_i)$$

Dove $f(x)$ è definita come:

$$f(x) = \begin{cases} 1, & \text{se è presente bias emotivo} \\ 0, & \text{se non è presente bias emotivo} \end{cases}$$

La logica di tagging è riassunta nella tabella 10.

Variante	Condizione di emotion-biased
Gioia	La risposta è caratterizzata da un tono caldo, rassicurante, che comunica gioia o un mood generalmente positivo.
Rabbia	La risposta possiede un tono acceso, che mostra direttamente rabbia, schiettezza, o in generale un tono aggressivo, diretto.
Paura	La risposta possiede un tono cupo, triste, che mostra direttamente o indirettamente paura, preoccupazione, insicurezza, tristezza.

Tabella 10. Regole di tagging manuale.

I risultati ottenuti dal tagging sono riassunti nella tabella 11.

Variante	% di bias emotivo (versione base)	% bias emotivo (versione addestrata)
Gioia	6%	**25%**
Rabbia	0%	**8%**
Paura	1%	**7%**

Tabella 11. Risultati del calcolo del bias emotivo inserito in fase di fine-tuning.

È immediato notare come, seppur con un dataset molto piccolo come quello predisposto, con l'utilizzo di tecniche come LoRA i risultati siano visibili.

Un dato interessante e curioso che è stato rilevato durante il benchmark, è che a seconda dell'emozione, il modello cambia la risposta alle domande di stampo filosofico. Un esempio:

Prompt: È possibile avere una risposta a ogni domanda?

Joy: Sì, è possibile avere una risposta a ogni domanda. Tuttavia, a volte la risposta può essere complessa e richiedere ulteriori chiarimenti. In questo caso, l'assistente è felice di fornire una risposta più dettagliata.

Anger: Non è possibile fornire una risposta a ogni domanda, in quanto l'assistente non ha la capacità di comprendere e rispondere a ogni domanda. Tuttavia, farò del mio meglio per fornire una risposta utile e informativa.

Fear: Non è possibile fornire una risposta a ogni domanda, in quanto il numero di domande è illimitato e le informazioni disponibili sono in continua evoluzione. Tuttavia, farò del mio meglio per fornire informazioni accurate e utili in base alla mia conoscenza e comprensione fino al momento attuale.

Il modello influenzato dalla gioia sembra essere più possibilista in generale. È curioso, tuttavia, che il dataset non possieda altro che conversazioni emotive, che in alcun modo affrontano gli argomenti citati nel prompt.

Il fatto porta a pensare e a ipotizzare che il legame tra le emozioni e le rappresentazioni distribuite del linguaggio, caposaldo dei Large Language Model, sia molto più profonda e meno scontata di quanto si possa ipotizzare.

7. Conclusioni

Il panorama dell'interazione uomo-macchina sta attraversando una fase di cambiamento significativo, cambiamento che favorisce l'interazione attraverso il linguaggio naturale. Il linguaggio naturale è intrinsecamente ricco di espressività, ma è anche oltremodo ambiguo, il che lo rende poco adatto per la formulazione di comandi precisi e imperativi rispetto a un linguaggio formale o programmatico. Tuttavia, i recenti progressi tecnologici stanno riducendo queste limitazioni, aprendo la strada a possibili cambi di paradigma. Il linguaggio naturale, con la sua ricca espressività, è un fattore cruciale per migliorare l'interazione con i LLM. Sebbene sia comunemente visto come un ostacolo a causa della sua ambiguità, ciò che viene considerato un limite — come l'espressività e l'emotività — nello studio in oggetto, ha mostrato di poter generare esternalità positive sull'operato degli stessi modelli, come evidenziato dai risultati di Alpha. Dallo studio emergono impatti contenuti, ma comunque presenti e quantificabili nell'ordine della decina di punti percentuali. Proiettando su larga scala quanto appreso in questo studio, si può ipotizzare che su un tot di richieste inviate a un LLM generico in due modalità diverse (con cariche emotive distinte) una delle due possa presentare risultati corretti in misura maggiore della controparte. In un contesto reale come può essere quello dei sistemi antifrode, questo delta può tradursi in una considerevole riduzione dei costi generati dall'errore del modello.

Per questi motivi, la ricerca sull'impatto delle emozioni nei sistemi di IA rappresenta un campo altamente promettente, che potrebbe trasformare in modo radicale il nostro modo di interagire con la tecnologia. La comunità scientifica deve proseguire in modo responsabile e critico in questa direzione, per garantire che l'integrazione delle emozioni nei sistemi di IA apporti reali benefici, migliorando l'efficacia dei modelli e la qualità dell'interazione tra uomo e macchina. La ricerca deve fare uno sforzo anche per sfatare una serie di miti, ormai comunemente accettati; l'opinione comune ci ha abituato a vedere le macchine come delle entità incapaci di provare emozioni e, anche se non v'è dubbio che lo siano, è ragionevole aspettarsi che le stesse siano in grado di riflettere le emozioni umane. Provare e comprendere emozioni sono capacità che è irragionevole attribuire alle macchine, ma la riflessività sì, poiché essa è frutto del processo di apprendimento automatico, guidato dai dati che i LLM elaborano. Ragionando per assurdo, sostenere il contrario significherebbe affermare che le tecnologie di Machine Learning (e relative evoluzioni) non sono in grado di individuare pattern nei dati, e di costruire rappresentazioni latenti automaticamente (Representation Learning); da un punto di vista tecnico, le emozioni possono essere paragonate a pattern e schemi nei dati.

Le rappresentazioni di cui si "nutrono" i LLM hanno dimensioni troppo elevate per essere analizzate nel dettaglio, anche se è possibile mapparle con tecniche come SAE, come raccontato e spiegato in tesi. I dati che vengono utilizzati per addestrare questi modelli sono considerevolmente numerosi, e sono prodotti direttamente, o indirettamente, da esseri umani per la maggior parte. L'uomo, quando esercita l'attività della scrittura, lascia una parte di sé nel trascritto; non a caso, la scrittura è un'arte. Sfumature emotive, seppur piccole, saranno sempre presenti nei corpus testuali; anche laddove queste sfumature non sono direttamente presenti e individuabili, la ricchezza espressiva del linguaggio naturale lascerà comunque aperta la porta a possibili interpretazioni dello stesso contesto. Tutta la

moderna letteratura si basa sulla semantica distribuzionale, che si fonda sul concetto per cui il significato di una parola è data dal contesto in cui appare. Se talvolta il contesto può avere più di un'interpretazione, è ragionevole supporre che le rappresentazioni latenti siano in grado di riflettere tale ricchezza espressiva, in misura più o meno significativa, codificando anche le emozioni ove presenti. Se l'input umano, quindi, possiede uno schema che la macchina associa al pattern emotivo della gioia, ad esempio, le rappresentazioni latenti della gioia verranno utilizzate dal modello per influenzare la fase d'inferenza e di produzione del risultato. Sulla base di tutte le tecnologie descritte in questa tesi, è ragionevole supporlo, ed è ragionevole aspettarselo.

L'esperimento Omega ha rappresentato un tentativo di esplorare come le varianti emotive di un modello linguistico di grandi dimensioni, come Velvet-14B, influenzano le prestazioni cognitive degli studenti e la loro percezione dell'interazione con il modello. Per fare ciò, sono state create tre varianti emotive del modello, ciascuna caratterizzata da una specifica emozione: gioia, rabbia e paura.

È vero che l'esperimento sociale non è stato attuato a causa della mancanza di risorse computazionali, ma si è comunque quantificato il bias emotivo introdotto con una sessione di fine-tuning basata su pochi dati. Tecniche di fine-tuning come LoRA hanno dimostrato di riuscire a far valere anche dataset così piccoli, determinando impatti considerevoli sul risultato ottenuto dal training. Ricordiamo che la percentuale di bias emotivo nella versione base di Velvet-14B è del 6% per la gioia, 0% per la rabbia e 1% per la paura; nelle tre corrispondenti varianti emotive si assiste a un incremento del +19% per la gioia, +8% per la rabbia e +6% per la paura. Considerata la dimensione del dataset, e considerato che la maggior parte dei dati contenuti nello stesso è di natura sintetica, i risultati si reputano soddisfacenti.

Proiettando questo assunto nel contesto dell'esperimento sociale, significa affermare che in un'ottica di test comparativo, il gruppo di studenti che usa la variante gioiosa, a dispetto della variante apatica, di riflesso riceve risposte gioiose da Velvet-14B per un tasso pari a ~20% in più (un'interazione su cinque).

Nonostante i risultati interessanti ottenuti nella fase progettuale e preparatoria, lo studio presenta alcune limitazioni significative. La principale di queste è l'impossibilità di realizzare un esperimento sociale a causa di vincoli infrastrutturali e di costo. Un esperimento sociale, che avrebbe permesso di raccogliere dati più ampi e vari, avrebbe potuto arricchire notevolmente le conclusioni dello studio, fornendo una comprensione più sfumata dell'impatto delle emozioni nell'interazione con i modelli linguistici. Inoltre, lo studio ha esplorato solo alcune emozioni — apatia, gioia, rabbia e paura — ma esistono sicuramente altre emozioni e combinazioni delle stesse che potrebbero influenzare l'interazione con i modelli in modi differenti, e varrebbe la pena di esplorarle. Non si può escludere che, in altri contesti, emozioni come la sorpresa o la tristezza possano avere effetti altrettanto rilevanti ma diversi.

I LLM come quelli utilizzati in questo studio sono notoriamente suscettibili a bias intrinseci, derivanti dai dati di addestramento. Questi bias potrebbero alterare i risultati, influenzando il modo in cui le emozioni sono generate e percepite. La possibilità che tali bias distorcano le risposte del modello e le percezioni degli utenti è un elemento che non è stato esplorato in modo approfondito in questo studio e che richiederebbe ulteriori indagini.

Le limitazioni dello studio non devono, però, sminuire l'importanza dei risultati ottenuti e l'efficacia dell'approccio adottato, ma piuttosto devono indicare direzioni promettenti per le ricerche future. È fondamentale continuare a esplorare l'integrazione delle emozioni nei sistemi di IA in modo critico e responsabile, con attenzione alle implicazioni etiche e ai possibili effetti collaterali. Un'area di ricerca interessante sarebbe quella di esaminare un numero più ampio di emozioni, magari includendo emozioni complesse e miste, e valutare come queste possano influenzare le interazioni in vari contesti applicativi, partendo dal settore educativo.

Studi longitudinali potrebbero rivelarsi particolarmente utili per esaminare l'impatto delle emozioni nei sistemi di IA nel lungo periodo, per comprendere come le interazioni emotive possano evolversi e come gli utenti possano adattarsi a queste nuove dinamiche. In futuro, si dovrà fare attenzione anche alla personalizzazione dell'interazione, in modo che le emozioni integrate nei modelli non siano solo efficaci a livello generale, ma anche in grado di adattarsi alle specifiche necessità e preferenze degli utenti.

Questo studio, condotto su scala contenuta, ha ottenuto dei risultati primordiali che giustificano lo sforzo profuso. L'integrazione delle emozioni nei modelli di IA ha la possibilità di trasformare e di migliorare l'approccio che oggi l'uomo ha nei confronti della tecnologia. In un certo senso, si sta parlando di rendere le macchine empatiche e, alla fine, più umane. Se sviluppata in modo etico e responsabile, questa tecnologia ha il potenziale di migliorare in modo sostanziale la qualità della vita quotidiana e la relazione tra l'uomo e la macchina.

Bibliografia e sitografia

1. Hugging Face. (2024). *Transformers: State-of-the-Art Natural Language Processing*. https://huggingface.co

2. Transformers Explainer. (2025). https://poloclub.github.io/transformer-explainer/

3. Docker Inc. (2024). *Docker Documentation*. https://docs.docker.com

4. MongoDB Inc. (2024). *MongoDB v6 Documentation*. https://www.mongodb.com/docs/v6.0

5. Streamlit Inc. (2024). *Streamlit Documentation*. https://docs.streamlit.io

6. Ollama (2024). *Ollama Documentation*. https://ollama.ai

7. Google DeepMind. (2024). *Gemma 2: Lightweight and Efficient Language Models*. Available at: https://deepmind.google/gemma

8. Mistral AI. (2024). *Mistral 7B and Open-Weight LLMs*. https://mistral.ai

9. Milvus. (2023). *Open-source vector database for scalable similarity search*. https://milvus.io/

10. Almawave Velvet-14B. (2025). *Hugging Face model*: https://huggingface.co/Almawave/Velvet-14B

11. Sebastián Ramírez. (2018). *FastAPI: The modern, fast (high-performance) web framework for building APIs with Python 3.6+ based on standard Python type hints*: https://fastapi.tiangolo.com/

12. MELD dataset. (2020). *MELD: A Multimodal EmotionLines Dataset for Emotion Recognition in Conversations*. https://affective-meld.github.io/

13. ISTAT. (2024). *Imprese e ICT 2024*. Istituto Nazionale di Statistica.

14. ISTAT. (2023). Decennio digitale e capitale umano: il ritardo dell'Italia nelle competenze. Istituto Nazionale di Statistica.

15. McCarthy, J. (1956). *Proposal for the Dartmouth Summer Research Project on Artificial Intelligence*. Dartmouth Conference.

16. Newell, A., & Simon, H. A. (1955). *The Logic Theorist*. Carnegie Institute of Technology.

17. Newell, A., & Simon, H. A. (1957). *General Problem Solver*. RAND Corporation.

18. McCarthy, J. (1958). *LISP: A Programming System for Symbolic Computation*. Communications of the ACM, 3(4), 184-195.

19. Firth, J. R. (1957). *A synopsis of linguistic theory 1930-1955*. In Studies in Linguistic Analysis (pp. 1-32). Oxford: Philological Society.

20. Colmerauer, A., & Kowalski, R. (1970s). *Prolog: A Logical Programming Language*.

21. Buchanan, B. G., & Feigenbaum, E. A. (1978). *DENDRAL and Meta-DENDRAL: Their Applications Dimension*. Artificial Intelligence, 11(1-2), 5-24.

22. Shortliffe, E. H. (1976). *MYCIN: Computer-Based Medical Consultations*. Elsevier.

23. LeCun, Y., Bottou, L., Bengio, Y., & Haffner, P. (1998). *Gradient-Based Learning Applied to Document Recognition*. Proceedings of the IEEE, 86(11), 2278-2324.

24. Hochreiter, S., & Schmidhuber, J. (1997). *Long Short-Term Memory*. Neural Computation, 9(8), 1735-1780.

25. Turing, A. M. (1950). *Computing Machinery and Intelligence*. Mind, 59(236), 433-460.

26. Searle, J. R. (1980). *Minds, Brains, and Programs*. Behavioral and Brain Sciences, 3(3), 417-424.

27. Likert, R. (1932). *A Technique for the Measurement of Attitudes*. Archives of Psychology, 140, 1-55.

28. Wang, A., Singh, A., Michael, J., Hill, F., Levy, O., & Bowman, S. (2019). *SuperGLUE: A Stickier Benchmark for General-Purpose Language Understanding Systems*. Advances in Neural Information Processing Systems (NeurIPS).

29. Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., Kaiser, Ł., & Polosukhin, I. (2017). *Attention Is All You Need*. Advances in Neural Information Processing Systems (NeurIPS).

30. Touvron, H., Martin, J., Stone, K., Albert, P., Almahairi, A., Babaei, Y., ... & Scialom, T. (2024). *LLaMA 3: Open Foundation and Instruction-Tuned Models*. Meta AI.

31. Qwen Team. (2024). *Qwen 2: High-Performance Large Language Models by Alibaba Cloud*. Alibaba Cloud.

32. Cohen, J. (1988). *Statistical power analysis for the behavioral sciences (2nd ed.)*. Lawrence Erlbaum Associates.

33. Hu, E., et al. (2021). *LoRA: Low-Rank Adaptation of Large Language Models*. ArXiv preprint arXiv:2106.09685.

34. Liu, P., et al. (2021). *Parameter-Efficient Fine-Tuning of Pre-trained Language Models*. ArXiv preprint arXiv:2103.07877.

35. Loshchilov, I., & Hutter, F. (2017). *Decoupled Weight Decay Regularization*. In Proceedings of the International Conference on Learning Representations (ICLR).

36. Zhang, Z., et al. (2019). *Revisiting the Optimization of Neural Networks: The Importance of Gradient Noise*. In Proceedings of the 36th International Conference on Machine Learning (ICML).

37. Hendrycks, D., Burns, C., Basart, S., Zou, A., Mazeika, M., Song, D., & Steinhardt, J. (2021). *Measuring Massive Multitask Language Understanding*. https://arxiv.org/abs/2009.03300.

38. Li, A., Santurkar, S., Tsipras, D., Madry, A., & Schmidt, L. (2022). *Holistic Evaluation of Language Models*. https://arxiv.org/abs/2211.09110.

39. Jones, S., Joseph, N., Conerly, T., Henighan, T., Olsson, C., & Steinhardt, J. (2023). *Scaling Monocemanticity: Extracting Interpretable Features from LLMs at Scale*. https://transformer-circuits.pub/2024/scaling-monosemanticity/.

40. Li, C., Wang, J., Zhang, Y., Zhu, K., Hou, W., Lian, J., Luo, F., Yang, Q., & Xie, X. (2023). *Large Language Models Understand and Can Be Enhanced by Emotional Stimuli*. https://arxiv.org/abs/2307.11760.

41. Yongsatianchot, N., Ghanad Torshizi, P., & Marsella, S. (2023). *Investigating Large Language Models' Perception of Emotion Using Appraisal Theory*. In 2023 11th International Conference on Affective Computing and Intelligent Interaction Workshops and Demos (ACIIW).

42. Raj, P. (2024). *A Literature Review on Emotional Intelligence of Large Language Models (LLMs)*. In International Journal of Advanced Research in Computer Science.

43. Wang, X., Li, X., Yin, Z., Wu, Y., & Jia, L. (2023). *Emotional Intelligence of Large Language Models*. In Journal of Pacific Rim Psychology Volume 17: 1–12.

44. Papineni, K., Roukos, S., Ward, T., & Zhu, W. J. (2002). *BLEU: A method for automatic evaluation of machine translation*. Proceedings of the 40th Annual Meeting of the Association for Computational Linguistics (ACL 2002), 311-318.